Chapter 3
The Data Link Layer

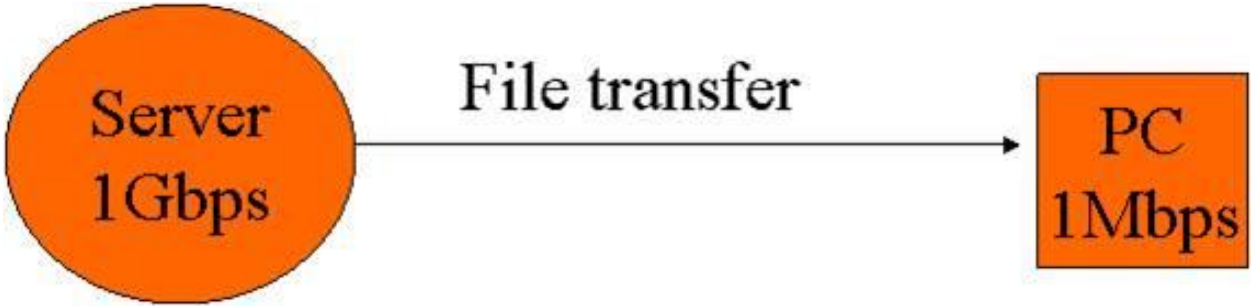
(8-9 hours' Lecture)

Outline

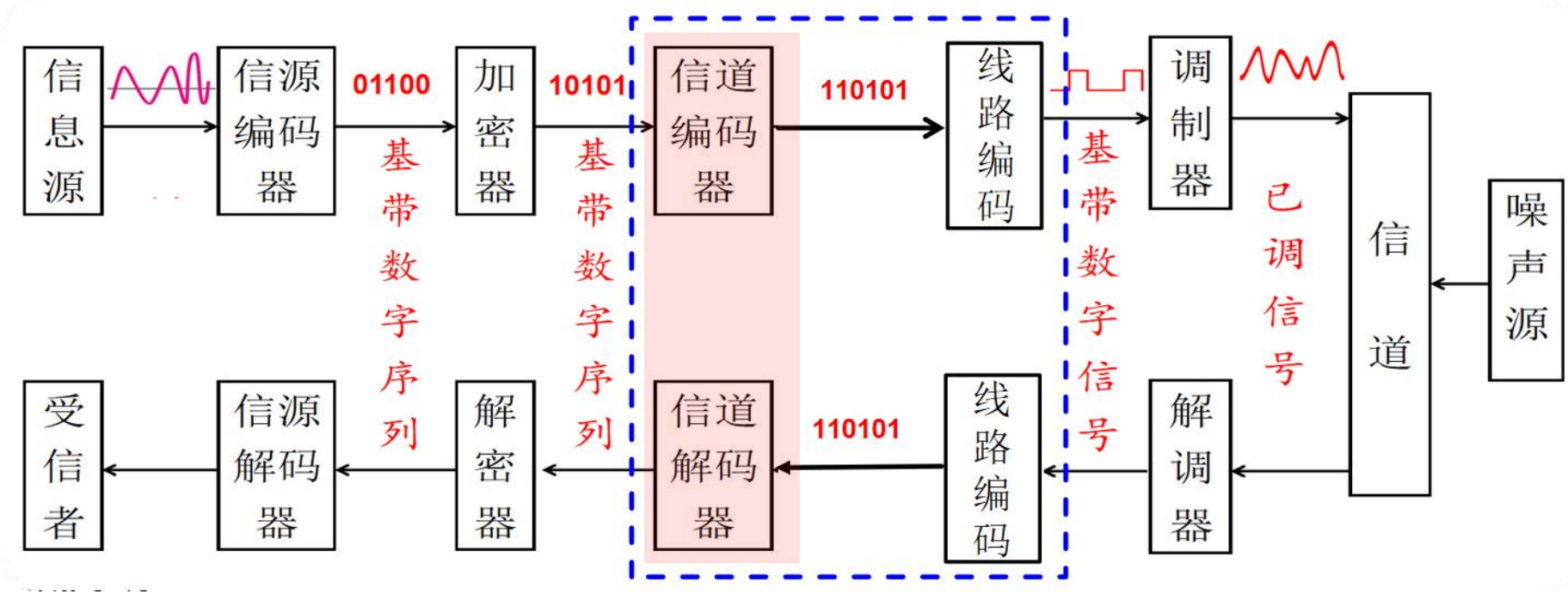
- Position, Functions and Services(include 3.1.1)
- 3.1.2 Framing
- 3.2 Error Detection and Correction (include 3.1.3)
- Flow control (textbook 3.1.4 & 3.3 & 3.4)
  - 3.3 Elementary Data Link Protocols
  - 3.4 Sliding Window Protocols
- 3.5 Example Protocols(PPP、HDLC)

Why do we need Data Link Control?

- Probable transmission bit errors in physical layer: **Error control**
- Receiver may need to prevent too much data coming in: **Flow control**



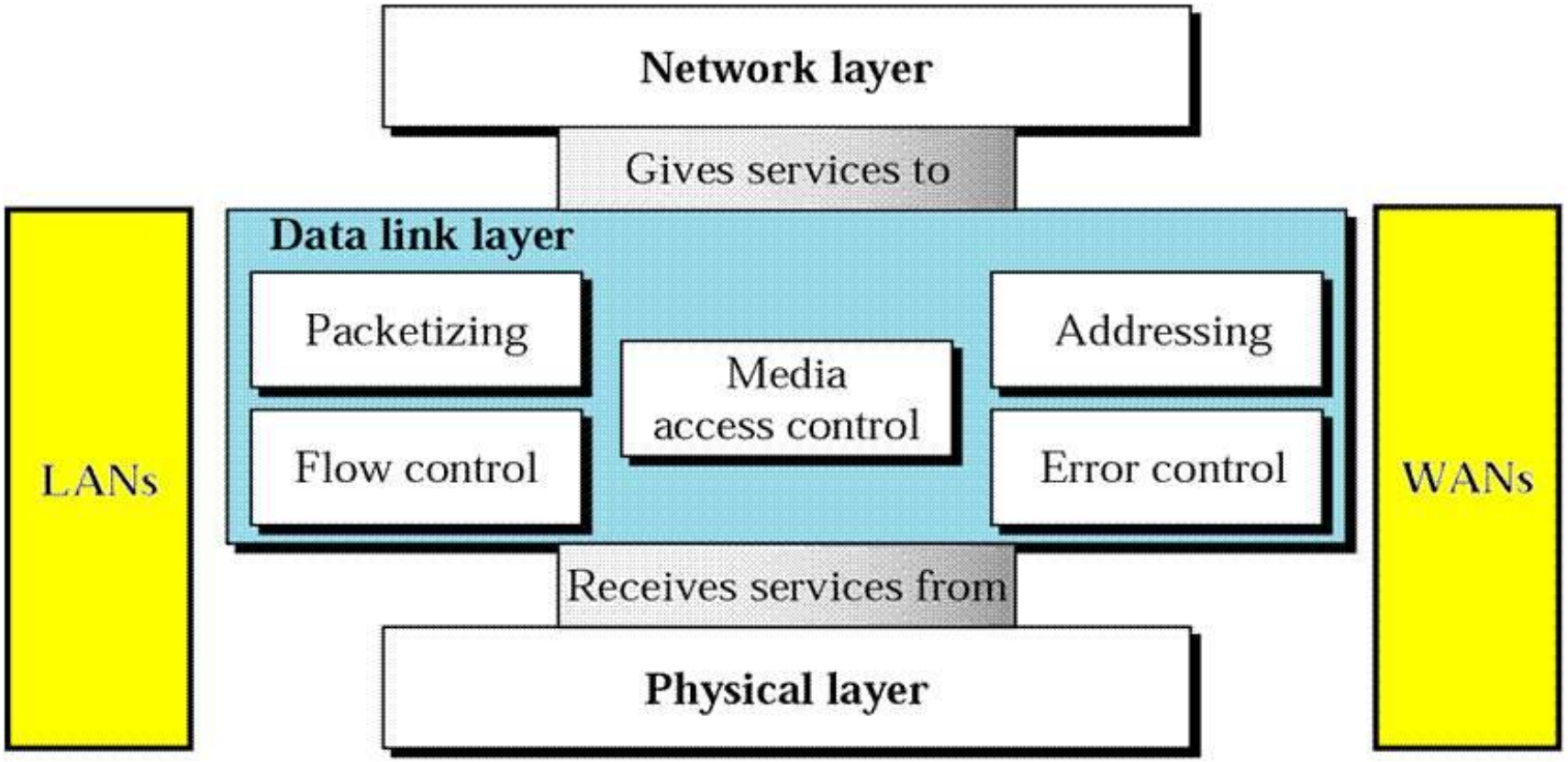
- Broadcast channel: **Addressing**, **Media access control**



信道编码/译码—以信息正确传输为目标，进行**差错控制编码**

Position and Functions

- Data link layer has responsibility of transferring data (frame) from one node to adjacent node in a reliable and efficient way over a link



[Forouzan]

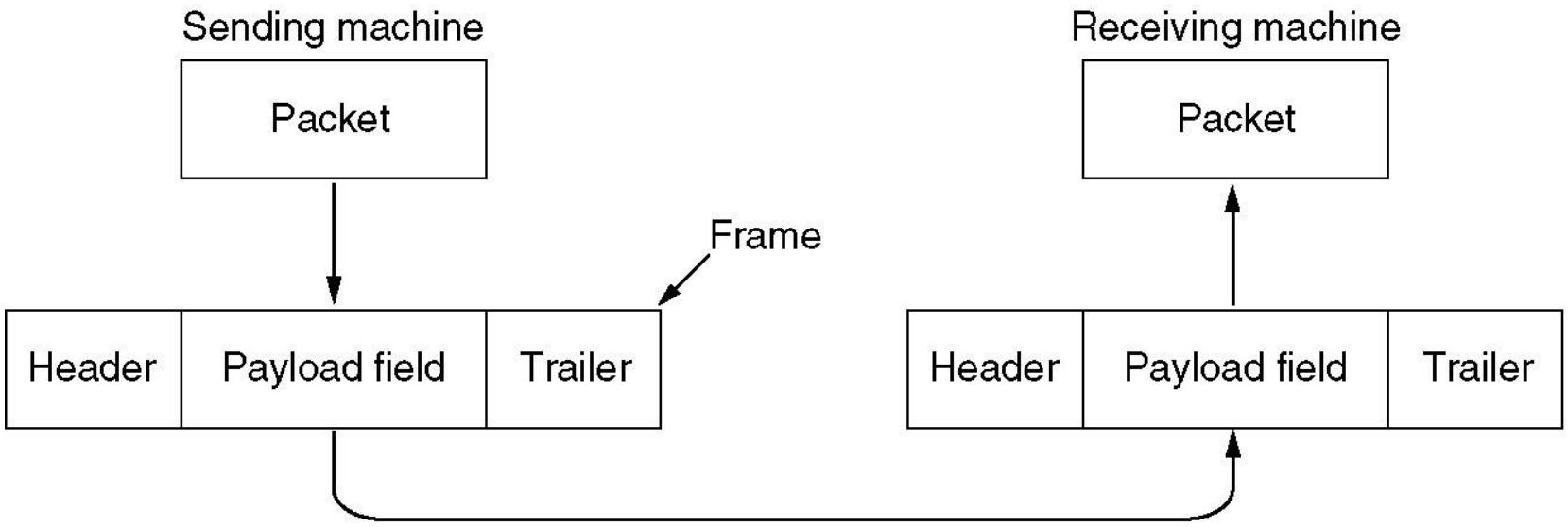
3.1.1 Data Link Layer Design Issues

- Goals
  - Study the algorithms for achieving reliable, efficient communication between two adjacent nodes
- Data Link Layer Design Issues
  - Framing
  - Error Control: Dealing with transmission errors
  - Flow Control: Slow receivers not swamped by fast senders
  - Addressing
  - Media access control

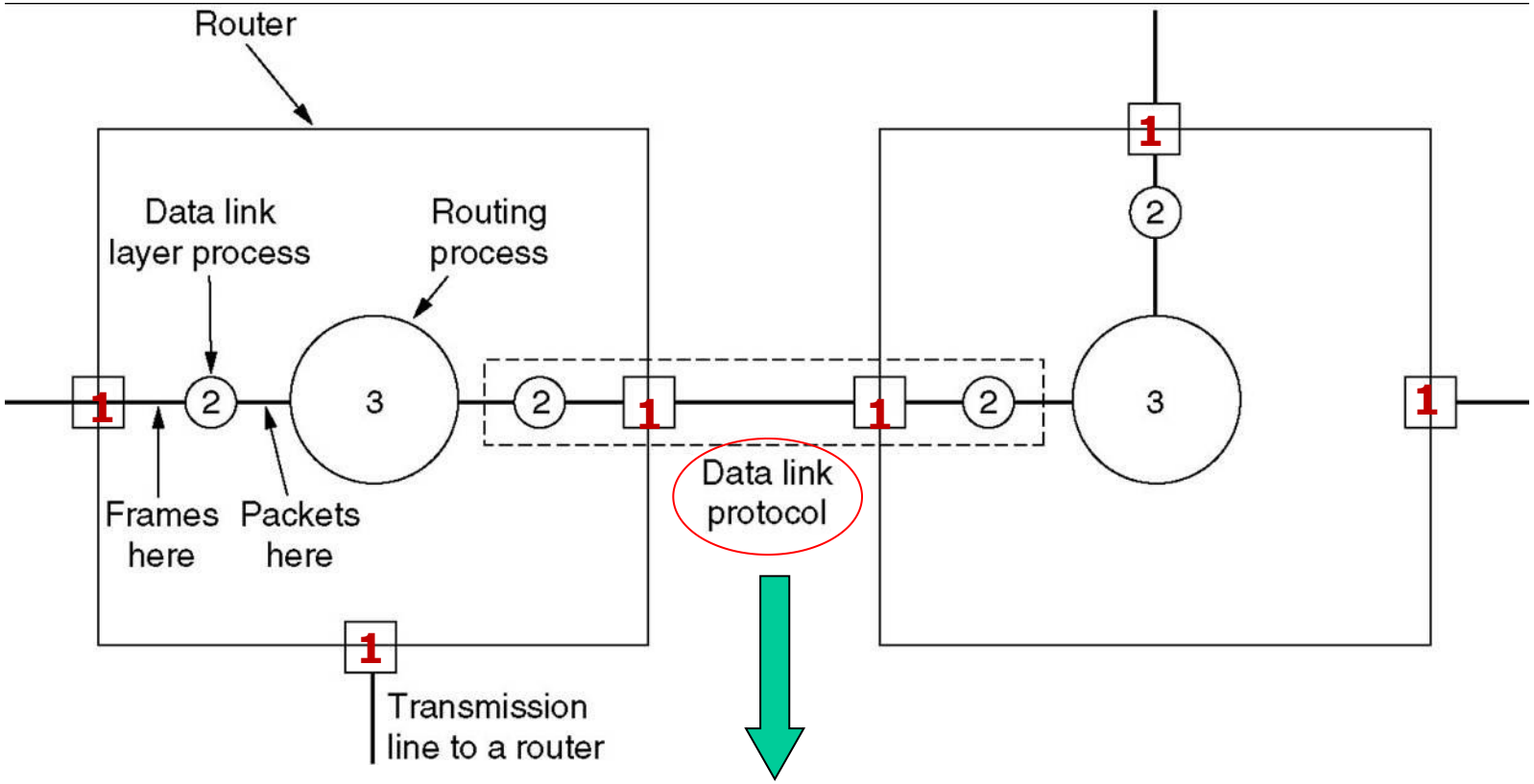
} chapter 4

Packets vs. Frames

Encapsulation(封装)

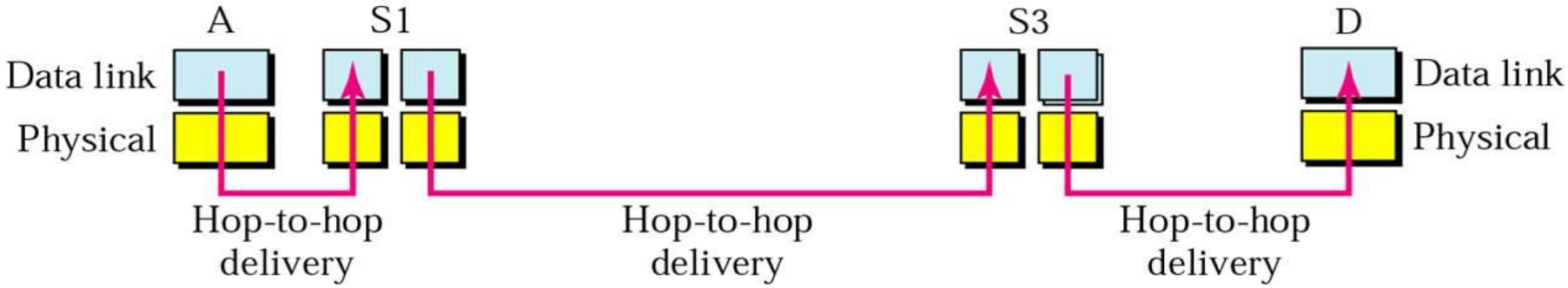


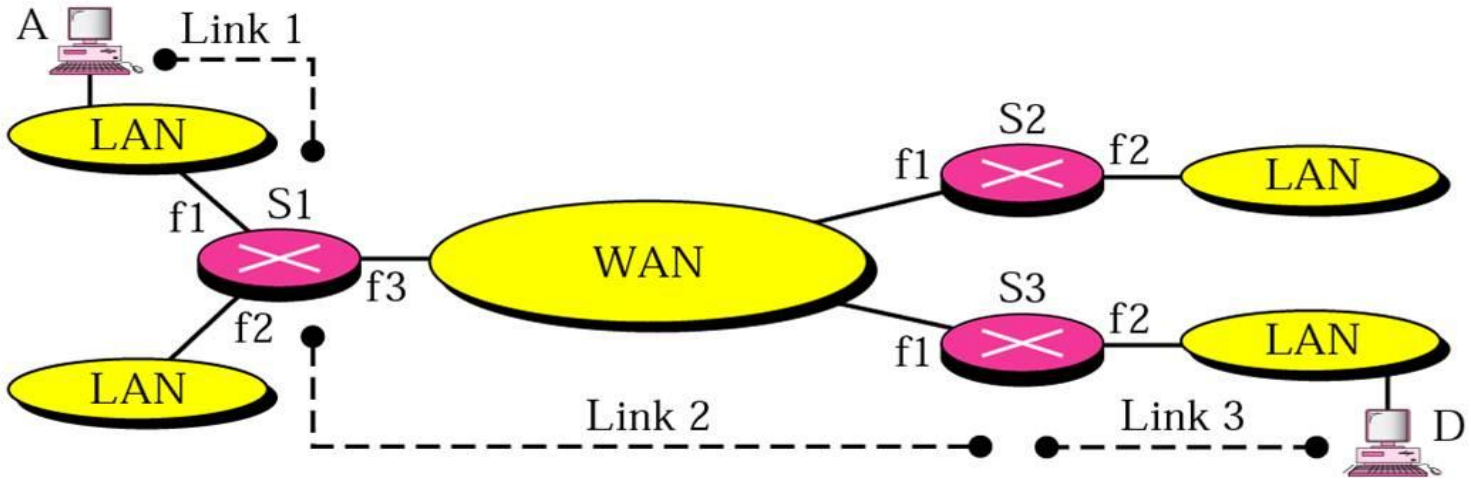
Functions of Data Link Layer: Illustration



- Flow Control: How much data shall be sent?
- Error Control: How can errors be corrected?
- Access Control: who shall send? → chapter 4

Data Link Layer: hop-to-hop communication(逐跳通信)

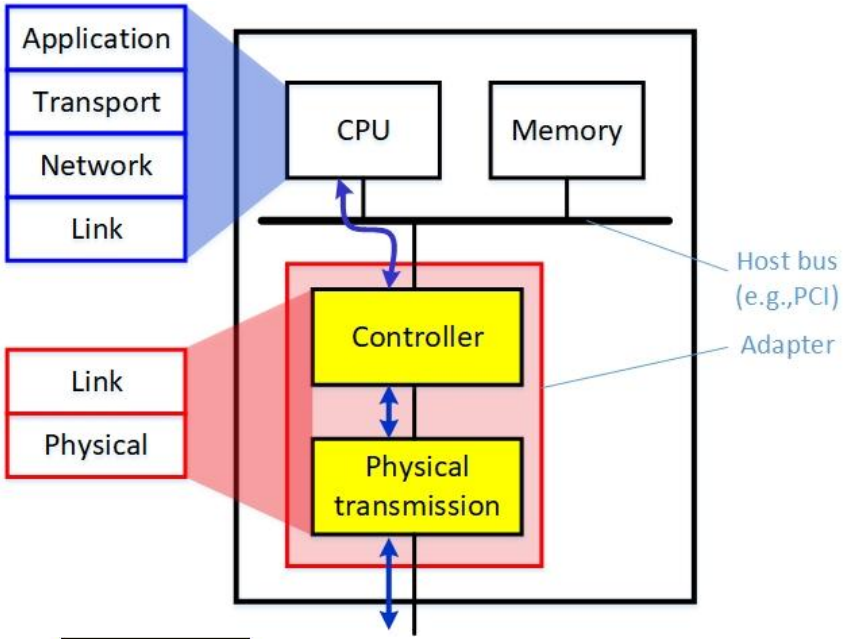


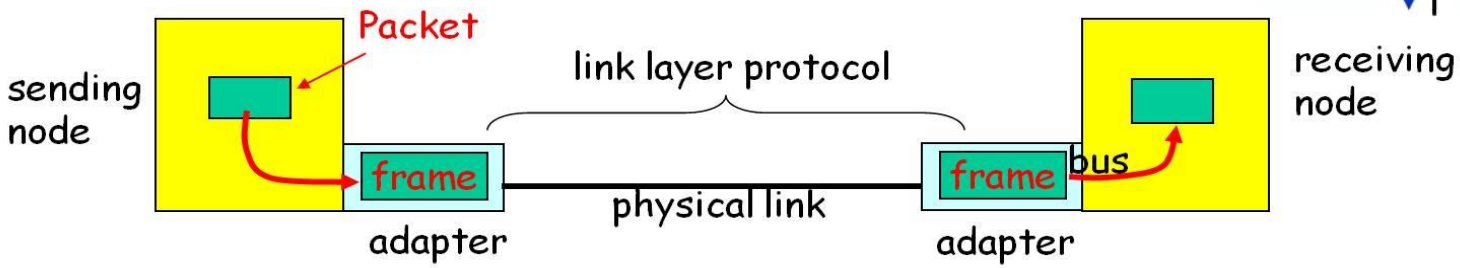


[Forouzan]

Data Link Layer: Adapters Communicating

- **Sending side**
  - Encapsulates network packet in a **frame**
  - Adds error checking bits, reliable transfer, flow control, etc.
- **Receiving side**
  - Looks for errors, reliable transfer, flow control, etc
  - Extracts packet, passes to receiving node
- **Implemented in "adapter" (NIC)**
  - Ethernet card, 802.11 card





[Kurose]

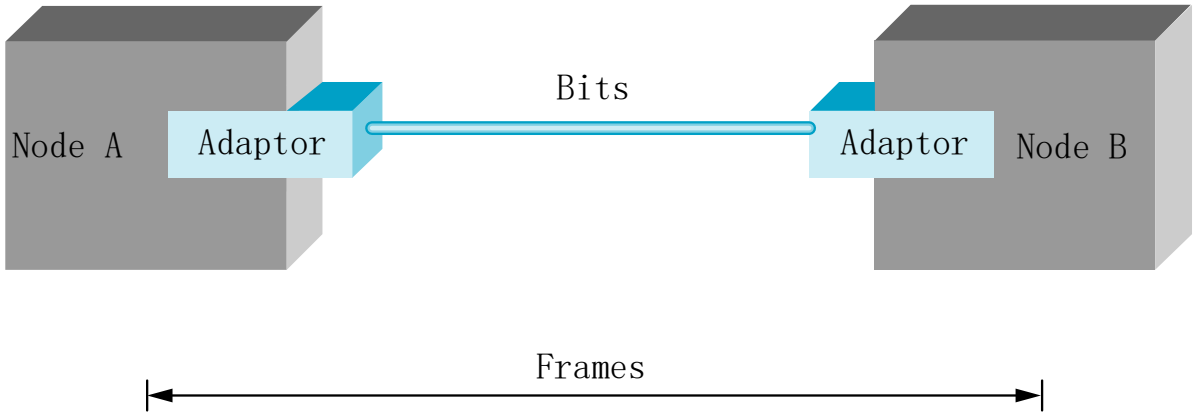
Services Provided to Network Layer(2)

- Connectionless services
  - Unacknowledged connectionless service
    - No acknowledgement, no logical connection
    - Most LANs use such service (eg. Ethernet)
  - Acknowledged connectionless service
    - Each frame acknowledged, used in unreliable channel, such as wireless systems (eg. Wifi)
- Acknowledged connection-oriented service
  - A connection established, frames are numbered, reliable transmission guaranteed (eg. HDLC)

Outline

- Position, Functions and Services(include 3.1.1)
- 3.1.2 Framing
- 3.2 Error Detection and Correction (include 3.1.3)
- Flow control (textbook 3.1.4 & 3.3 & 3.4)
  - 3.3 Elementary Data Link Protocols
  - 3.4 Sliding Window Protocols
- 3.5 Example Protocols(PPP、HDLC)

3.1.2 Framing



- A sequence of bits is sent from node A to node B over a point-to-point link.
- Node B must recognize exactly what set of bits constitutes a **frame**, that is, it must determine where the frame begins and ends.

Framing(成帧)

- Framing：Marking the start and end of each frame, 拆分比特流，识别出帧(帧同步)
- Requirements
  - Simple--容易实现
  - code independence——与传输的消息内容无关
  - efficient—使用的信道带宽尽量少
  - robustness——不易出错，出错后易重新同步

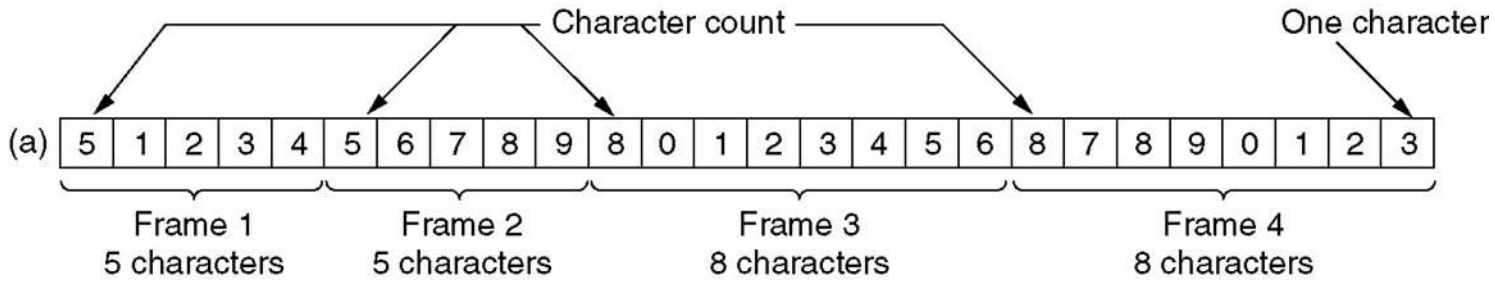
Framing(成帧)

- Methods
  - Character count (字符计数法)
  - Flag byte with byte stuffing (字符填充法)
  - Starting and ending flags, with bit stuffing(比特填充法)
  - Physical layer coding violations (物理层编码违例法)

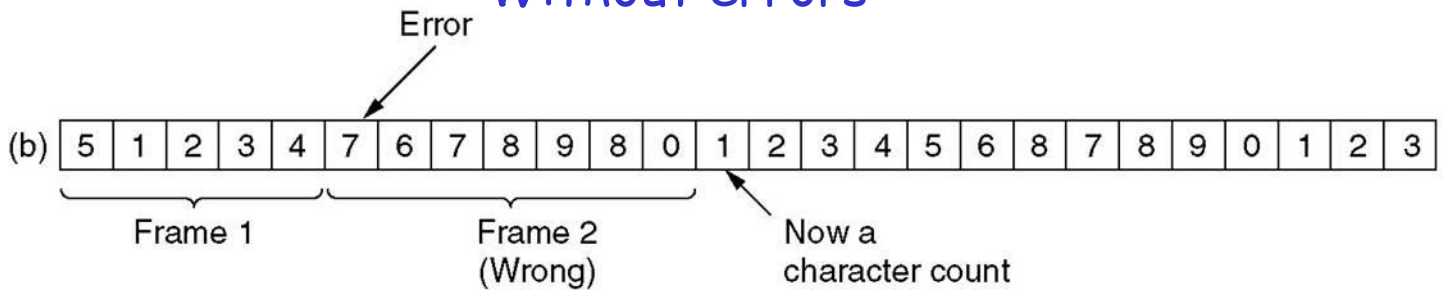
Character Count(字符计数法)

- Using a field in header to specify number of characters in the frame
- The count may be garbled by a **transmission error**

(If lost synchronisation, no way to get in control again)



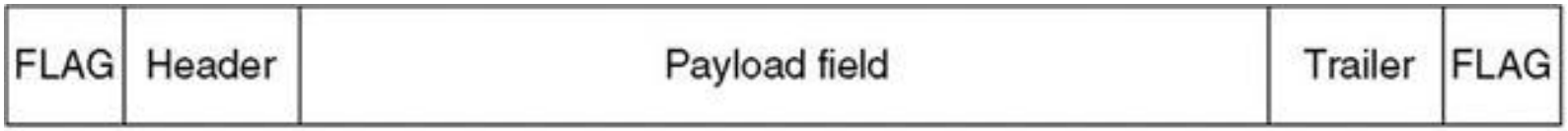
Without errors



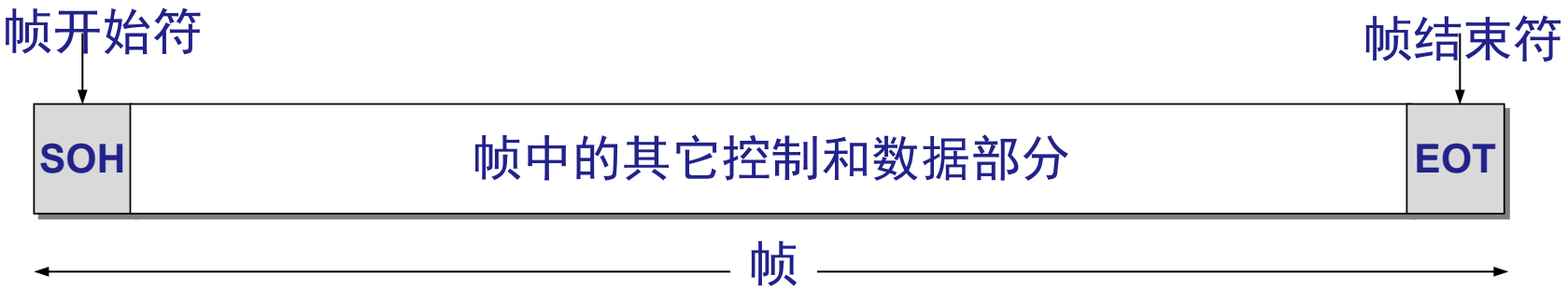
With one error

Byte (Character) Stuffing(字节/字符填充)

- Using flag bytes to mark the starting and ending of a frame
- Closely tied to the use of 8-bit characters

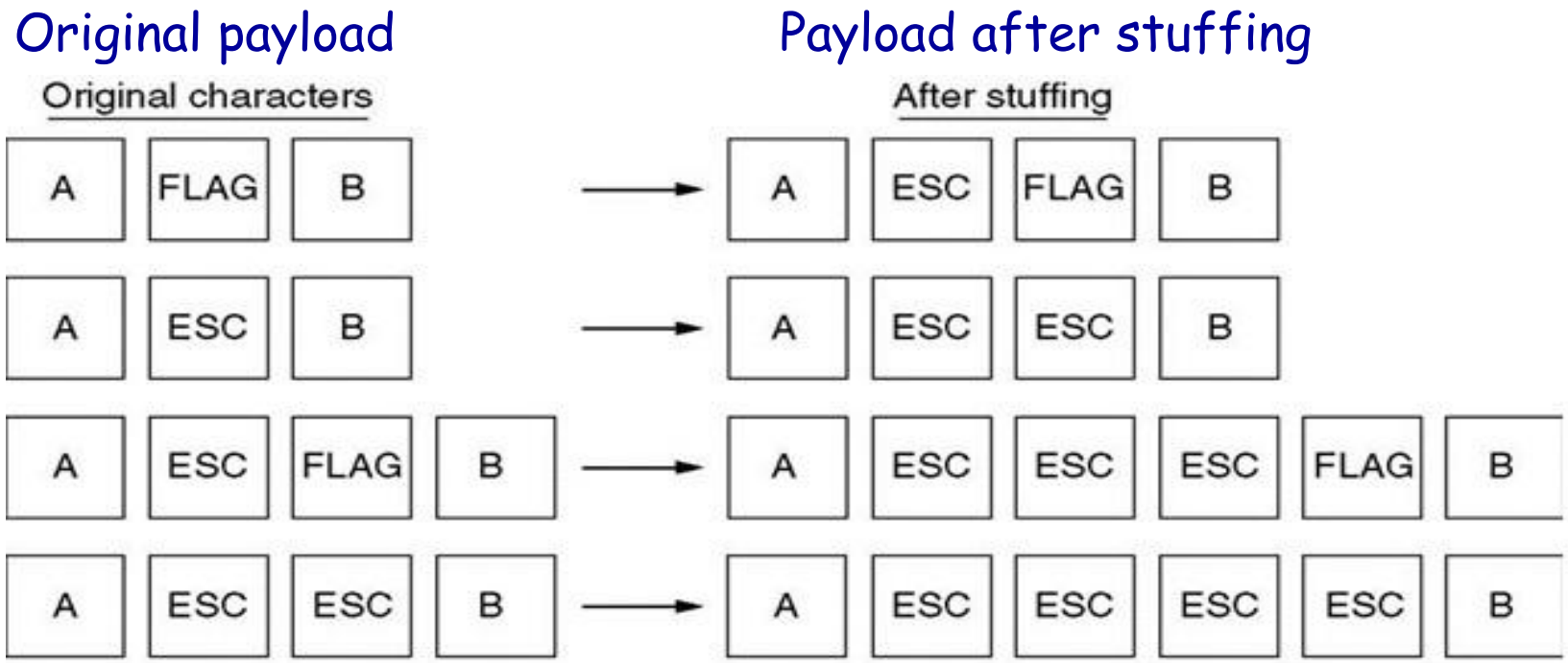


For example,



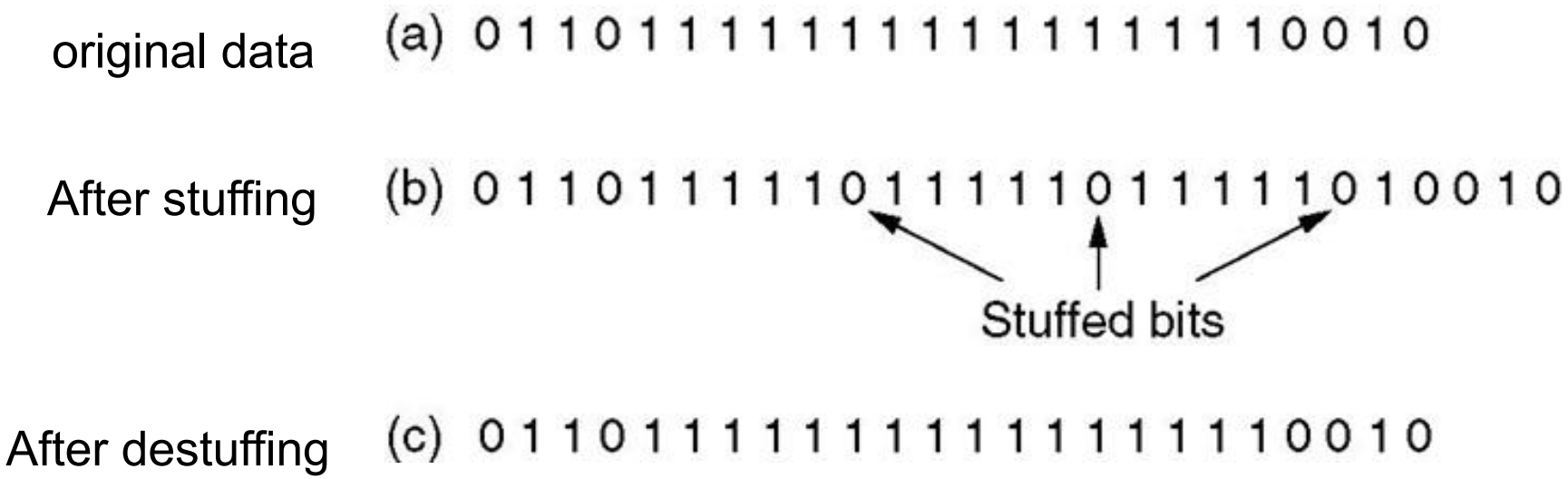
Byte stuffing: transparent transmission（透明传输）

- What if payload includes same byte with the flag?
  - Using special stuffing byte: ESC (eg. 0x1B)
  - Inserted before "flag" in payload

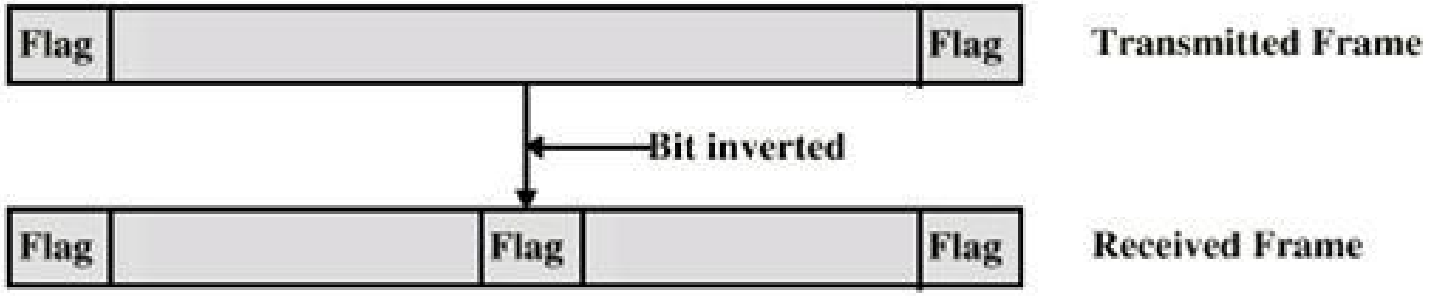


Bit Stuffing（比特填充）

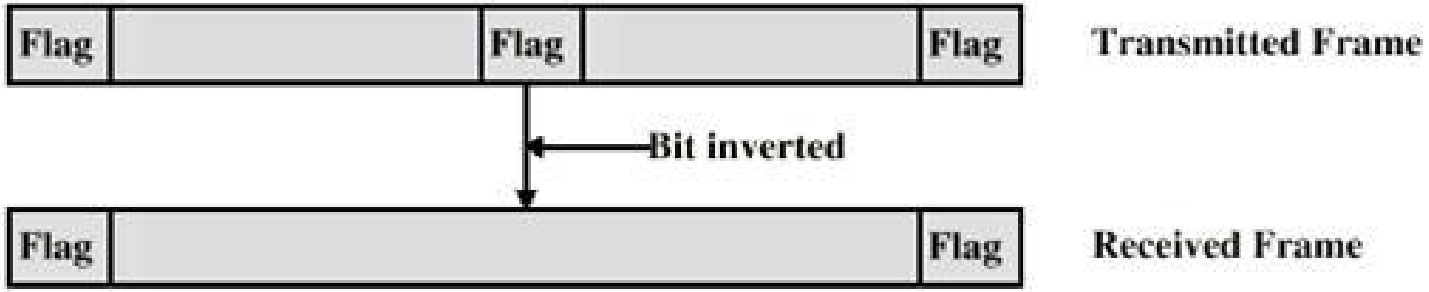
- Allowing a frame to contain arbitrary number of bits
- Starting and ending flag: 0111,1110
- Stuffing: a '0' is stuffed after five consecutive 1s
- Used in HDLC



Possible Errors of Bit Stuffing



(b) An inverted bit splits a frame in two



(c) An inverted bit merges two frames

[Stallings]

比特填充

- HDLC使用比特填充法成帧，帧定界符为01111110，在帧的内容中若出现连续的5个1，则立即插入1个0，在接收方应该去掉加入的0。

  从信道上收到比特序列： 1101 0111 1110 0111 1110 1101 1011 1110 0010 1100 0101 1111 0101 1001 1111 1001，该比特序列中包含一个完整的帧，用十六进制写出该帧的内容（不包含帧的首尾标志）。

1101 1011 1110 0010 1100 0101 1111 0101 10

1101 1011 1110 0010 1100 0101 1111 0101 10

1101 1011 111 0 010 1 100 0 101 1 111 1 01 10

D B E 5 8 B F 6

Physical Layer Coding Violations
（物理层编码违例法）

- Applicable to line encoding schemes with redundancy
  - idea: in physical layer, some signals will not occur in regular data, these reserved signals can be uesed to indicate the start and the end of frame.
  - *For example:* Manchester encoding--each valid bit has a transition in the middle of signal, then code without transition, i.e. high-high and low-low are used for frame delimitation
  - *Application*: **Ethernet** uses a preamble followed by a length field in the header to locate the start and the end of the frame

preamble = 7 octet(10101010)+1 octet(10101011)

Outline

- Position, Functions and Services(include 3.1.1)
- 3.1.2 Framing
- 3.2 Error Detection and Correction (include 3.1.3)
  - 3.2.1 Error-Correcting Codes (ECC)
  - 3.2.2 Error-Detecting Codes (EDC)
- Flow control (textbook 3.1.4 & 3.3 & 3.4)
  - 3.3 Elementary Data Link Protocols
  - 3.4 Sliding Window Protocols
- 3.5 Example Protocols(PPP、HDLC)

Error Control (textbook 3.1.3)

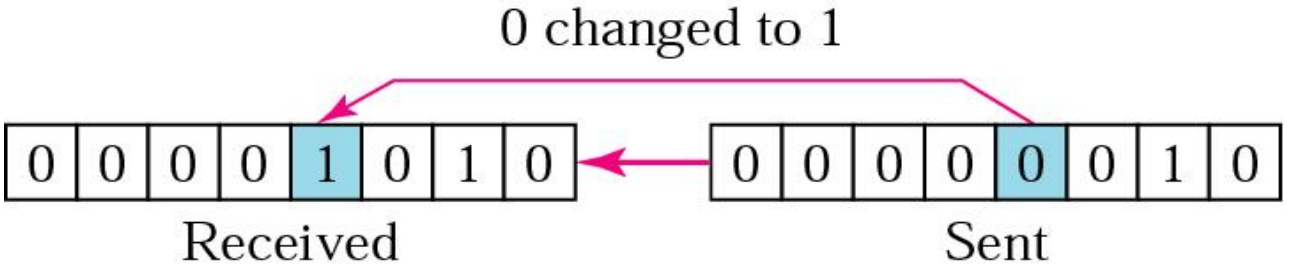
- Make sure all frames are eventually delivered to the network layer at the destination and in the proper order
- Type of Errors
  - Lost frames: a frame failed to arrive to the other side
    - Noise burst
    - Hardware troubles may cause a frame to vanish completely
  - Damaged frames: some bits are in error ➡

3.2 Error Control (差错控制)

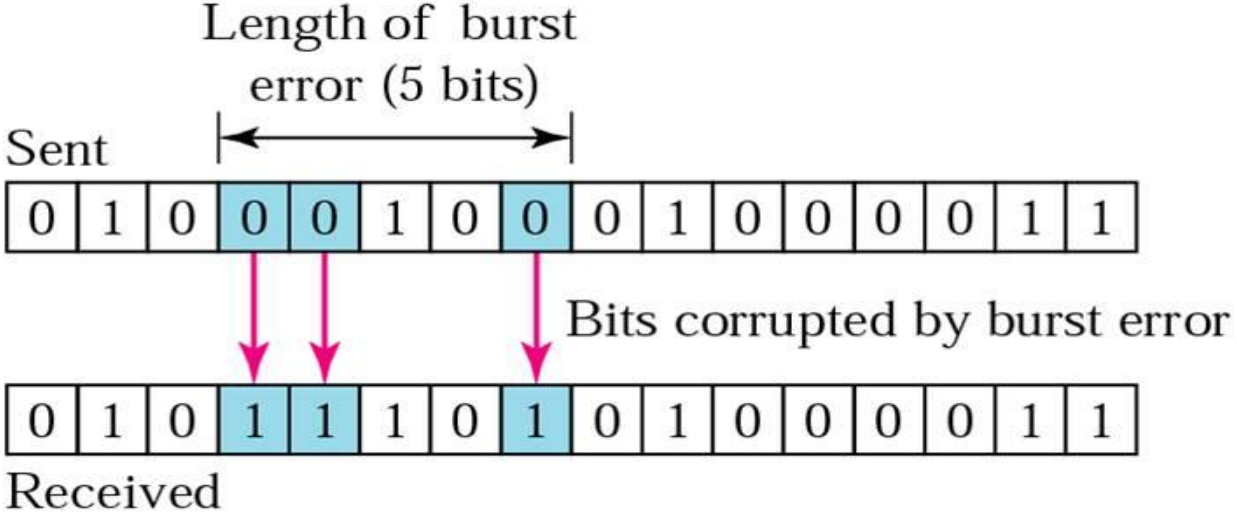
- Error detection
  - Provide the sender with some feedback(eg. ACK, NAK) about what is happening at the other end of the line
    - Parity check: able to detect single bit errors
    - Cyclic Redundancy Check (CRC): detecting some burst errors
    - Checksum, to be studied in Network layer
  - If either the frame or the ACK is lost, the timer will go off, alerting the sender to a potential problem
- Error correction
  - Forward Error Correction (FEC，前向纠错) ➡
  - Retransmission: Automatic Repeat reQuest (ARQ)

Single-bit Error & Burst Error

- Single-bit error, only one bit has changed

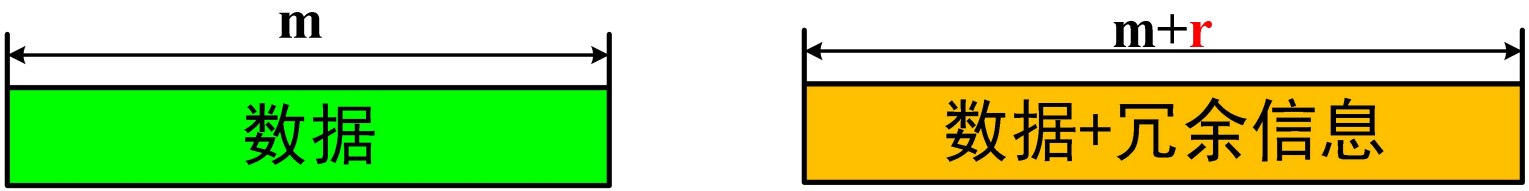


- A burst error(突发差错) means that 2 or more bits have changed.



3.2.1Error-Correcting Codes（纠错码）

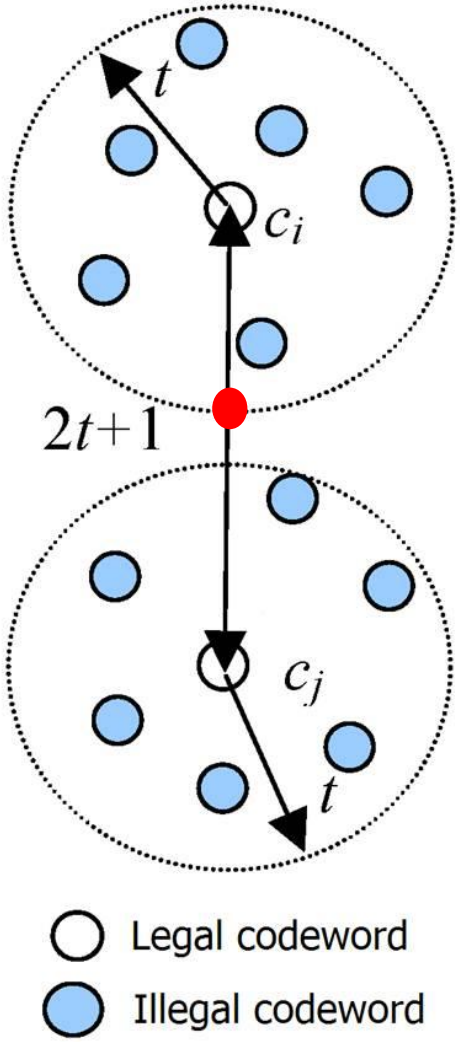
- 基本思路：将冗余信息加入到待发送的信息中



- 基本术语
  - Block code(块编码): *r* check bits are computed solely as a function of the *m* data bits
  - Systematic code(系统码):先发m位数据,再发r位校验
  - Line code(线性码): *r* check bits are computed as a linear function of the *m* data bits
    - eg. XOR
  - Codeword(码字):n (n=m+r)
  - Code rate(码率): m/n (n=m+r)

Error Correcting Code（纠错码）：Hamming Distance(汉明/海明距离)

- Hamming distance: The number of bit positions in which two codewords differ
  - eg. 1000 1001 and 1011 0001, Hamming distance = 3
- To detect ***t-bit*** errors, need a distance ***t+1*** code
- To correct ***t*** -bit errors, need a distance ***2t+1*** code



Example of an ECC

- Consider a code with only four valid codewords:

  0000000000, 00000 11111

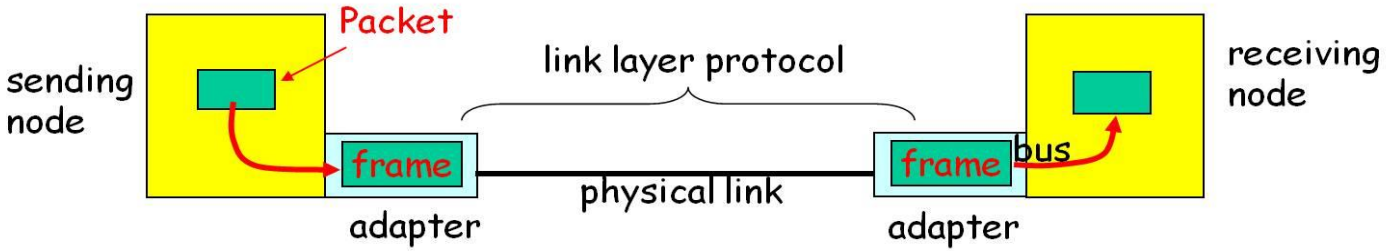
  11111 00000, 11111 11111

- This code has a distance 5, which means that it can correct 2-bit errors.
  - If the codeword 00000 00111 arrives, the receiver knows that the original must have been 00000 11111
  - If a triple error changes 00000 00000 into 00000 00111, the error will not be corrected properly

- Hamming distance of Hamming Code: 3
  - Able to correct 1-bit errors **or** detect 2-bit errors

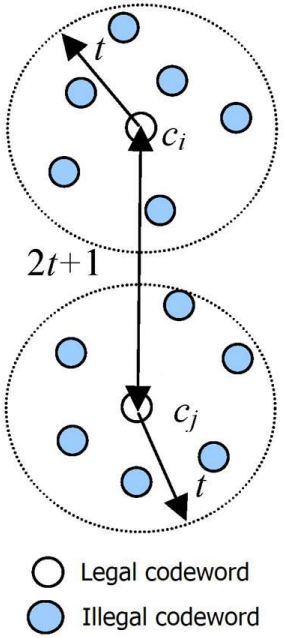
Review(1)

- Functions：reliable, efficient communication between two adjacent machines (hop-to-hop)
- Services
  - Unacknowledged connectionless service
  - Acknowledged connectionless service
  - Acknowledged connection-oriented service
- Technologies
  - Framing, Error control, Flow control, Media Access Control
- Implemented in network adapter (NIC)



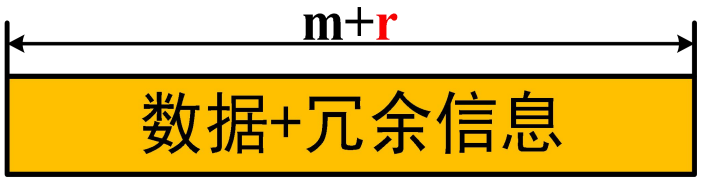
Review(2)

- Framing
  - DL:Character count, byte stuffing, bit stuffing
  - PHY:coding violations
- Error control
  - Error detection: Error detecting codes
  - Error correction
    - FEC: Hamming code
    - ARQ: error detecing code + retransmission
    - HEC(Hybrid Error Correction): 混合纠错
  - Hamming Distance
    - To detect ***t-bit*** errors, need a distance ***t+1*** code
    - To correct ***t-bit*** errors, need a distance ***2t+1*** code



The Number of Check Bits

- Find out the number of check bits needed to correct single bit error
- Coding
  - m: message bits
  - r: check bits
  - n: codeword length (n=m+r)



- 2^m legal messages
- Each legal message has n illegal codewords at a distance 1 from it. Altogether there are $(n + 1)2^m$ codewords

$$(n + 1)2^m \le 2^n$$

$$\Rightarrow (m + r + 1) \le 2^r.$$

Example: m=7, r≥4

二进制运算(复习)

- 异或运算：

  A⊕A=0，X⊕0=X，A⊕B=C=>A=C⊕B

- 非进位加减法运算

已知：A=10001，B=01001

A+B=11000，A-B=11000，A⊕B=11000

A+B=A-B=A⊕B

Hamming Code: correction of single bit error

- The bits of the codeword are numbered, start with 1 at left end, bit 2 to its immediate right, and so on.
- Check bits
  - The bits that are powers of 2 (1, 2, 4, 8, 16…) are check bits
  - Each check bit forces the parity of some collection of bits, including itself, to be even (or odd).
- Data bits
  - The rest (3,5,6,…) are filled up with the m data bits

Example of Hamming Code(1)

- Message: 'A' (7-bit ASCII 100,0001) m=7

  Data Bits: 3,5,6,7,9,10,11 $m+r+1 \le 2^r$

- 4 Check bits, Bit 1,2,4,8 (even parity, 偶校验) r≥4

  Bit Number: 1 2 3 4 5 6 7 8 9 10 11

  Codeword: 0 0 1 0 0 0 0 1 0 0 1

发送前编码：

3=1+2, 5=1+4, 6=2+4, 7=1+2+4

9=1+8, 10=2+8, 11=1+2+8

接收者纠错：

b1⊕ b3⊕ b5 ⊕ b7⊕ b9 ⊕ b11=0 (k=1)

b2⊕ b3 ⊕ b6⊕ b7 ⊕ b10⊕ b11=0 (k=2)

b4 ⊕ b5⊕ b6⊕ b7 =0 (k=4)

b8 ⊕ b9⊕ b10⊕ b11=0 (k=8)

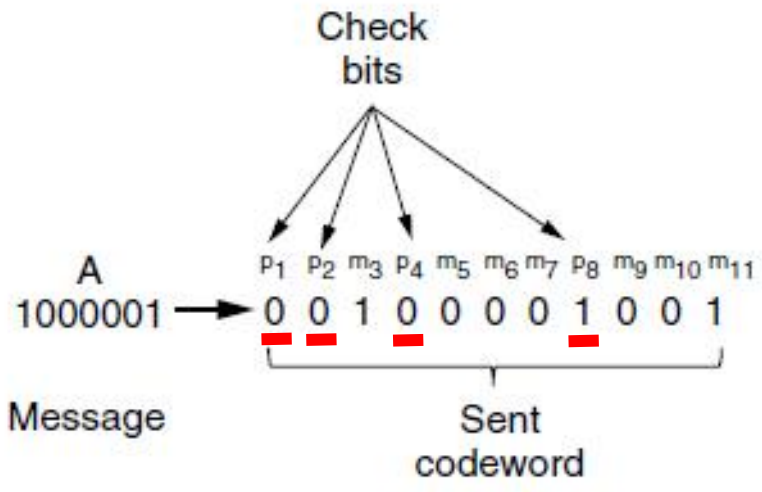
b3	0	0	1	1
b5	0	1	0	1
b6	0	1	1	0
b7	0	1	1	1
b9	1	0	0	1
b10	1	0	1	0
b11	1	0	1	1
		b4		b1

b4=0⊕b5⊕b6⊕b7

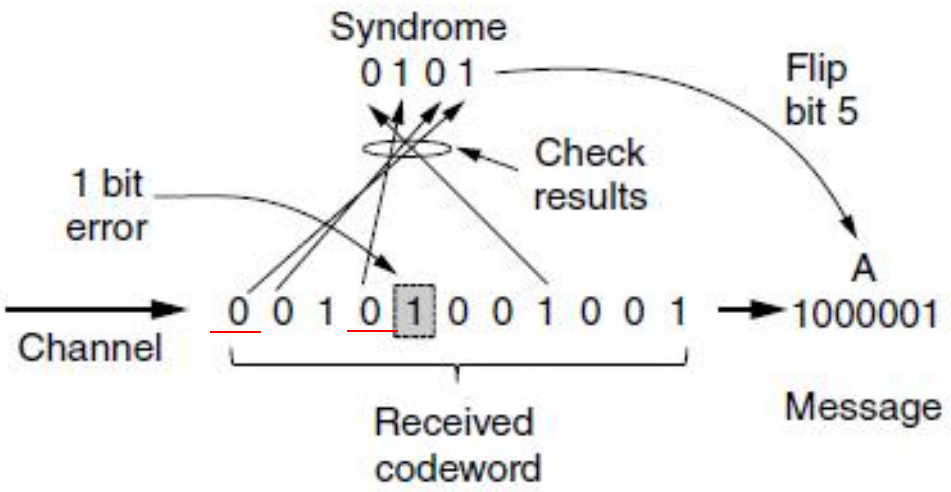
b4=b5⊕b6⊕b7

Example of Hamming Code(2)

采用偶校验来计算校验位



差错综合症



b8⊕b9⊕b10⊕b11=**0**

b4⊕b5⊕b6⊕b7=**1**

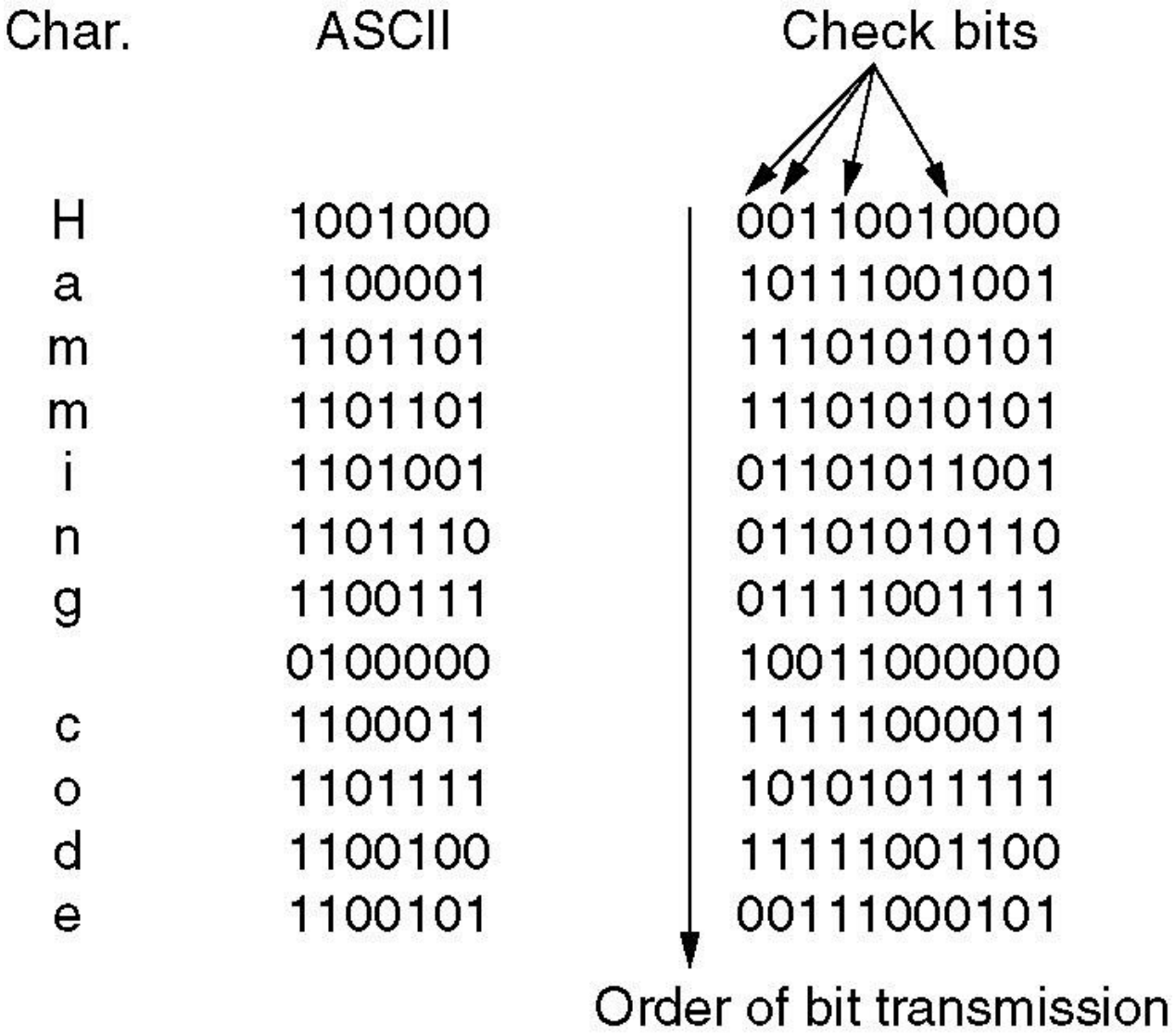
b2⊕b3⊕b6⊕b7⊕b10⊕b11=**0**

b1⊕b3⊕b5⊕b7⊕b9⊕b11=**1**

与且仅与b1和b4相关的数据位是 b5

b3	0	0	1	1
b5	0	1	0	1
b6	0	1	1	0
b7	0	1	1	1
b9	1	0	0	1
b10	1	0	1	0
b11	1	0	1	1
		b4		b1

Hamming code examples



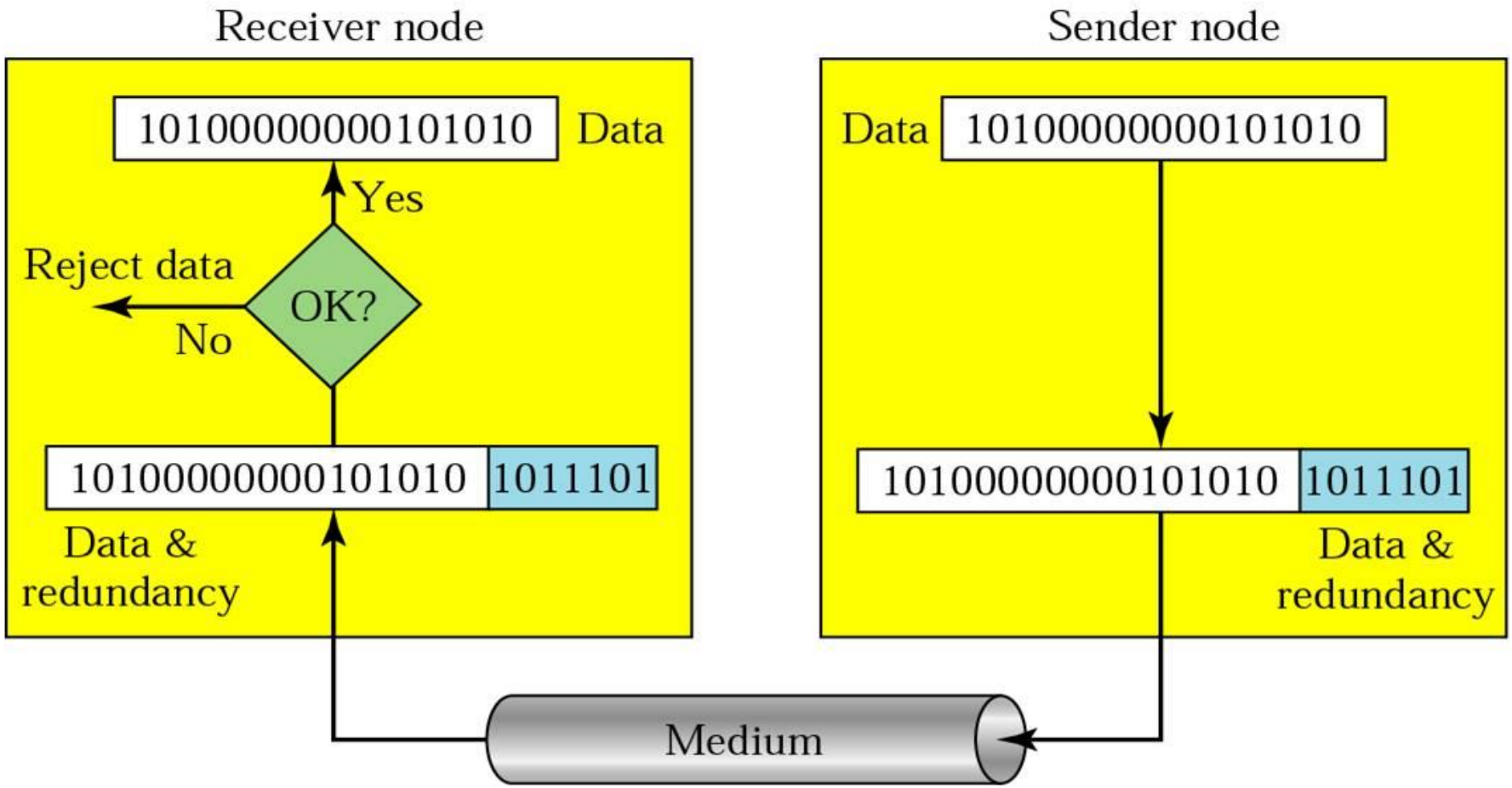
Outline

- Position, Functions and Services(include 3.1.1)
- 3.1.2 Framing
- 3.2 Error Detection and Correction (include 3.1.3)
  - 3.2.1 Error-Correcting Codes (ECC)
  - 3.2.2 Error-Detecting Codes (EDC)
- Flow control (textbook 3.1.4 & 3.3 & 3.4)
  - 3.3 Elementary Data Link Protocols
  - 3.4 Sliding Window Protocols
- 3.5 Example Protocols(PPP、HDLC)

3.2.2 Error-Detecting Codes

- On channels that are reliable, such as fiber, it is cheaper to use an error detecting code and just retransmit the occasional block found to be faulty
- On channels such as wireless links that make many errors, it is better to add enough redundancy to each block for the receiver to be able to figure out what the original block was, i.e. using error correcting code
- Example:
  - Bit Error Rate(BER)=10^{-6}, 1 block=1000bits, Data=1Mbits
  - The overhead for 1000 blocks
    - Error detection(parity check, 1 bit) + retransmission: 2001 bits
    - Hamming code : 10,000 bits, with 10 check bits for each block (m=1000bit, r>=10)

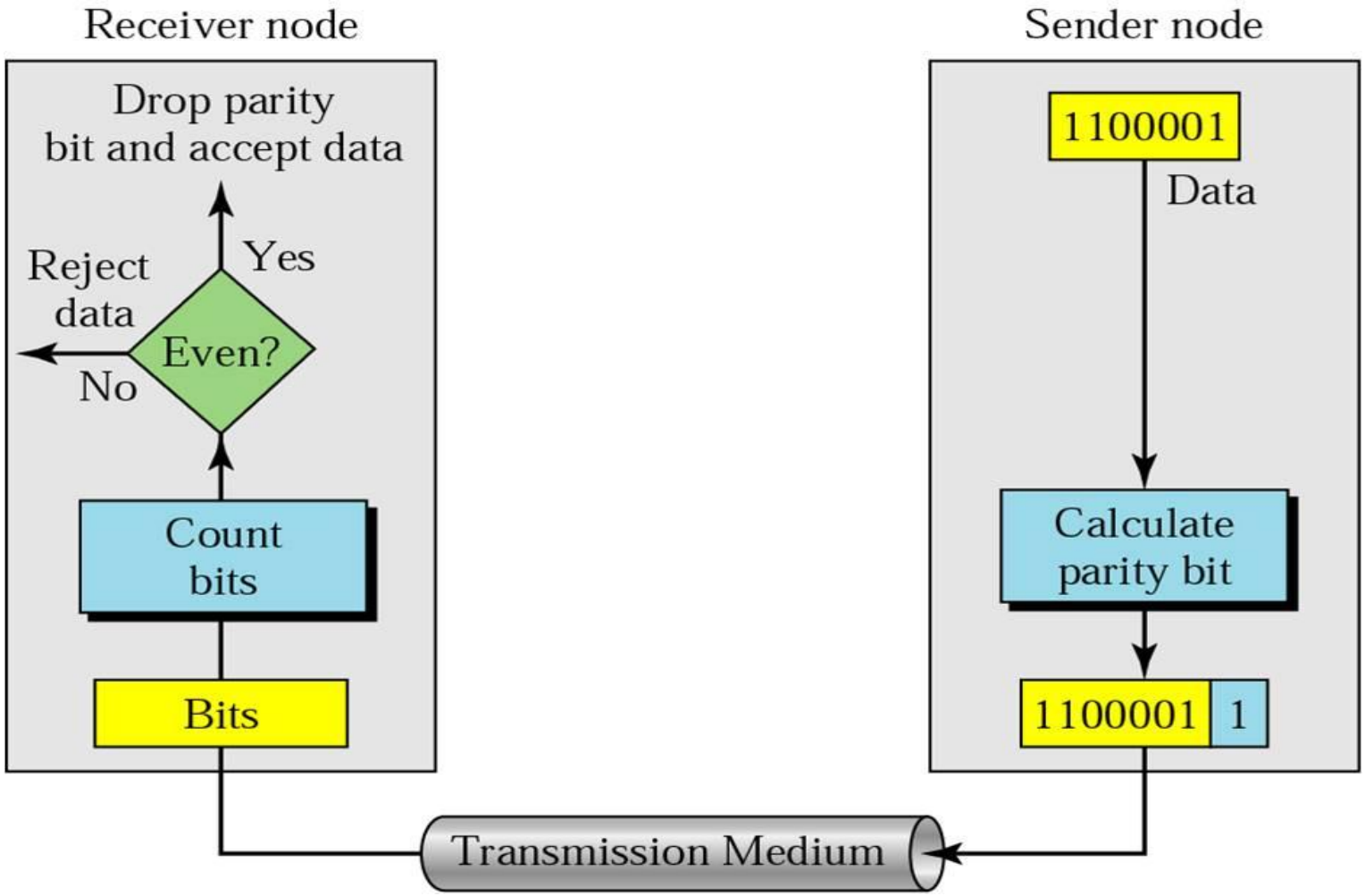
Error Detecting



目标：检错效率高(即用尽量少的冗余检验出尽量多种可能的错误)

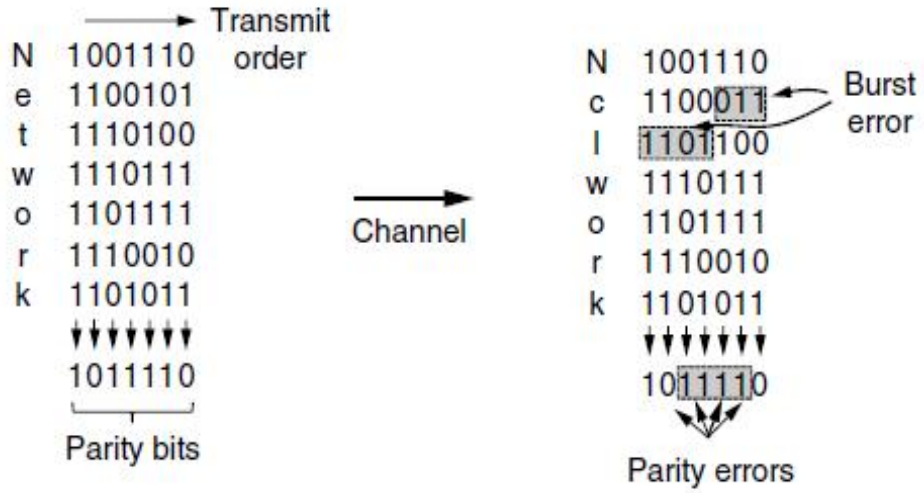
[Forouzan]

Even-parity Checking（偶校验）



[Forouzan]

Parity Check for Blocks (Interleaving, 交错校验)



- Coding (n check bits)
  - Each block is a $k \times n$ matrix
  - The matrix is transmitted one row at a time
  - A parity bit is computed separately for each column
- Performance
  - Can detect a single burst of length n
  - A burst of length $n+1$ will pass undetected if the first bit is inverted, the last bit is inverted, and all the other bits are correct
  - If block is garbled by a long burst, the probability of being accepted is 2^{-n}

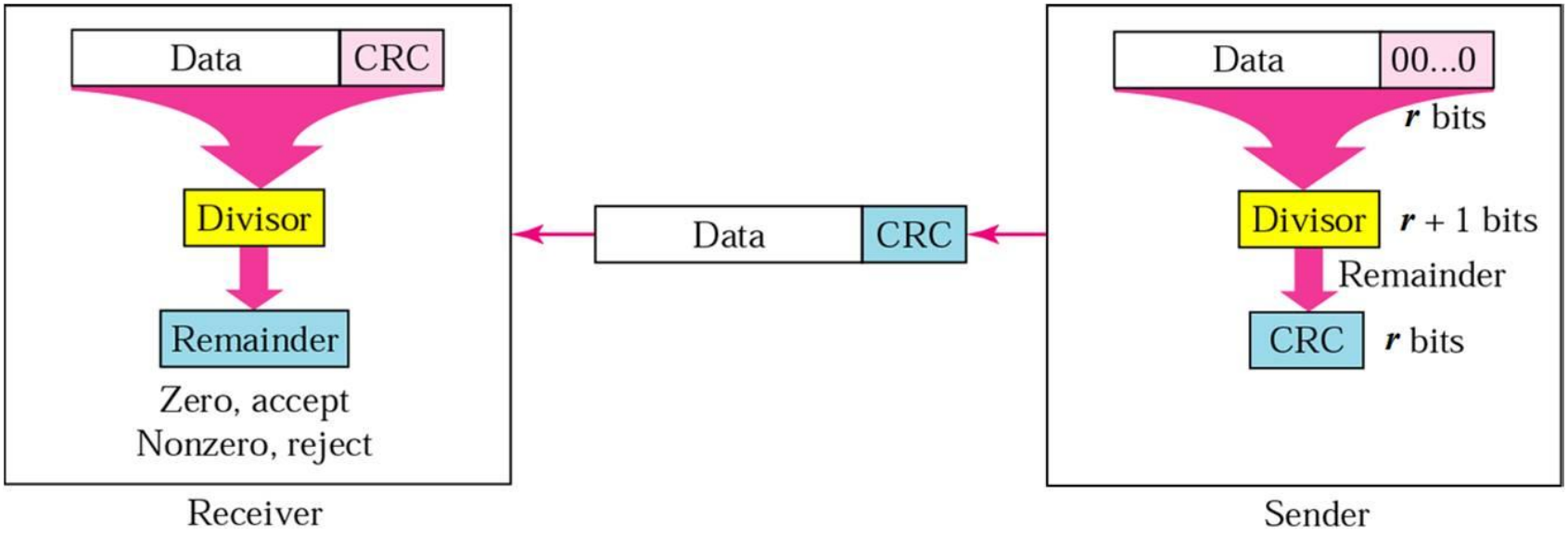
Polynomial Code（多项式编码）

Also known as CRC (Cyclic Redundancy Check，循环冗余校验), FCS (Frame Check Sequence帧校验序列)

- M(x): Treating bit strings as representations of polynomials with coefficients of 0 and 1
  - A k-bit frame is regarded as the coefficient list for a polynomial with k terms, ranging from x^{k-1} to x^0: 110001=> $x^5 + x^4 + x^0$
- Polynomial arithmetic is done modulo 2, addition and subtraction are identical to XOR
- G(x): Generator polynomial(生成多项式)

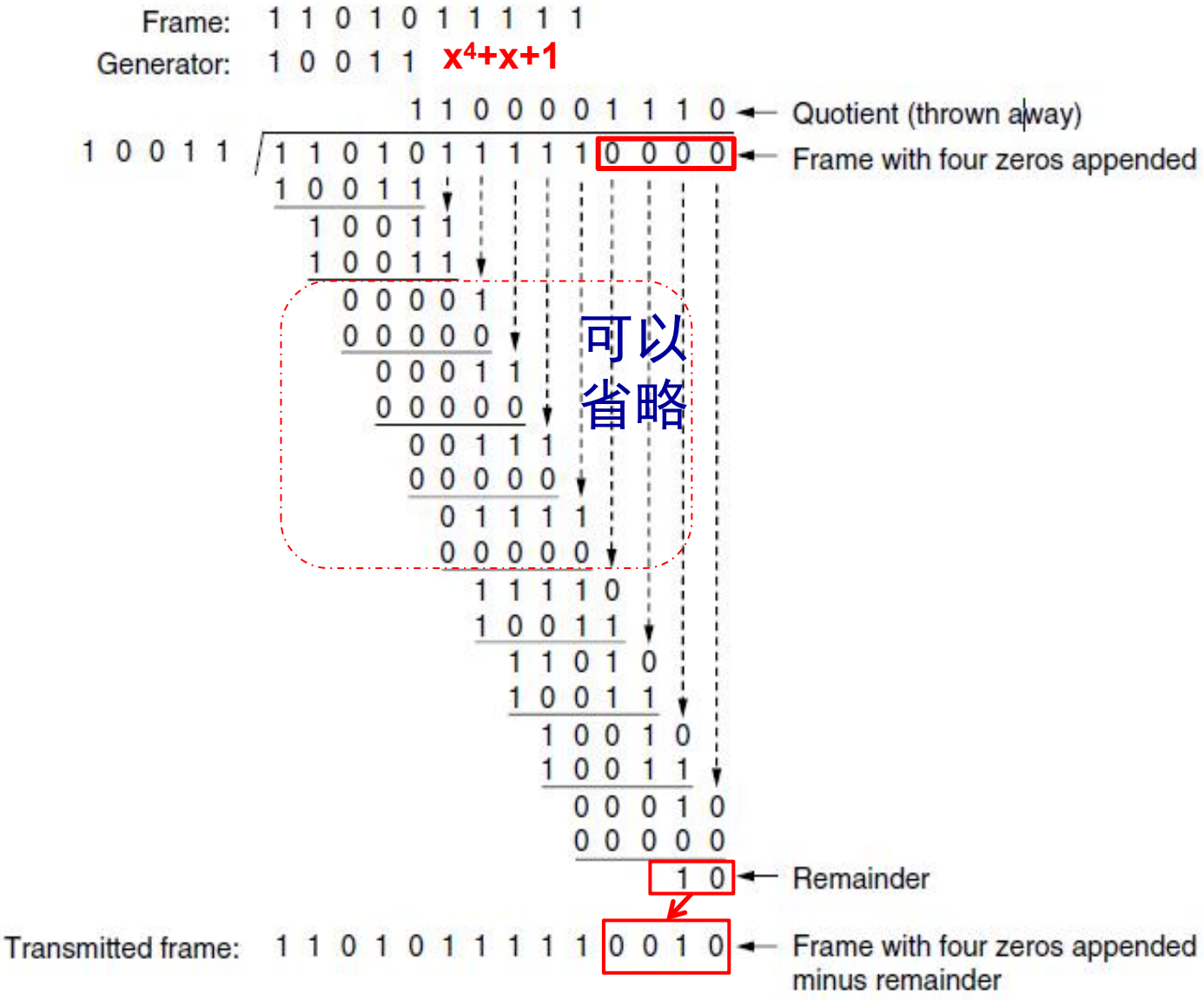
Sender: Computing the CRC

- The algorithm for computing the checksum:
  - M(x) is the frame to be sent
  - Append r zero bits to the low-order end of the frame: $M(x)*x^r$
  - Divide the bit string corresponding to G(x)
  - R(x) is the reminder of $[M(x)*x^r]/G(x)$
  - Sender: $T(x)=M(x)*x^r+R(x)$ **$=[M(x)*x^r-R(x)]/G(x)$**
  - Receiver: T(x)/G(x)



[Forouzan]

Example of CRC Calculation



Receiver: Verify the CRC

- If a transmission error occurs, so that instead of the bit string for T(x) arriving, T(x) + E(x) arrives.
- The receiver computes [T(x) + E(x)]/G(x). T(x)/G(x) is 0, so the result is E(x)/G(x)
  - Those errors that happen to correspond to polynomials containing G(x) as a factor will slip by; all other errors will be caught.

The Power of CRC

- Single bit error: $E(x) = x^i$
- r check bits will detect all burst errors of length ≤ r
  - $E(x)=x^i(x^{k-1}+x^{k-2}+...+1)$, k-1<r--> E(x)/G(x)余数≠0
- Two isolated single-bit errors are detected
  - $E(x) = x^i + x^j, = x^j(x^{i-j} + 1)$. where i > j
  - Choose G(x): G(x) does not divide $x^k + 1$ for any k up to maximum frame length
    - For example, $x^{15} + x^{14} + 1$ will not divide $x^k + 1$ for any value of k below 32,768 (4096 bytes)
- Catch all errors consisting of an odd number of inverted bits: by making *x+1* a factor of ***G(x)***
  - No polynomial with an odd number of terms has x+1 as a factor in the modulo 2 system.
- Other burst errors (length > r)
  - The probability of a bad frame been accepted is 2^{-r}

Generator Polynomial

- Certain polynomials have become international standards
- CRC-12(Telecomm system)

$$x^{12}+x^{11}+x^3+x^2+x+1$$

- CRC-16(Bisync, Modbus, USB, ANSI X3.28, SIA DC-07, many others)

$$x^{16}+x^{15}+x^2+1$$

- CRC-CCITT (HDLC, ITU X.25, V.34/V.41/V.42, PPP-FCS,ZigBee)

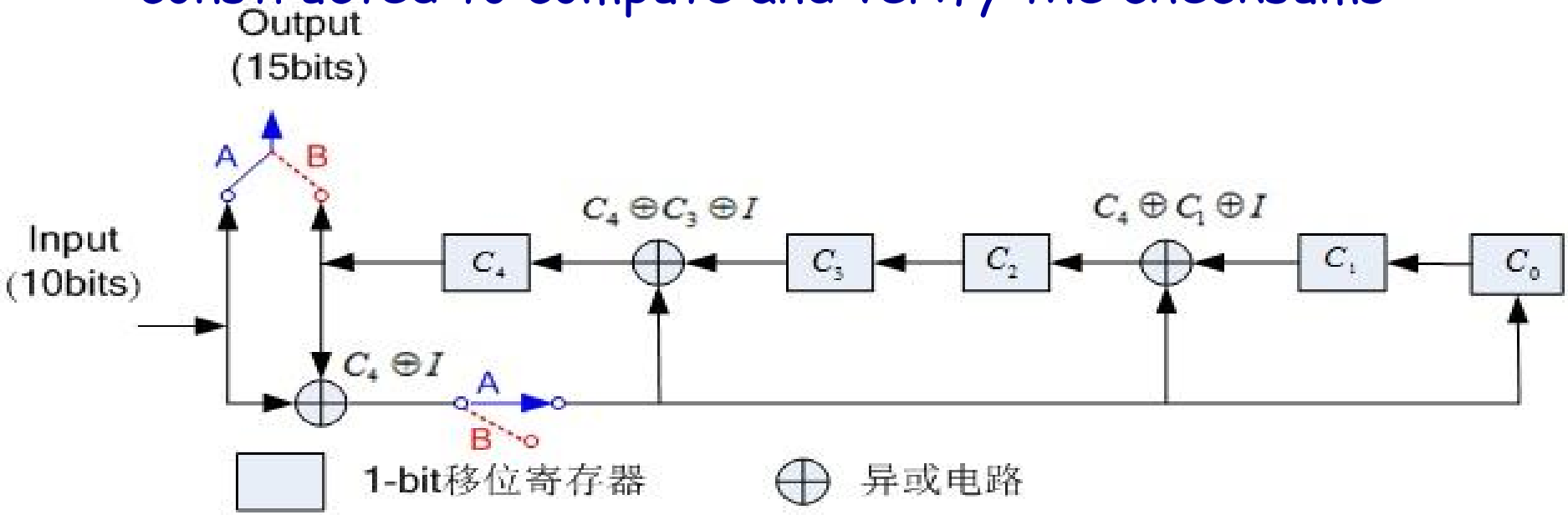
$$x^{16}+x^{12}+x^5+1$$

- CRC-32 (ZIP, RAR, IEEE 802 LAN/FDDI, IEEE 1394, PPP-FCS)

$$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x^1+1$$

The Calculation of CRC (Hardware)(略)

- A simple shift register circuit(移位寄存电路) can be constructed to compute and verify the checksums



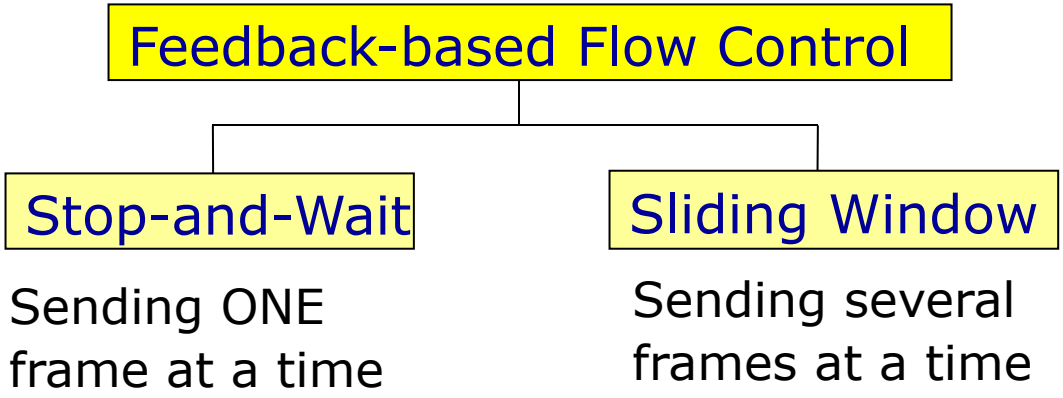
- G(x)=$x^5+x^4+x^2+1$，D=1010001101；D(X)= $x^9+x^7+x^3+x^2+1$
- 初始时移位寄存器C_0~C_4清0
- 10个信息位发送之后，断开A接通B,发送5位校验和，并将C_0~C_4清零，为下次发送做好准备

Outline

- Position, Functions and Services(include 3.1.1)
- 3.1.2 Framing
- 3.2 Error Detection and Correction (include 3.1.3)
- Flow control (textbook 3.1.4 & 3.3 & 3.4)
  - 3.3 Elementary Data Link Protocols
  - 3.4 Sliding Window Protocols
- 3.5 Example Protocols(PPP、HDLC)

Flow control (textbook 3.1.4)

- Ensuring the sending entity does not overwhelm(淹没) the receiving entity, i.e. preventing buffer overflow
- **Feedback-based flow control**: the receiver sends back information to the sender giving it permission to send more data or at least telling the sender how the receiver is doing



- **Rate-based flow control**: the protocol has a built-in mechanism that limits the transmitting rate of senders, without using feedback from the receiver, to be studied in network layer (chapter4:Token bucket &leaky bucket)

Basic Flow Control Protocols

- *3.3 Elementary Protocols*
  - 3.3.1 A Utopian Simplex Protocol
  - 3.3.2 A Simplex Stop-and-Wait Protocol for an Error-Free Channel
  - 3.3.3 A Simplex Protocol for a Noisy Channel
- 3.4 Sliding Window Protocols
  - 3.4.1 A One-Bit Sliding Window Protocol
  - 3.4.2 A Protocol Using Go Back N
  - 3.4.3 A Protocol Using Selective Repeat
  - Performance of sliding window protocols

Assumptions and lib procedures

- Some assumptions
  - Physical layer, data link layer, and network layer are independent processes
  - Machine A wants to send a long stream of data to machine B, using a reliable, connection-oriented service
  - Machines do not crash

Assumptions and lib procedures

- Library procedures
  - wait_for_event
    - frame_arrival
    - timeout
    - chsum_err…
  - from_network_layer / to_network_layer
  - to_physical_layer / from_physical_layer
  - timer operations
    - start_timer
    - stop_timer
    - timeout…
  - enable_network_layer / disable_network_layer
  - sequence number operation
    - inc (+1),

Protocol Definitions

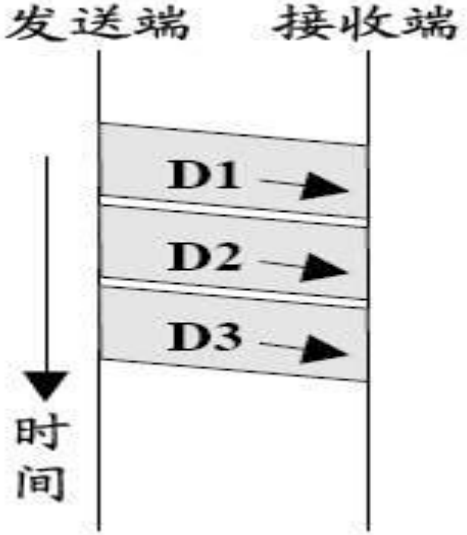
```
#define MAX_PKT 1024                                  /* determines packet size in bytes */

typedef enum {false, true} boolean;                   /* boolean type */
typedef unsigned int seq_nr;                          /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet;/*    packet definition */
typedef enum {data, ack, nak} frame_kind;             /* frame_kind definition */

typedef struct {                                      /* frames are transported in this layer */
  frame_kind kind;                                    /* what kind of a frame is it? */
  seq_nr seq;                                         /* sequence number */
  seq_nr ack;                                         /* acknowledgement number */
  packet info;                                        /* the network layer packet */
} frame;
```

Some definitions needed in the protocols to follow.
These are located in the file protocol.h.

```
/* Wait for an event to happen; return its type in event. */
void wait_for_event(event_type *event);

/* Fetch a packet from the network layer for transmission on the channel. */
void from_network_layer(packet *p);

/* Deliver information from an inbound frame to the network layer. */
void to_network_layer(packet *p);

/* Go get an inbound frame from the physical layer and copy it to r. */
void from_physical_layer(frame *r);

/* Pass the frame to the physical layer for transmission. */
void to_physical_layer(frame *s);

/* Start the clock running and enable the timeout event. */
void start_timer(seq_nr k);

/* Stop the clock and disable the timeout event. */
void stop_timer(seq_nr k);

/* Start an auxiliary timer and enable the ack_timeout event. */
void start_ack_timer(void);

/* Stop the auxiliary timer and disable the ack_timeout event. */
void stop_ack_timer(void);

/* Allow the network layer to cause a network_layer_ready event. */
void enable_network_layer(void);

/* Forbid the network layer from causing a network_layer_ready event. */
void disable_network_layer(void);

/* Macro inc is expanded in line: Increment k circularly. */
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0
```

3.3.1 Protocol 1:Utopian Simplex Protocol

- Assumptions
  - Channel is simplex
  - Channel is error-free
  - Receiver has unlimited size of buffers
- No need of error control and flow control



Protocol 1: A Utopian Simplex Protocol (乌托邦)

```
/* Protocol 1 (utopia) provides for data transmission in one direction only, from
   sender to receiver.  The communication channel is assumed to be error free,
   and the receiver is assumed to be able to process all the input infinitely quickly.
   Consequently, the sender just sits in a loop pumping data out onto the line as
   fast as it can. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender1(void)
{
  frame s;                              /* buffer for an outbound frame */
  packet buffer;                        /* buffer for an outbound packet */

  while (true) {
      from_network_layer(&buffer);      /* go get something to send */
      s.info = buffer;                  /* copy it into s for transmission */
      to_physical_layer(&s);            /* send it on its way */
  }                                      * Tomorrow, and tomorrow, and tomorrow,
                                           Creeps in this petty pace from day to day
                                           To the last syllable of recorded time
                                               - Macbeth, V, v */
}

void receiver1(void)
{
  frame r;
  event_type event;                     /* filled in by wait, but not used here */

  while (true) {
      wait_for_event(&event);           /* only possibility is frame_arrival */
      from_physical_layer(&r);          /* go get the inbound frame */
      to_network_layer(&r.info);        /* pass the data to the network layer */
  }
}
```

3.3.2 Protocol 2: Simplex Stop-and-Wait Protocol for an Error-Free Channel

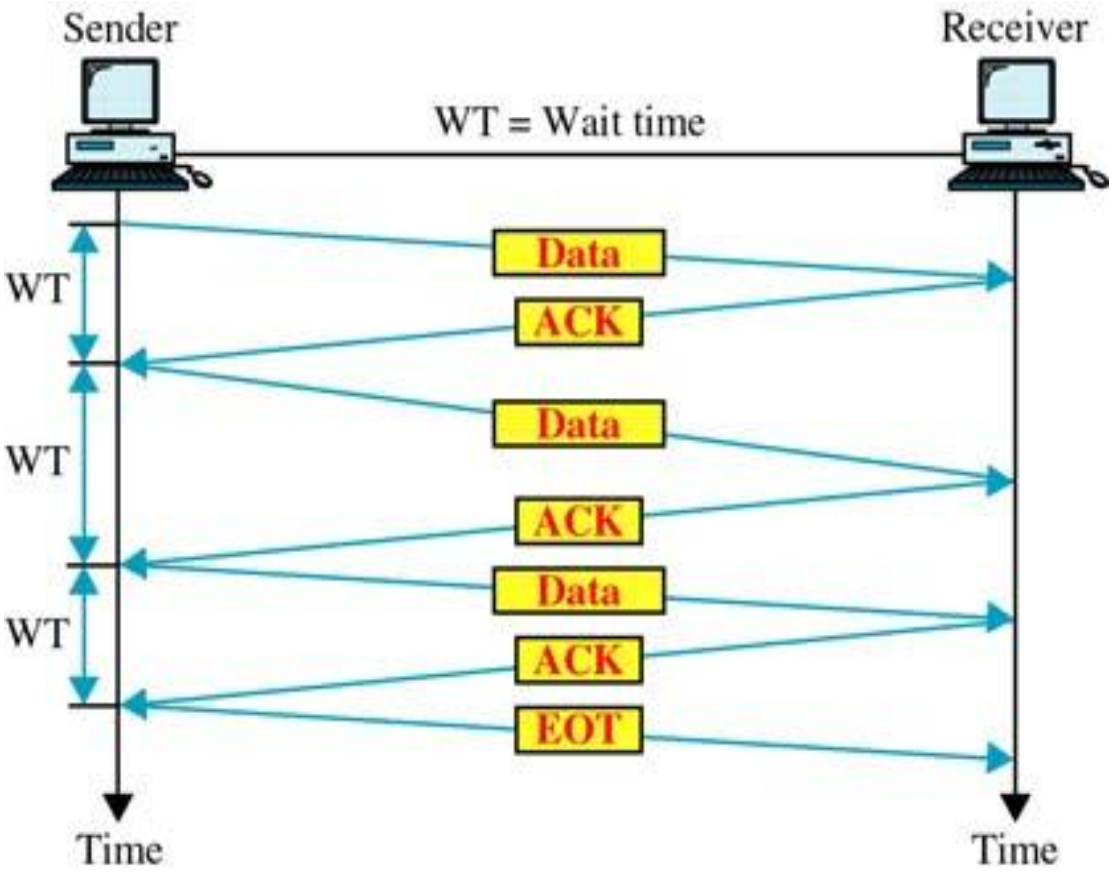
- Assumptions
  - Channel is simplex
  - Channel is error-free
  - Receiver has limited size of buffers
- No need of error control
- Flow control is necessary
  - Stop-and-wait（停止等待/停等）
    - Sender sends a frame, and wait
    - Receive sends a dummy fame(acknowledgement, ACK，应答)
    - Receiving the ACK, sender sends next frame

Protocol 2: Simplex Stop-and-Wait Protocol

```
/* Protocol 2 (stop-and-wait) also provides for a one-directional flow of data from
   sender to receiver. The communication channel is once again assumed to be error
   free, as in protocol 1. However, this time, the receiver has only a finite buffer
   capacity and a finite processing speed, so the protocol must explicitly prevent
   the sender from flooding the receiver with data faster than it can be handled. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender2(void)
{
  frame s;                          /* buffer for an outbound frame */
  packet buffer;                    /* buffer for an outbound packet */
  event_type event;                 /* frame_arrival is the only possibility */

  while (true) {
      from_network_layer(&buffer);  /* go get something to send */
      s.info = buffer;              /* copy it into s for transmission */
      to_physical_layer(&s);        /* bye bye little frame */
      wait_for_event(&event);       /* do not proceed until given the go ahead */
  }
}

void receiver2(void)
{
  frame r, s;                       /* buffers for frames */
  event_type event;                 /* frame_arrival is the only possibility */
  while (true) {
      wait_for_event(&event);       /* only possibility is frame_arrival */
      from_physical_layer(&r);      /* go get the inbound frame */
      to_network_layer(&r.info);    /* pass the data to the network layer */
      to_physical_layer(&s);        /* send a dummy frame to awaken sender */
  }
}
```

Stop and Wait

- Source transmits a frame
- Destination receives frame and replies with ACK
- Source waits for ACK before sending next frame
- Destination can stop flow by not sending ACK

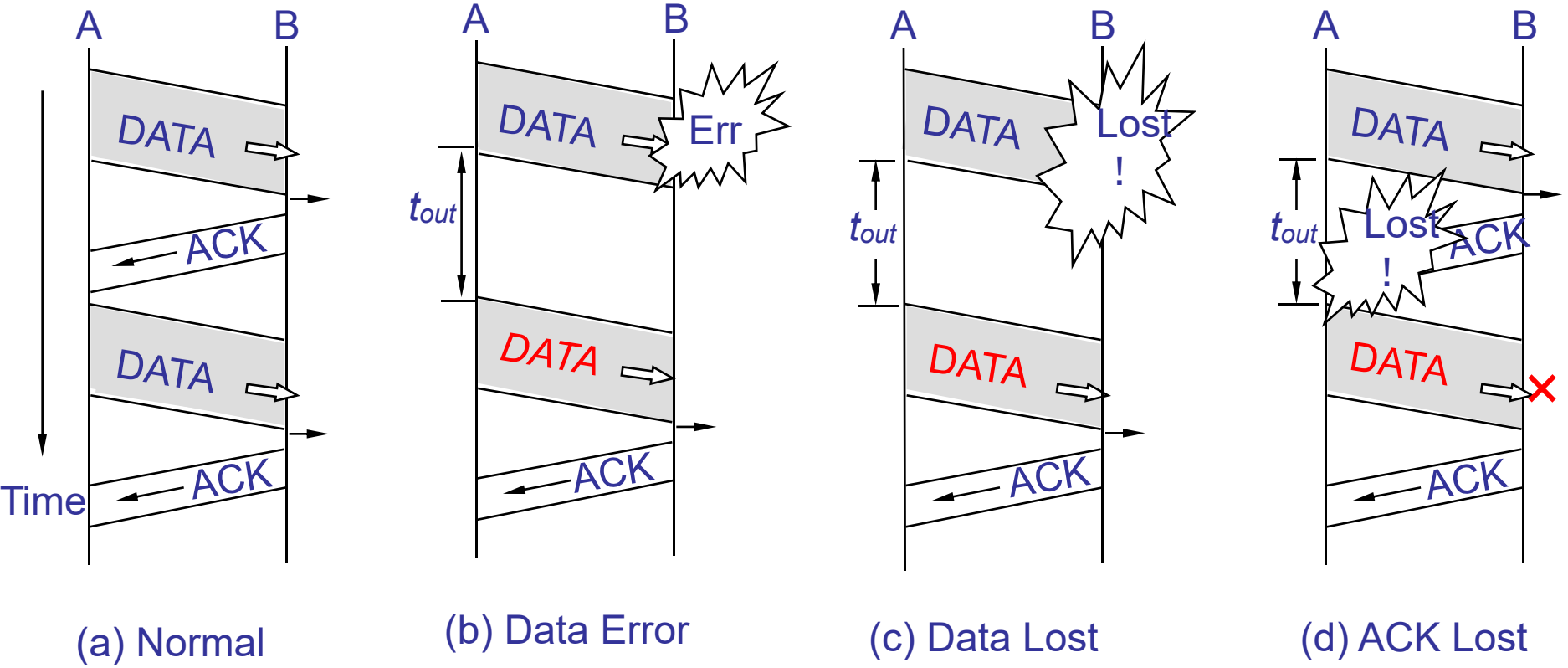


Problems of protocol 2: If channel is not reliable

- When a frame is damaged
  - Receiver detects errors with CRC and discard the frame
  - Sender retransmits the frame when timing out
- When a frame is lost
  - Data frame lost
    - Sender retransmits when timing out
  - ACK frame lost
    - Sender retransmits when timing out

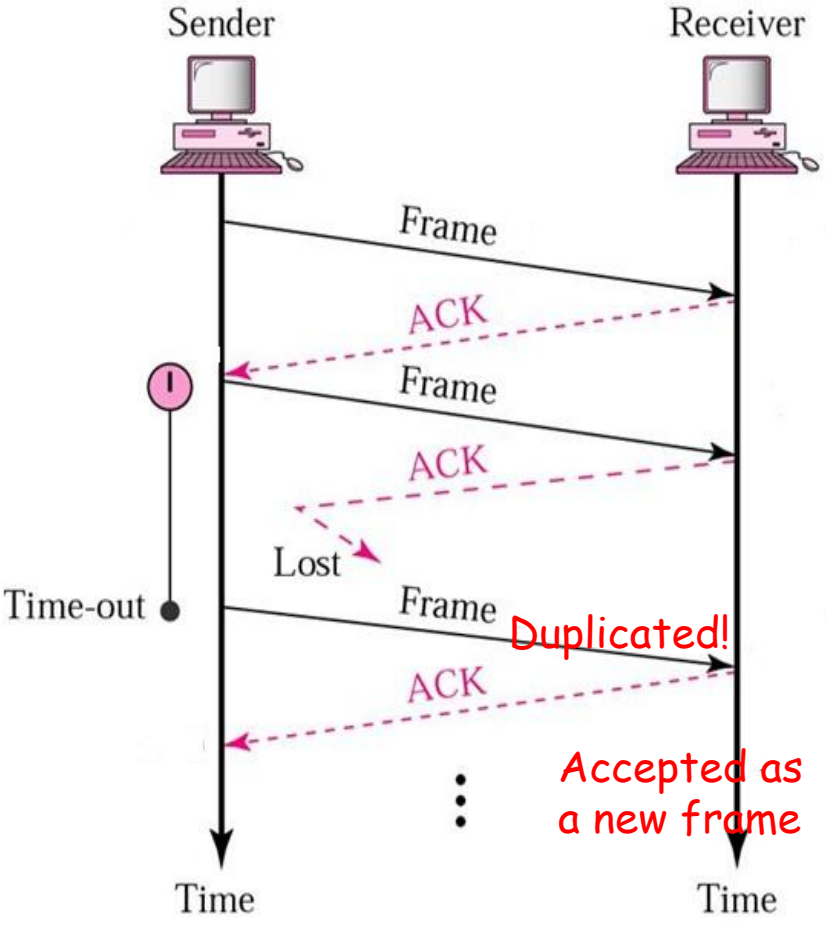
Sender: adding a timer

Stop-And-Wait



More Problems

- Case 1: Protocol failed on Receiver(**ACK lost**)
  - Sender A gives packet 1 to its data link layer. The packet is correctly received at receiver B and passed to the network layer on B. B sends an ACK frame back to A.
  - The ACK gets lost completely
  - The data link layer on A eventually times out. Not having received an acknowledgement, it sends the frame containing pack 1 again
  - The duplicate frame also arrives B and is passed to the network layer

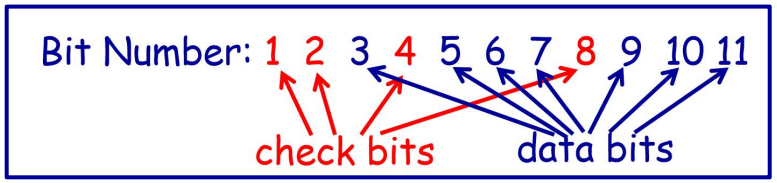


Review(1)

Error correction codes

Hamming code(single bit error)

- Check bits vs. Data bits
- $m+r+1 \le 2^r$
- example: ASCII → m=7, r=4



b1⊕ b3⊕ b5 ⊕ b7⊕ b9 ⊕ b11=0	(k=1)
b2⊕ b3 ⊕ b6⊕ b7 ⊕ b10⊕ b11=0	(k=2)
b4 ⊕ b5⊕ b6⊕ b7 =0	(k=4)
b8 ⊕ b9⊕ b10⊕ b11=0	(k=8)

Error-Detecting Codes

Parity check

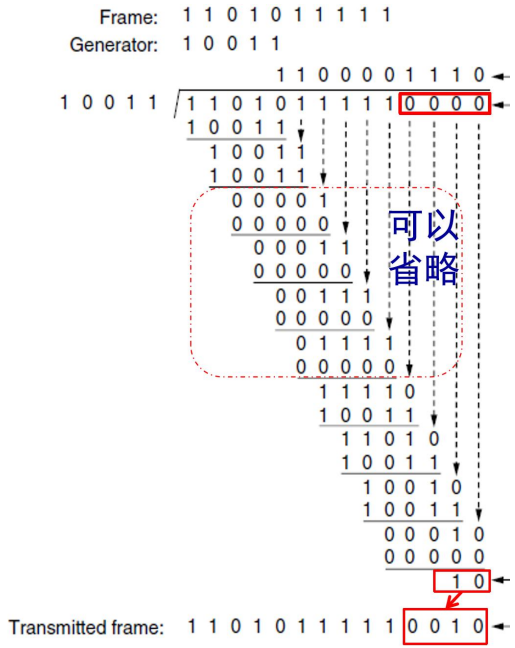
- ASCII Code: 1100001 | 1
- For Blocks(interleaving)
- Performance

code distance is 2-bit
detect *1-bit* error

length of error-detection code	*1*	*n*
error detecting capacity	*1*	*n*
Probability of being accepted	*1/2*	2^{-n}

CRC

- Sender: $R(x)=[M(x)*x^r]/G(x)$, $T(x)=M(x)*x^r+R(x)$
- Receiver: $[T(x)]/G(x)$
- Detecting capacity: $[T(x)+E(x)]/G(x)$
- Hardware：shift register circuit(移位寄存电路)



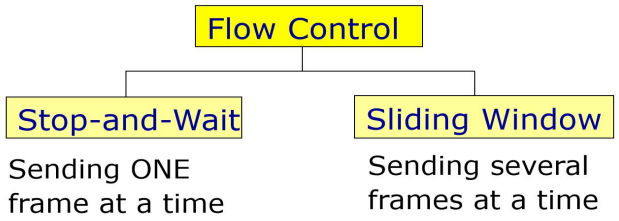
Review(2)

Error Correction

- Technologies
  - On reliable channels: ARQ (error detection code + retransmission)
  - On wireless channels: FEC (error correction code)
- Application: CDMA air interface--> FEC+ARQ

Flow control

- Rate-based flow control
- Feedback-based flow control
- Elementary Protocols
  - reliable, connection-oriented service
  - channel is simplex



	protocol 1	protocol 2	protocol 3
Assumptions	error-free, unlimited buffers	error-free, limited buffers	unreliable channel, limited buffers
Functions		Flow control: stop-and-wait	Flow control: stop-and-wait Error control: CRC+retransmission(ARQ) Data seq_number (n=1), frame_timer, buffer(sender)
Frames	Data	Data, Ack	Data, Ack
Acknowledges		Ack	Ack

Solutions: using Sequence Number

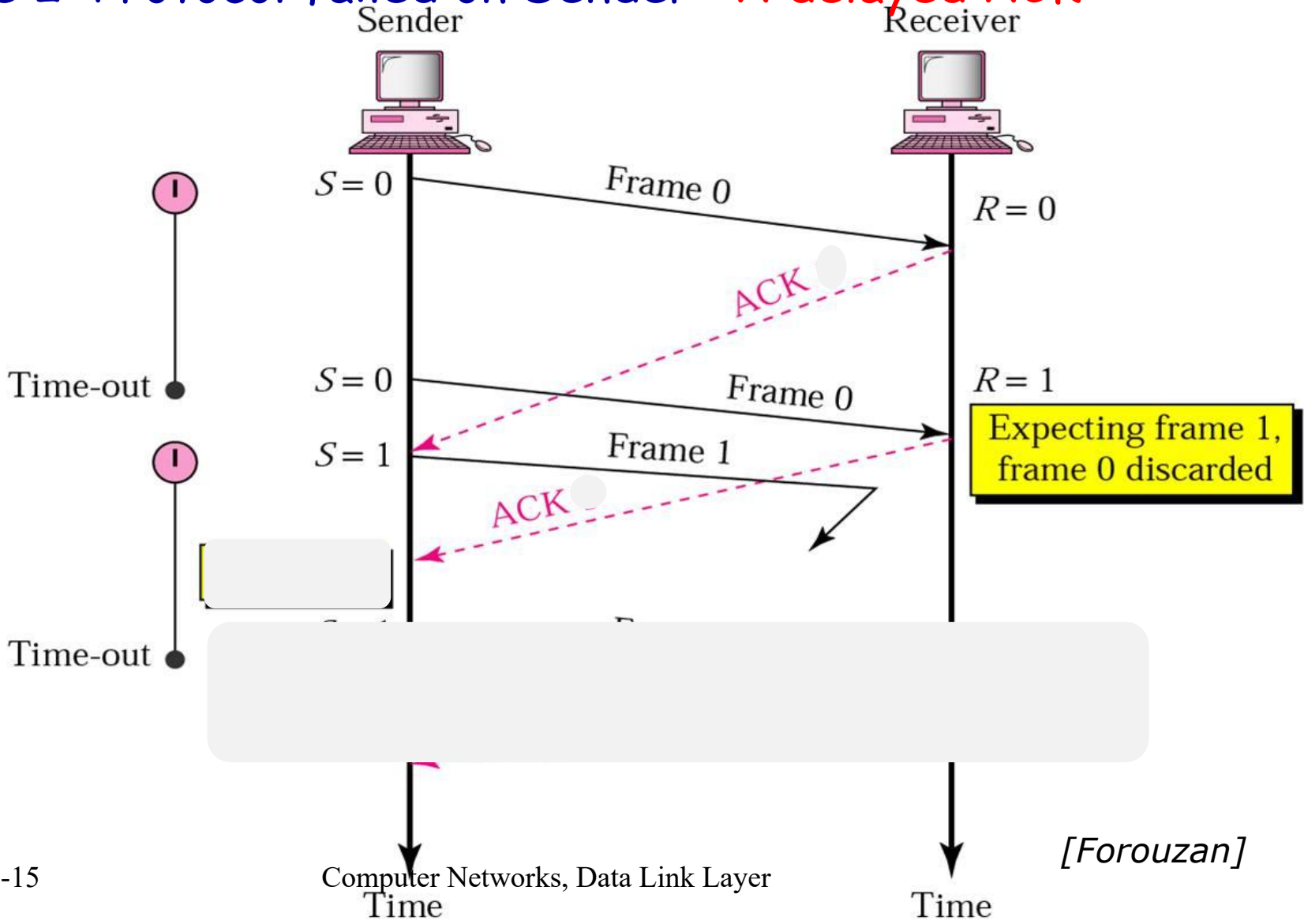
- Distinguish a frame that it is seeing for the first time from a retransmission
  - Put a sequence number(序号) in the header of each frame
- What is the minimum number of bits needed for the sequence number?
  - Data Link Layer：1 bit, with 2 sequence numbers (0 and 1) in all
  - Transport Layer: needing a much larger number space, eg. 16 bits

→ ARQ: Automatic Repeat reQuest (自动请求重传)

when & which

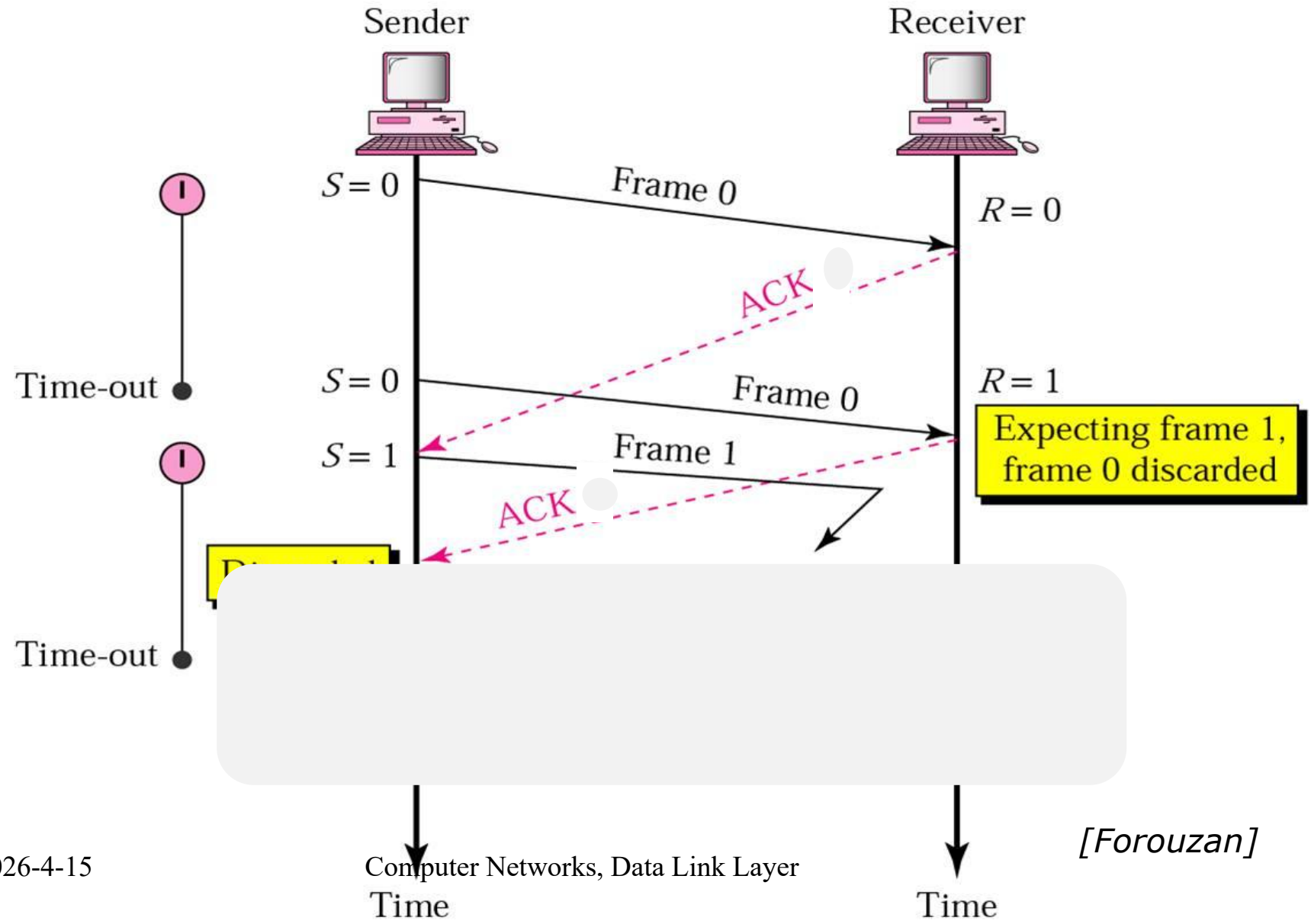
Case 2: Delayed ACK

- Case 2: Protocol failed on Sender--A delayed ACK



[Forouzan]

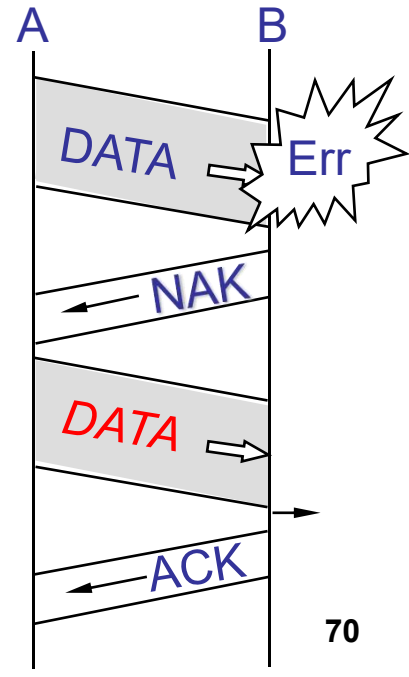
- Why ACK needs SN?



[Forouzan]

Procedure of ARQ

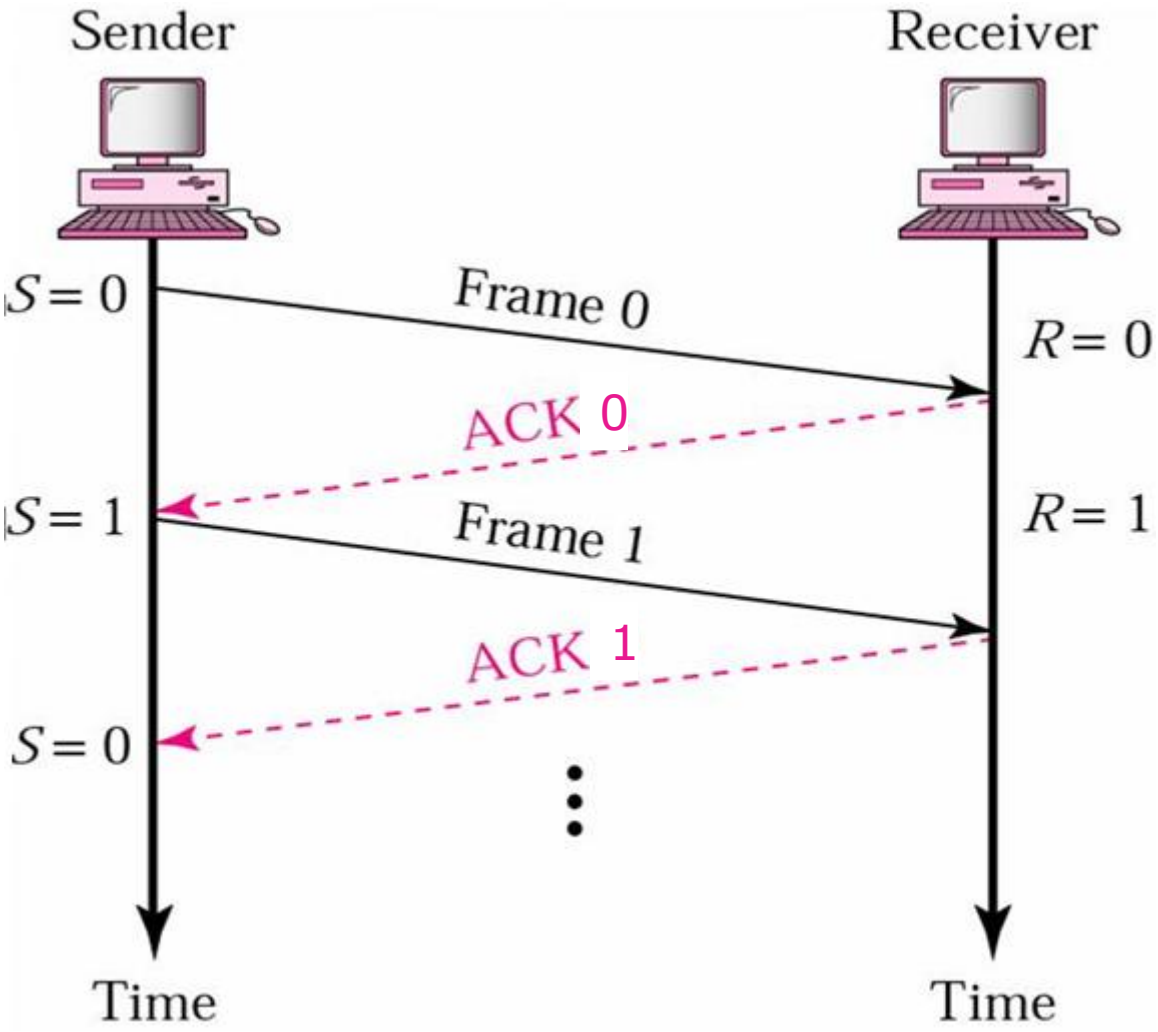
- Error detection
  - Bad CRC/FCS (Frame Check Sequence) or lost frame
- Acknowledgment
  - Positive acknowledgment (确认), ACK returned by receiver - may not be an ACK for each frame
  - Negative acknowledgement(否认, optional), NAK returned by receiver and retransmission by sender
- Retransmission by sender after timeout (or NAK)
  - Sequence number (in frame and Ack) used to detect duplicated frame or ack
  - Buffers (sender_buffer)



About Timer

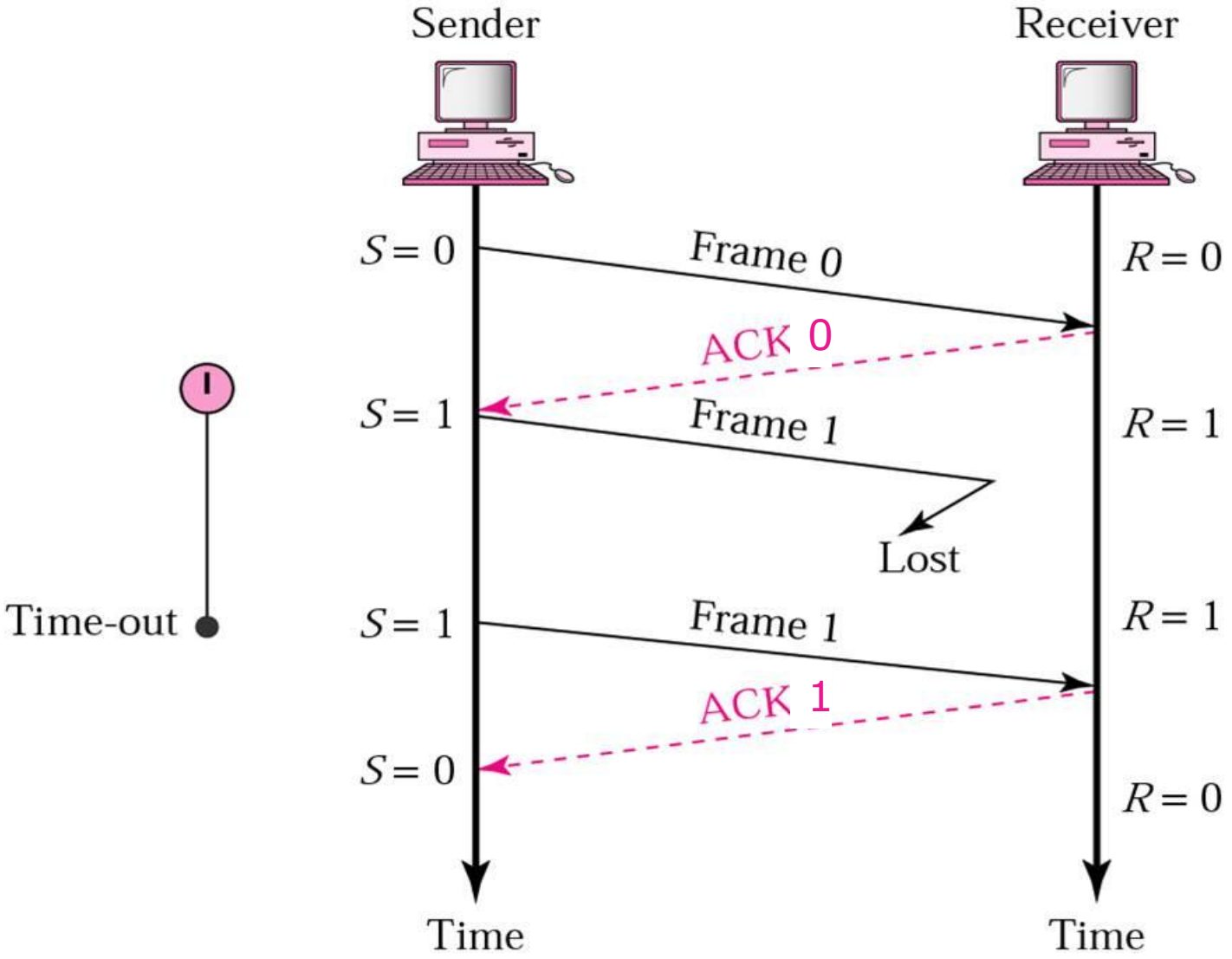
- Timeout
  - If the timeout interval is set too short, the sender will transmit unnecessary frames. While these extra frames will not affect the correctness of the protocol, they will hurt performance
- About NAK
  - Is NAK mandatory?
  - Good things using NAK? Fast Retransmission

Stop-and-Wait ARQ: Normal Operation



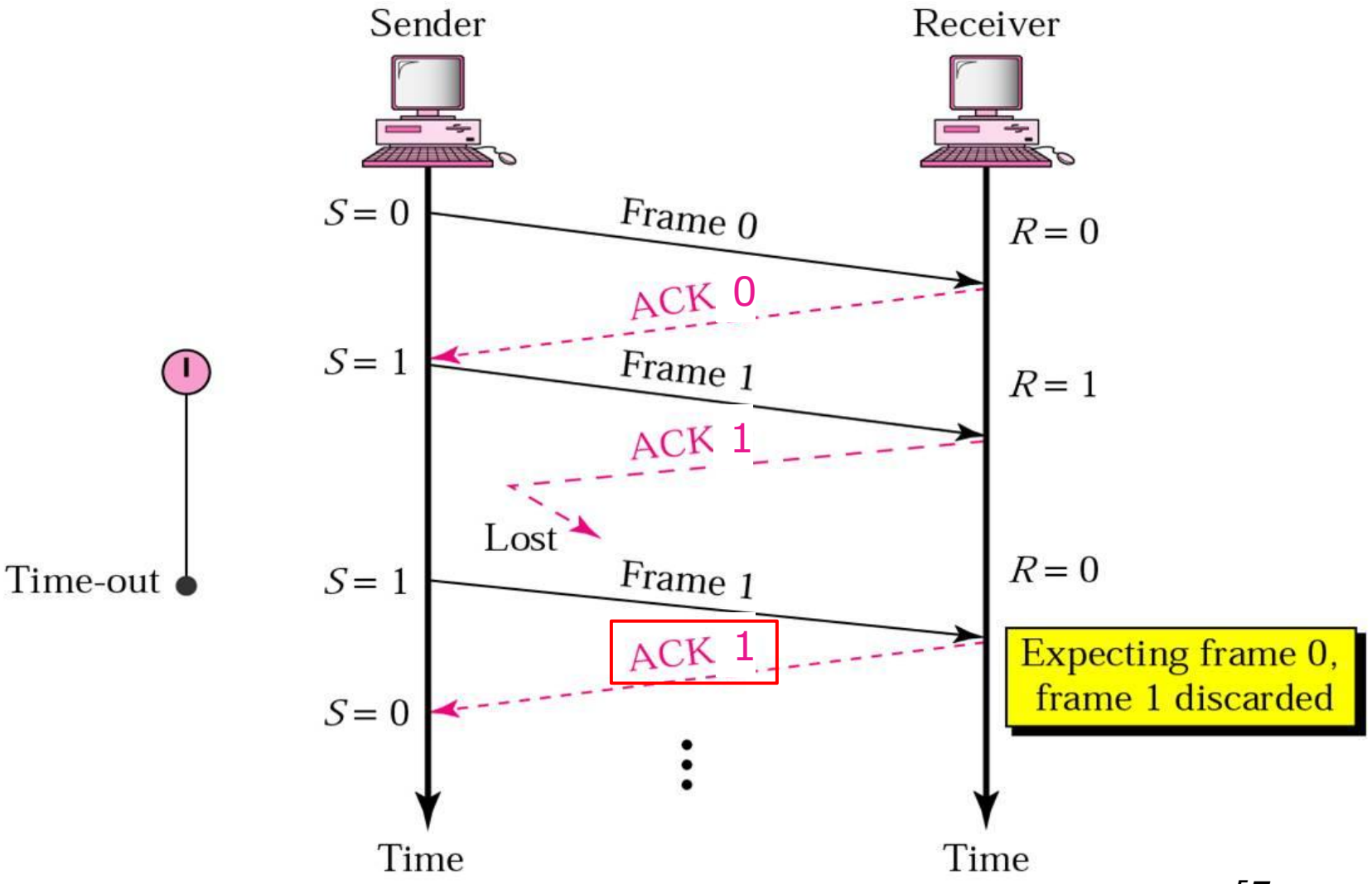
[Forouzan]

Case 1: Frame gets lost



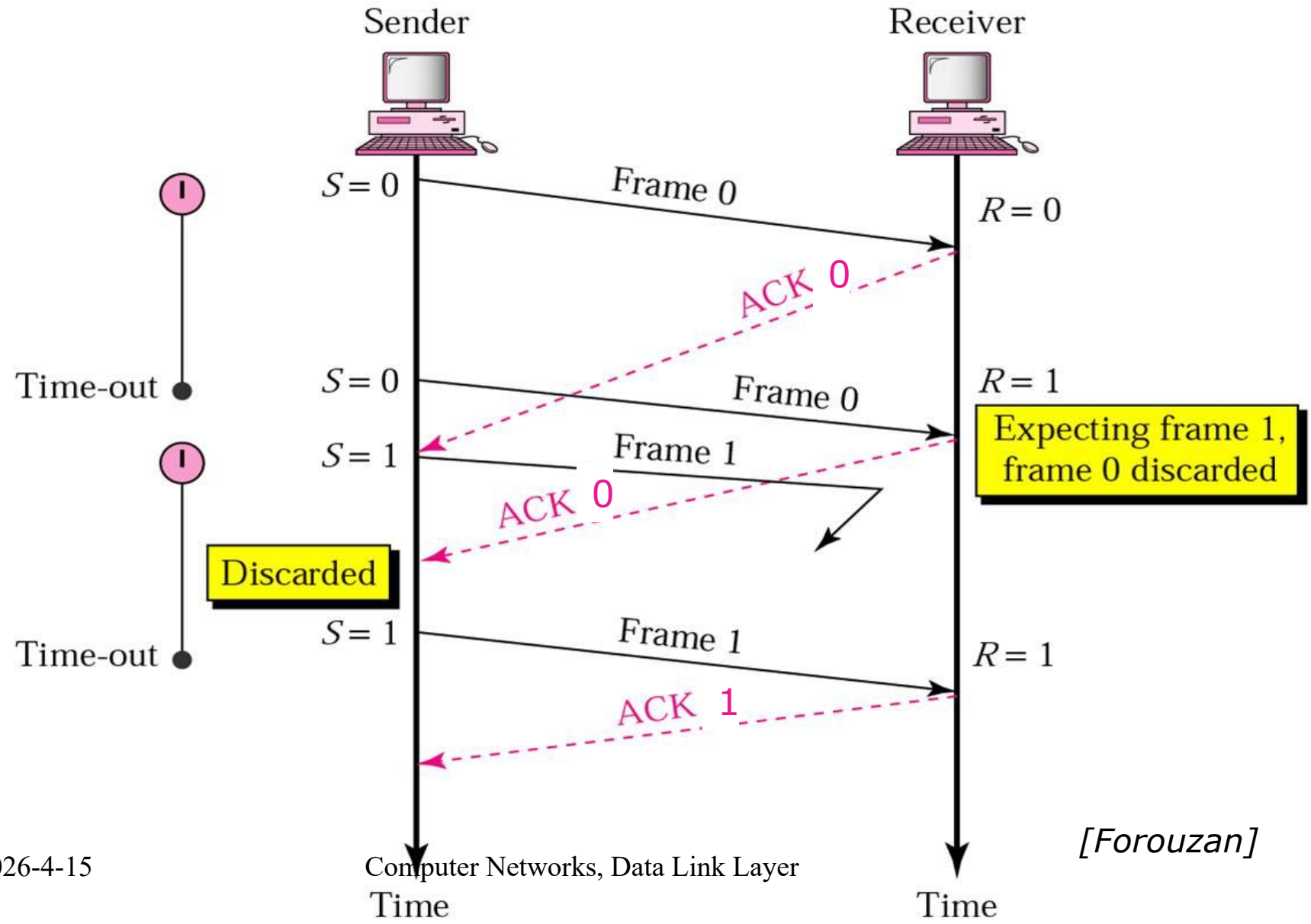
[Forouzan]

Case 2: ACK gets lost



[Forouzan]

Case 3: Delayed ACK



[Forouzan]

Summary

Elementary Protocols

- reliable transmission
- channel is simplex

	protocol 1	protocol 2	protocol 3
Assumptions	error-free unlimited buffers	error-free limited buffers	channel is unreliable limited buffers
Functions		Flow control: stop-and-wait	Error control: CRC+retransmission Flow control: stop-and-wait
Frames	Data	Data, Ack	Data, Ack
Acknowledges	X	Ack	Ack
seq-number	X	X	Data seq_number (n=1) Ack seq_number
Timers	X	X	frame_timer
Buffers	X	X	sender_buffer
Protocol PDU	**data**	**data**	**data** + Data seq_number
			Ack seq_number

3.3.3 Protocol 3: Positive ACK with Retransmission(PAR)

- Assumptions
  - Channel is simplex
  - Channel is unreliable
  - Receiver has limited size of buffers
- What are enhanced?
  - Sequence number in frame and ACK
  - ARQ: Time out and retransmission

```
    从网络层获取一个分组放入buffer
    发送buffer中的数据，新启动定时器
label1:
    wait_for_event()
    switch (event) {
    case 定时器超时：
        发送buffer中的数据（重传的数据），新启动定时器
    case 收到ACK帧：
        if（ack序号正确） {
            关闭旧定时器
            从网络层获取下一个分组放入buffer
            发送buffer中的数据，新启动定时器
        }
    }
    goto label1
```

```
    从网络层获取一个分组放入buffer
    发送buffer中的数据，新启动定时器
label1:
    wait_for_event()
    switch (event) {
    case 定时器超时：
        发送buffer中的数据（重传的数据），新启动定时器
    case 收到ACK帧：
        if（ack序号正确） {
            关闭旧定时器
            从网络层获取下一个分组放入buffer
            发送buffer中的数据，新启动定时器
        }
    }
    goto label1
```

```
    从网络层获取一个分组放入buffer
    发送buffer中的数据，启动定时器
while(true) {
    wait_for_event()
    if (收到ACK帧) {
        if（ack序号正确） {
          关闭旧定时器
          从网络层获取下一个分组放入buffer
        }
    }
    发送buffer中的数据，启动定时器
}
```

```
    从网络层获取一个分组放入buffer
while(true) {
    发送buffer中的数据，启动定时器
    wait_for_event()
    if (收到ACK帧) {
        if（ack序号正确） {
            关闭旧定时器
            从网络层获取下一个分组放入buffer
         }
    }
}
```

```
    从网络层获取一个分组放入buffer
    next_frame_to_send = 0;

while(true) {
    发送buffer中的数据，启动定时器
    帧序号填写为next_frame_to_send
    wait_for_event()
    if (收到ACK帧) {
        if（ack序号正确） {
            关闭旧定时器
            从网络层获取下一个分组放入buffer
            inc(next_frame_to_send)
        }
    }
}
```

（next_frame_to_send 记录目前缓冲在buffer中的数据的编号。
buffer中装入下一个数据， next_frame_to_send 加1）

```
/* Protocol 3 (par) allows unidirectional data flow over an unreliable channel. */

#define MAX_SEQ 1                                   /* must be 1 for protocol 3 */
typedef enum  {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
  seq_nr next_frame_to_send;                        /* seq number of next outgoing frame */
  frame s;                                          /* scratch variable */
  packet buffer;                                    /* buffer for an outbound packet */
  event_type event;

  next_frame_to_send = 0;                           /* initialize outbound sequence numbers */
  from_network_layer(&buffer);                      /* fetch first packet */
  while (true) {
       s.info = buffer;                             /* construct a frame for transmission */
       s.seq = next_frame_to_send;                  /* insert sequence number in frame */
       to_physical_layer(&s);                       /* send it on its way */
       start_timer(s.seq);                          /* if answer takes too long, time out */
       wait_for_event(&event);                      /* frame_arrival, cksum_err, timeout */
       if (event == frame_arrival) {
            from_physical_layer(&s);                /* get the acknowledgement */
            if (s.ack == next_frame_to_send) {
                 stop_timer(s.ack);                 /* turn the timer off */
                 from_network_layer(&buffer);       /* get the next one to send */
                 inc(next_frame_to_send);           /* invert next_frame_to_send */
            }
       }
  }
}
```


Sender

```
frame_expected=0
while(true) {
    wait_for_event()
    switch(event) {
    case 坏帧：
        do_nothing
    case 收到校验和正确的数据帧:
        if（序号==frame_expected） {
            向网络层上交分组
            回ACK（序号为frame_expected)
            inc(frame_expected)
        }  else {
            回ACK（序号为frame_expected-1)
        }
    }
}
```

```
frame_expected=0
while(true) {
    wait_for_event()
    if (event==收到校验和正确的数据帧) {
        if（序号==frame_expected） {
            向网络层上交分组
            inc(frame_expected)
            回ACK（序号为frame_expected-1)
        } else {
            回ACK（序号为frame_expected-1)
        }
    }
}
```

```
frame_expected=0
while(true) {
    wait_for_event()
    if (event==收到校验和正确的数据帧) {
        if（序号==frame_expected） {
            向网络层上交分组
            inc(frame_expected)
        }
        回ACK（序号为frame_expected-1)
    }
}
```

```
void receiver3(void)
{
  seq_nr frame_expected;
  frame r, s;
  event_type event;

  frame_expected = 0;
  while (true) {
       wait_for_event(&event);                  /* possibilities: frame_arrival, cksum_err */
       if (event == frame_arrival) {            /* a valid frame has arrived. */
            from_physical_layer(&r);            /* go get the newly arrived frame */
            if (r.seq == frame  expected) {     /* this is what we have been waiting for. */
                 to_network_layer(&r.info);     /* pass the data to the network layer */
                 inc(frame_expected);           /* next time expect the other sequence nr */
            }
            s.ack = 1 – frame_expected;         /* tell which frame is being acked */
            to_physical_layer(&s);              /* send acknowledgement */
       }
  }
}
```

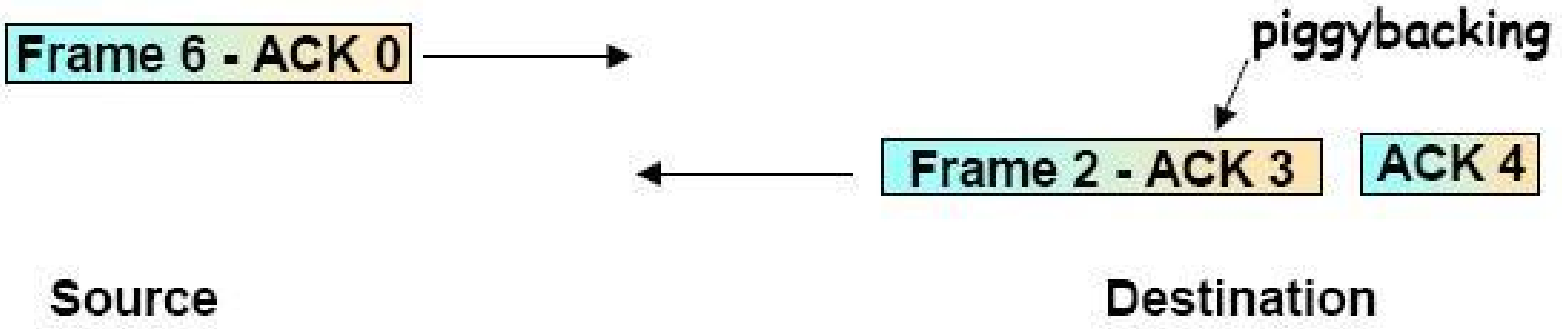
A positive acknowledgement with retransmission protocol.

Basic Flow Control Protocols

- 3.3 Elementary Protocols
  - 3.3.1 A Utopian Simplex Protocol
  - 3.3.2 A Simplex Stop-and-Wait Protocol for an Error-Free Channel
  - 3.3.3 A Simplex Protocol for a Noisy Channel
- *3.4 Sliding Window Protocols*
  - 3.4.1 A One-Bit Sliding Window Protocol
  - 3.4.2 A Protocol Using Go Back N
  - 3.4.3 A Protocol Using Selective Repeat
  - Performance of sliding window protocols

Piggybacking(捎带应答/确认)

- Full-duplex(全双工) data transmission
- ACK: separate control frame
- Piggybacking(捎带应答/确认)
  - When a data frame arrives, the receiver restrains ACK and waits until the network layer passes it the next packet
  - ACK is attached to the outgoing data frame (using the ACK field in the frame header)



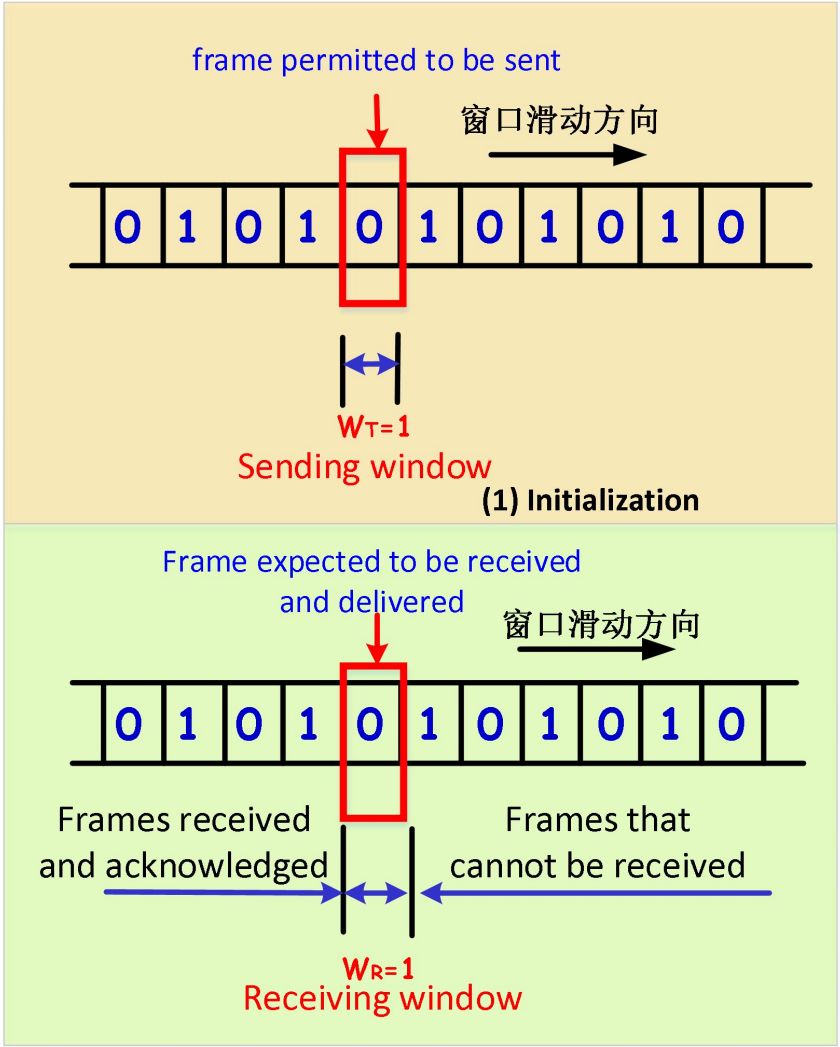
Piggybacking

- Advantage of using piggybacking
  - A better use of the available channel bandwidth
  - A lighter processing load at the receiver
- Introduce a complication
  - How long should the data link layer wait for a packet onto which to piggyback the acknowledgement?
  - Ack Timer
    - ack timer vs. frame timer

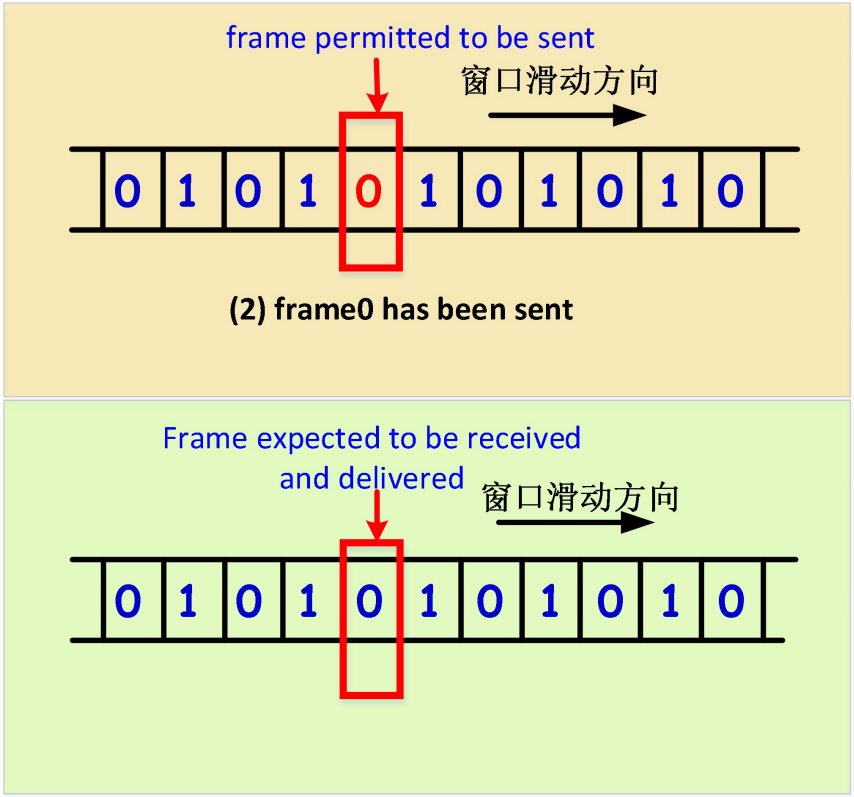
3.4.1 protocol4：Sliding Window of Size 1

- A sliding window of size 1, with a 1-bit sequence number.
  - Full-duplex(全双工) data transmission: Piggybacking
  - sequence number:n=1, [0, 1]
  - sending window W_T=1
  - receiving window W_R=1

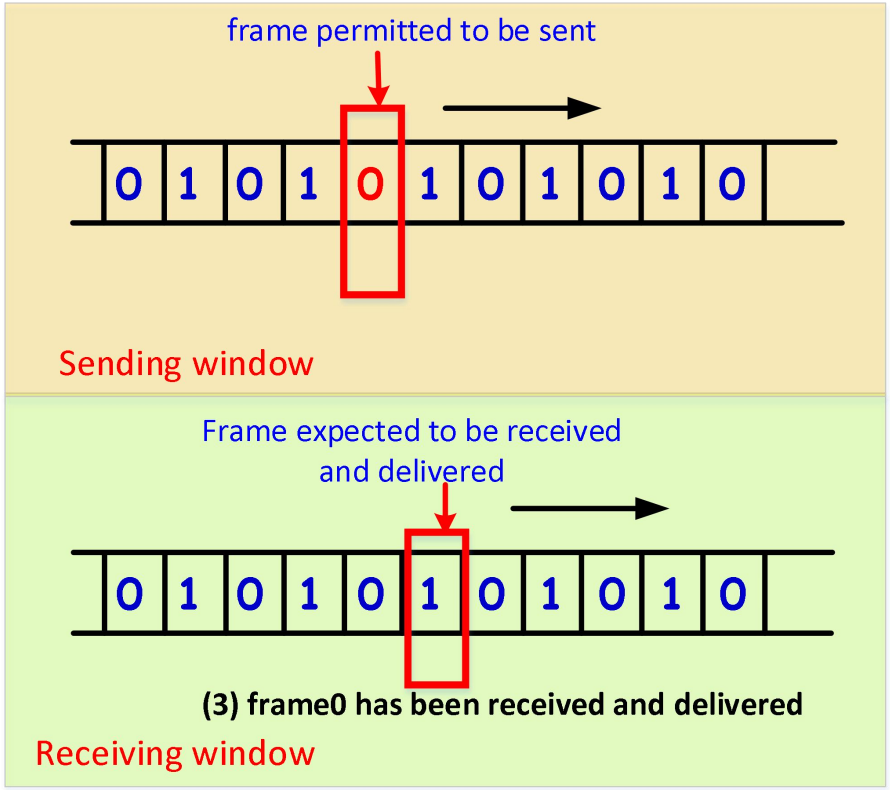
Sliding Window of Size 1



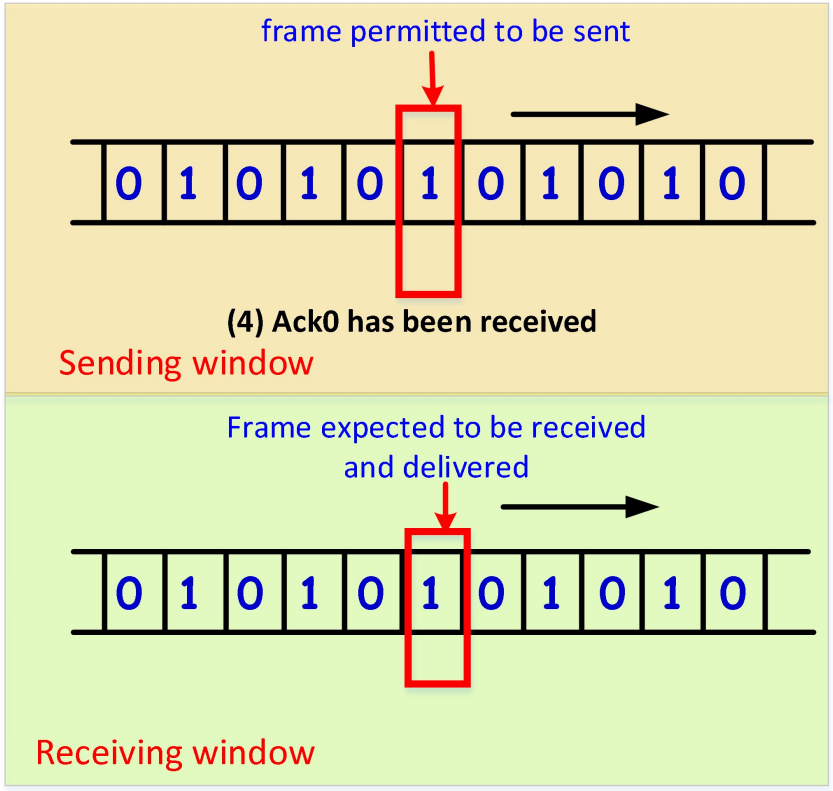
Sliding Window of Size 1



Sliding Window of Size 1



Sliding Window of Size 1



next_frame_to_send

frame_expected

One-Bit Sliding Window Protocol (1)

```
/* Protocol 4 (sliding window) is bidirectional. */
#define MAX_SEQ 1                                   /* must be 1 for protocol 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void protocol4 (void)
{
  seq_nr next_frame_to_send;                        /* 0 or 1 only */
  seq_nr frame_expected;                            /* 0 or 1 only */
  frame r, s;                                       /* scratch variables */
  packet buffer;                                    /* current packet being sent */
  event_type event;

  next_frame_to_send = 0;                           /* next frame on the outbound stream */
  frame_expected = 0;                               /* frame expected next */
  from_network_layer(&buffer);                      /* fetch a packet from the network layer */
  s.info = buffer;                                  /* prepare to send the initial frame */
  s.seq = next_frame_to_send;                       /* insert sequence number into frame */
  s.ack = 1 – frame_expected;                       /* piggybacked ack */
  to_physical_layer(&s);                            /* transmit the frame */
  start_timer(s.seq);                               /* start the timer running */
```

s.seq初始值=0

s.ack初始值=1

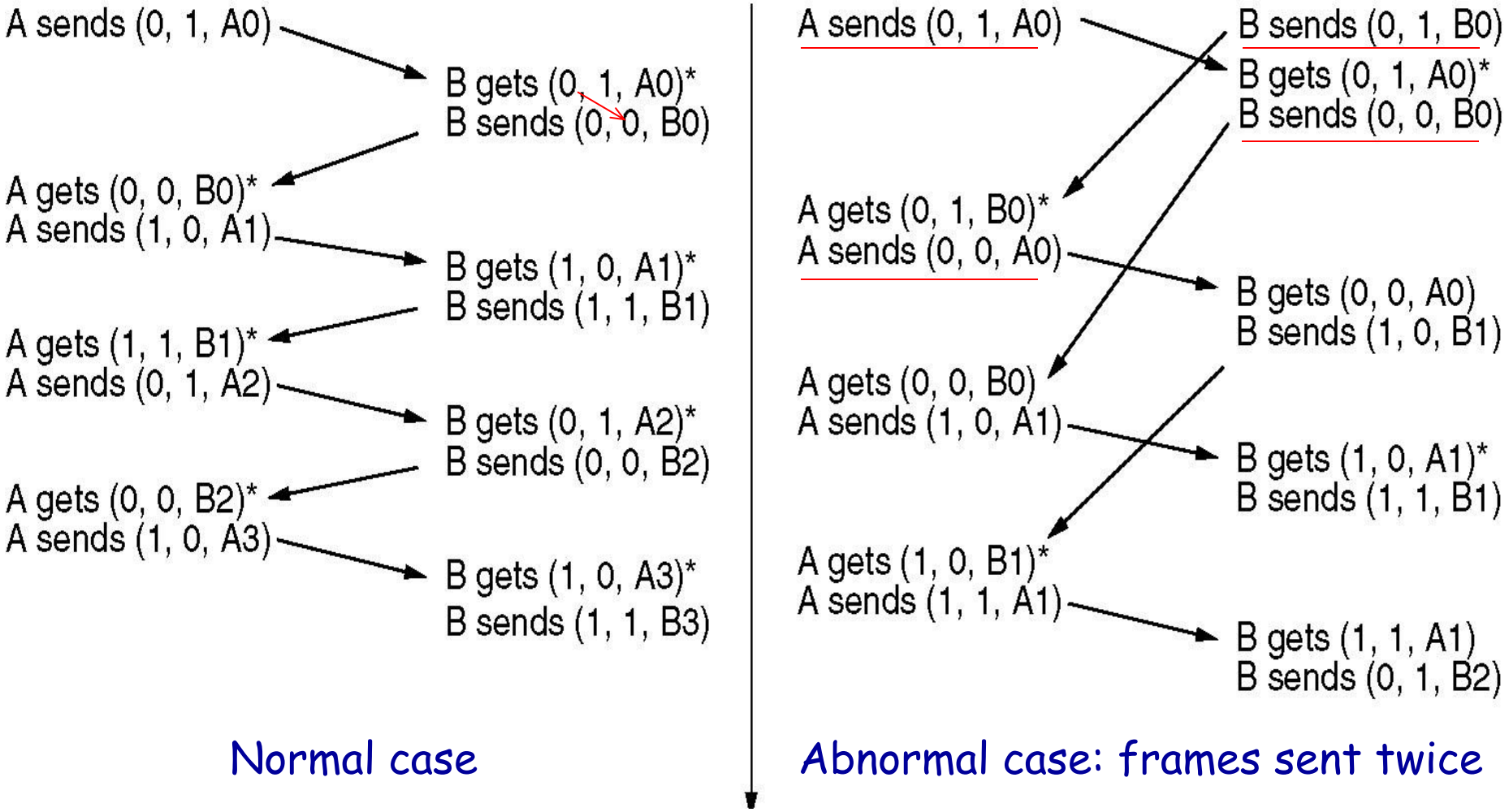
One-Bit Sliding Window Protocol (2)

```
while (true) {
    wait_for_event(&event);                 /* frame_arrival, cksum_err, or timeout */
    if (event == frame_arrival) {           /* a frame has arrived undamaged. */
        from_physical_layer(&r);            /* go get it */
        if (r.seq == frame_expected) {      /* handle inbound frame stream. */
            to_network_layer(&r.info);      /* pass packet to network layer */
            inc(frame_expected);            /* invert seq number expected next */
        }
        if (r.ack == next_frame_to_send) {  /* handle outbound frame stream. */
            stop_timer(r.ack);              /* turn the timer off */
            from_network_layer(&buffer);    /* fetch new pkt from network layer */
            inc(next_frame_to_send);        /* invert senderís sequence number */
        }
    }
    s.info = buffer;                        /* construct outbound frame */
    s.seq = next_frame_to_send;             /* insert sequence number into it */
    s.ack = 1 – frame_expected;             /* seq number of last received frame */
    to_physical_layer(&s);                  /* transmit a frame */
    start_timer(s.seq);                     /* start the timer running */
  }
}
```

收到正确序号数据帧，提交网络层，接收窗口前移1格

收到正确序号ack帧(上帧已正确到达)，获取网络层新包，发送窗口前移一格

Peculiar Situation of Protocol 4: Simultaneously Sending



Normal case

Abnormal case: frames sent twice

The notation is *(seq, ack, packet)*

* indicates where a network layer accepts a packet

#ack= #frame-just-received

Problem of Protocol 4: low efficiency

- Consider a 50-kbps satellite channel with a 500-msec round-trip propagation delay, send 1000-bit frames
  - t=0: sender starts sending, (1000/50k)=20ms
  - t=20 msec: the frame has been completely sent
  - t=270 msec: frame fully arrived at the receiver
  - t=520 msec: ACK arrived back at the sender, under the best of circumstances (no waiting in the receiver and a short acknowledgement frame).

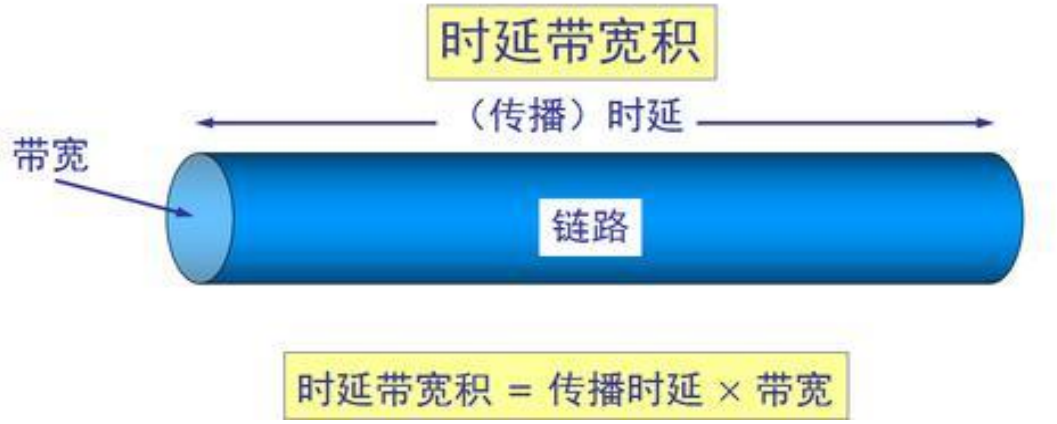
- Conclusion
  - Sender was blocked during 500/520 or 96% of the time. only 4% of the available bandwidth was used

Sliding-Window(滑动窗口) Flow Control: Principles

- bandwidth-delay product(带宽时延积)
- Allow multiple frames to be in transit at a time
- frame sequence number bounded by size of field (n)
  - Frames are numbered modulo 2^n (0…2^n-1)
  - Window size does not have to be max allowed($1 \leq W < 2^n$)

- **带宽时延积(bandwidth-delay product)：表示充满整个链路的比特数，描述信道的容量性能.**

Bandwidth-Delay Product = propagation delay*bandwidth

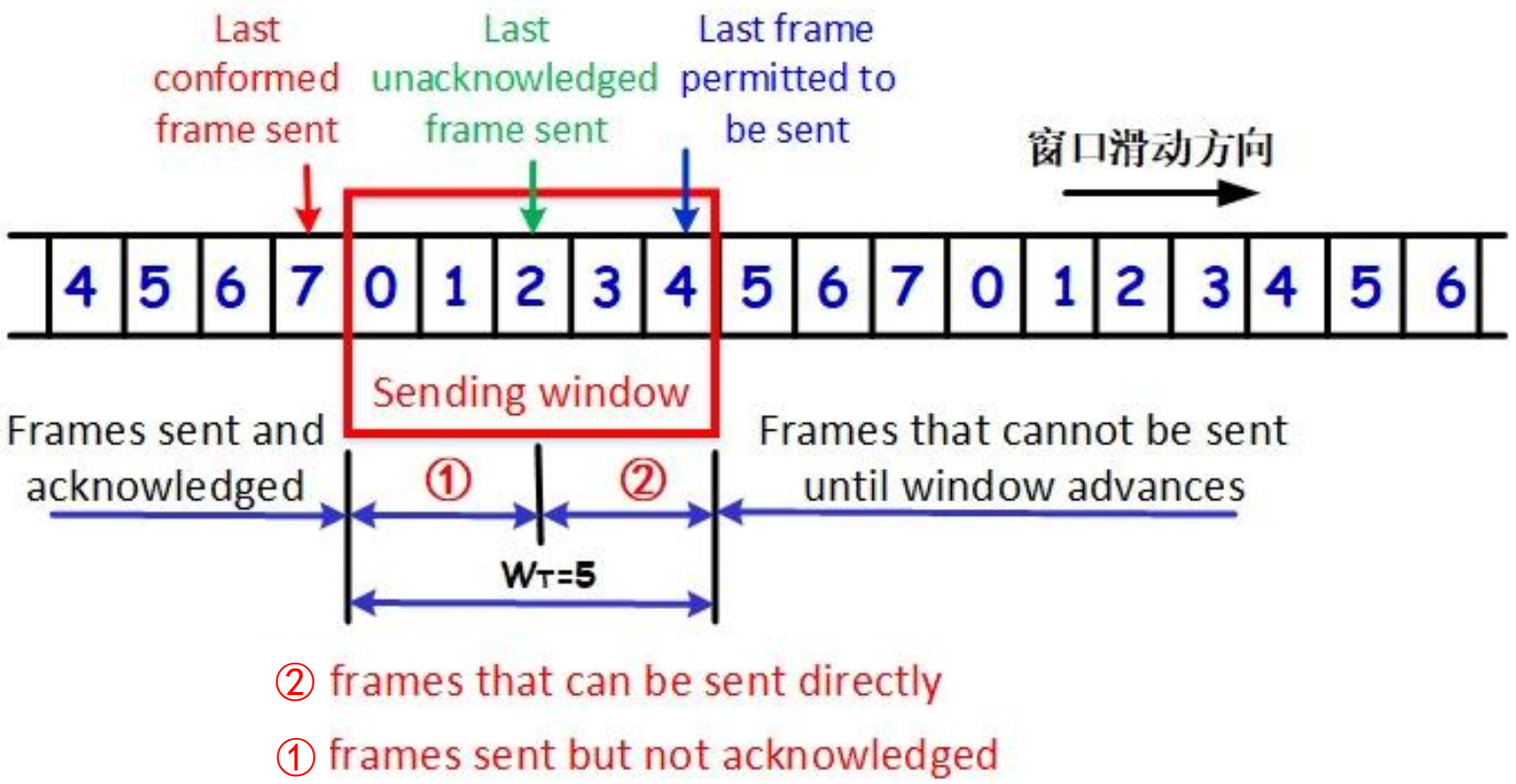


[谢]

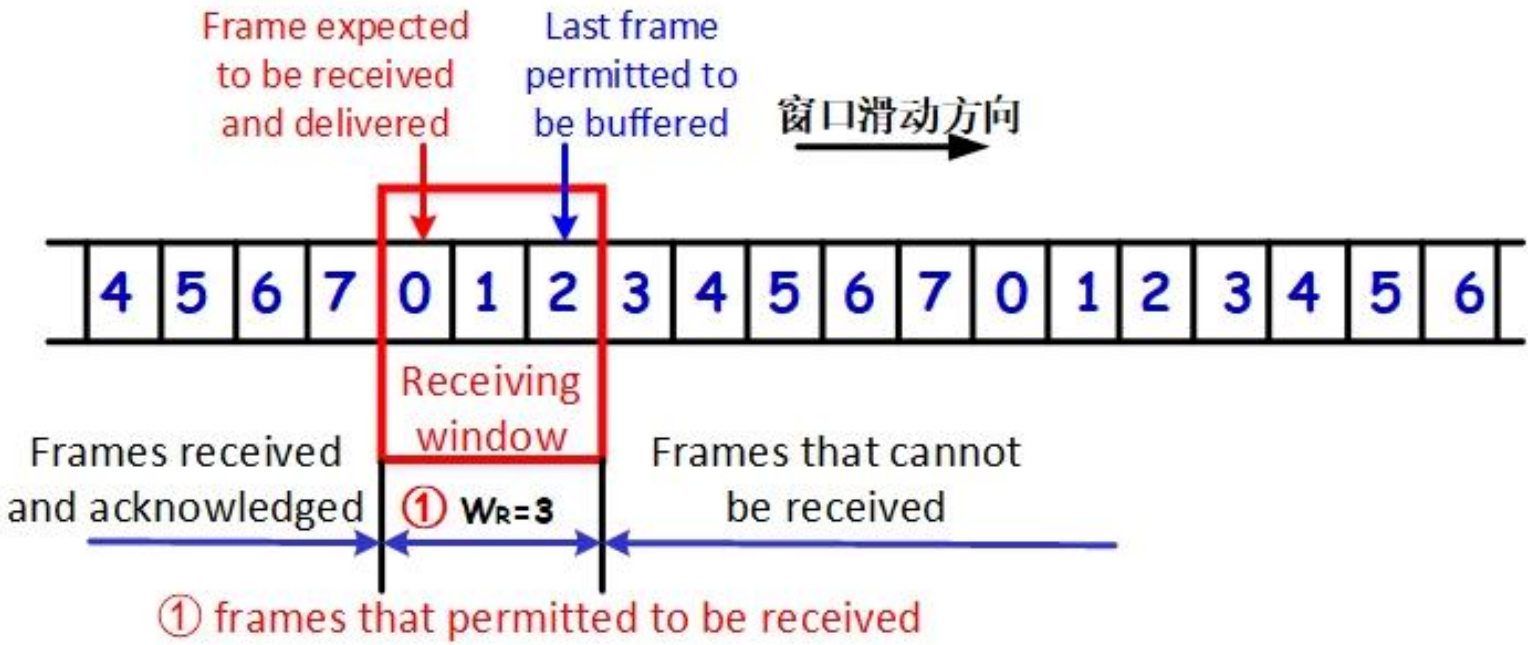
Sending window & Receiving window

- Sender can send up to W_T frames without ACK
  - W is the sending window size
- Sending window
  - At any instant of time, the sender maintains a set of sequence numbers corresponding to frames it is permitted to send.
  - These frames are said to fall within the sending window
- Receiver may be able to buffer multiple(W_R) frames
- Receiving window
  - Receiver also maintains a receiving window corresponding to the set of frames it is permitted to accept
- W_T vs. W_R

Sending window



Receiving window



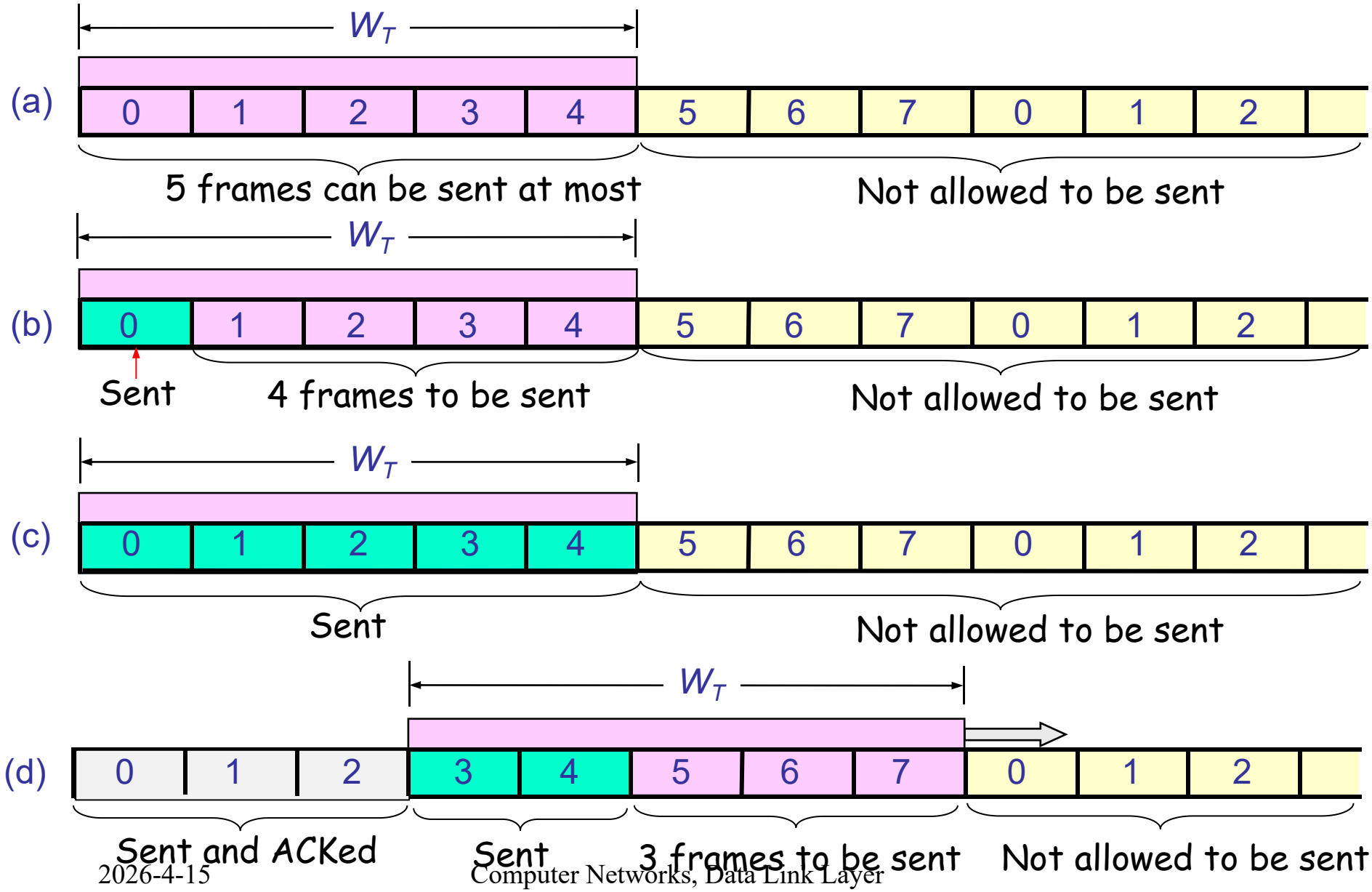
Basic Flow Control Protocols

- 3.3 Elementary Protocols
  - 3.3.1 An Unrestricted Simplex Protocol
  - 3.3.2 A Simplex Stop-and-Wait Protocol
  - 3.3.3 A Simplex Protocol for a Noisy Channel
- 3.4 Sliding Window Protocols
  - 3.4.1 A One-Bit Sliding Window Protocol
  - *3.4.2 A Protocol Using Go Back N*
  - 3.4.3 A Protocol Using Selective Repeat
  - Performance of sliding window protocols

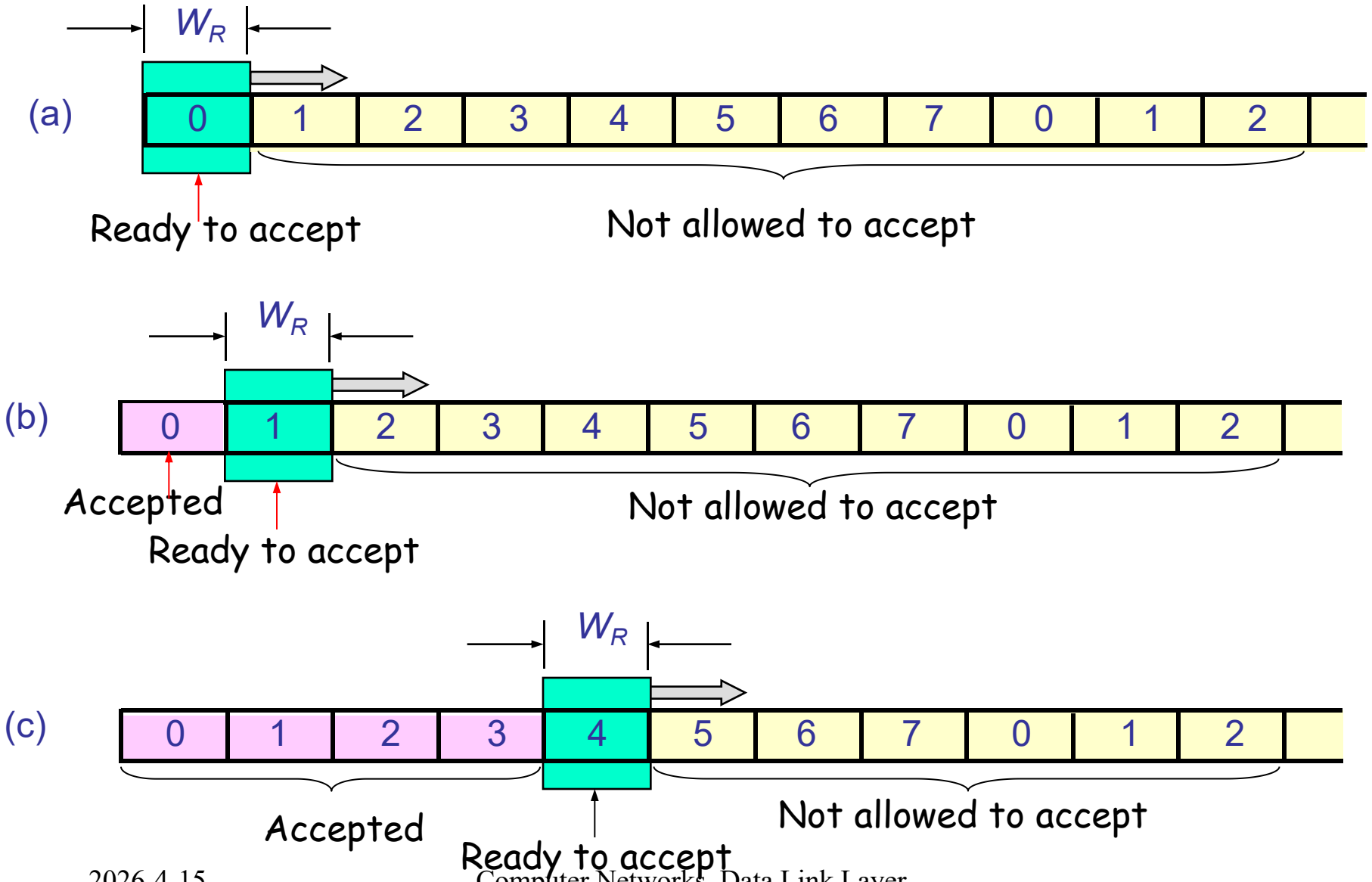
3.4.2 Go Back N（回退N步）：Principles

- Based on sliding window
  - Sending window: buffers for outstanding frames
  - Receiving window: buffers to receive frames
- Use window to control number of outstanding frames
  - Sender is allowed to **send multiple frames** before receiving ACK: pipelining(流水线), (W_T>1)
- Receiving window size is 1 (W_R=1)

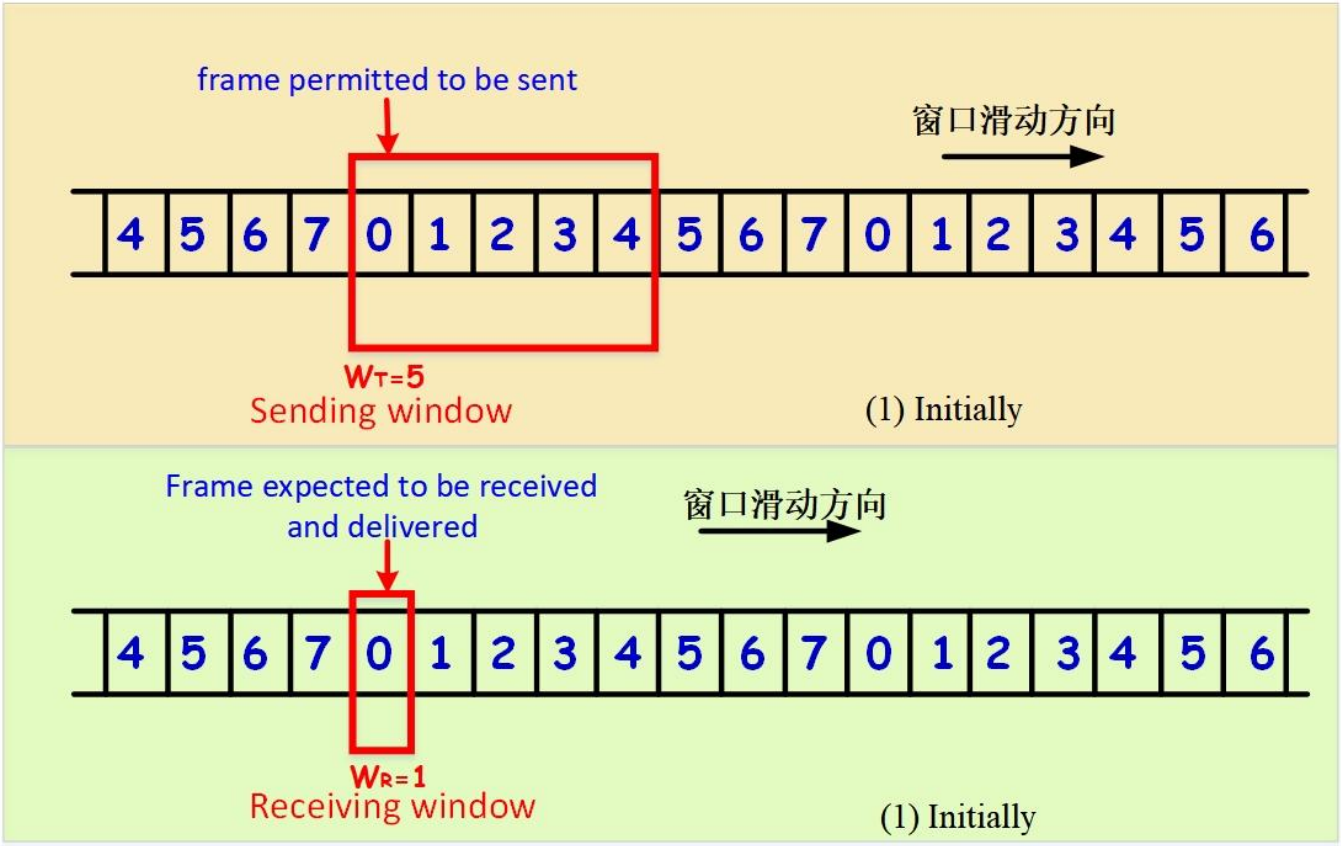
Idea of Sending Window in GBN: W_T>1



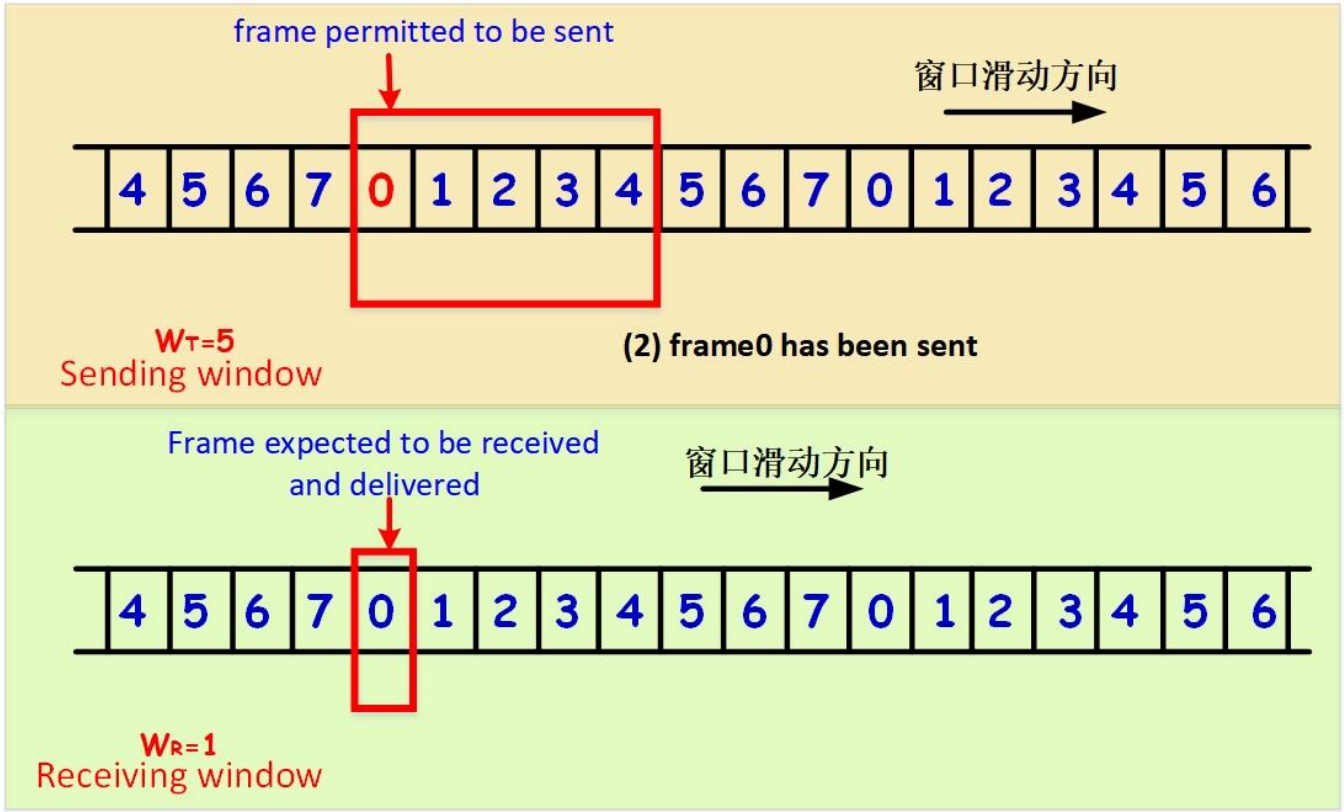
Idea of Receiving Window in GBN: W_R=1



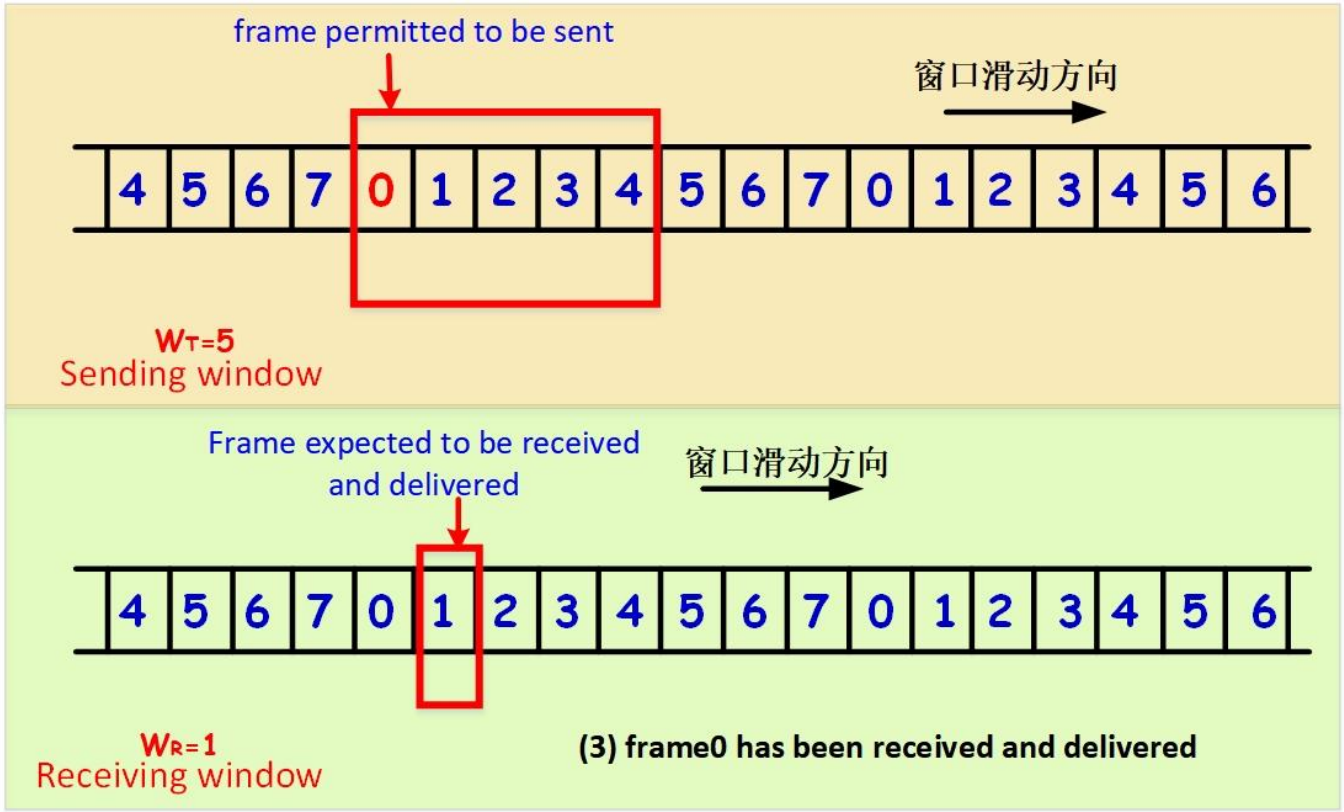
GBN Example(1)



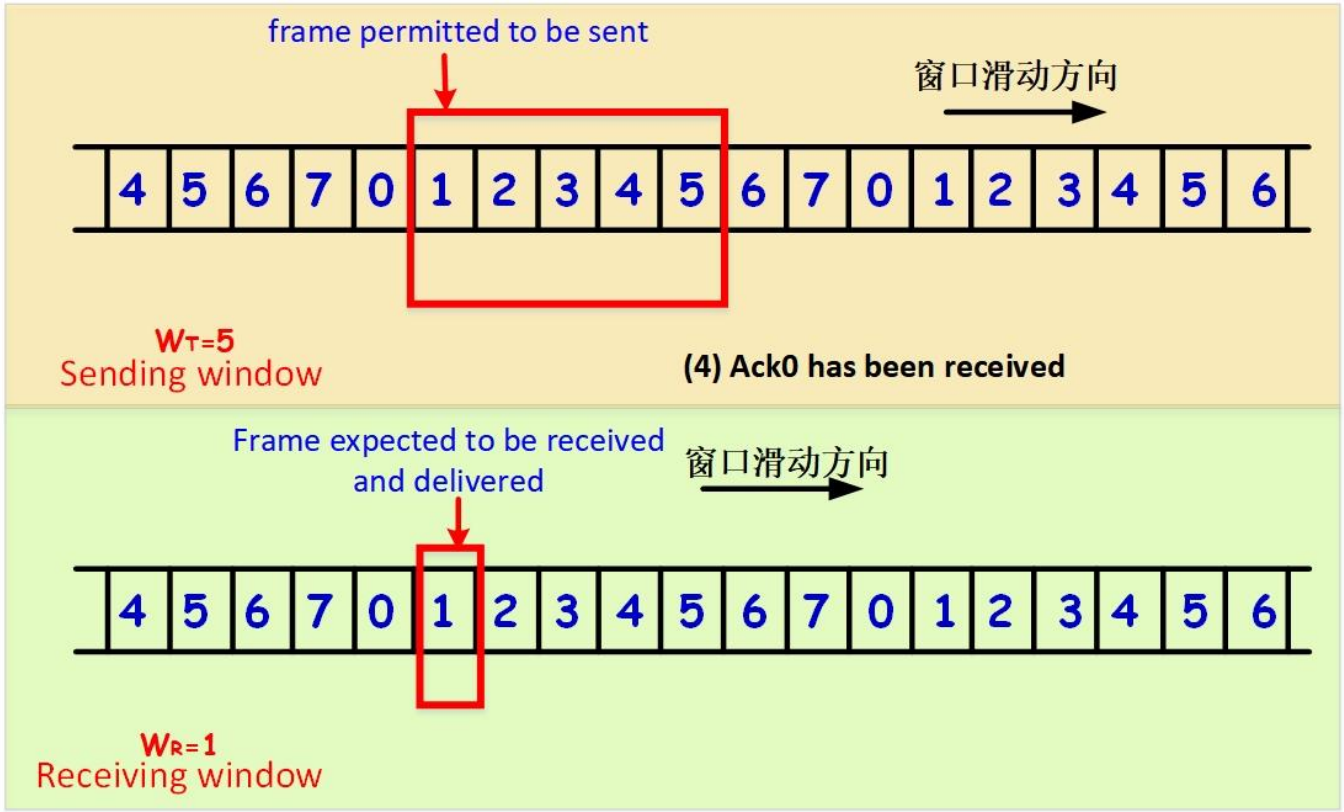
GBN Example(2)



GBN Example(3)

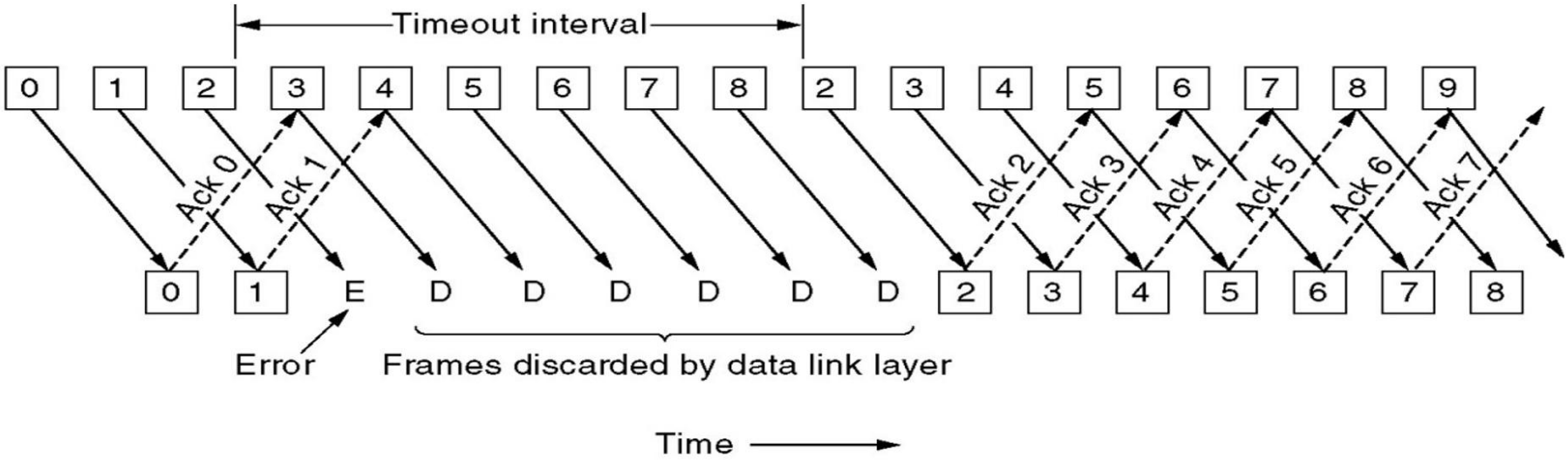


GBN Example(4)



Go Back N（回退N步）：Principles

- If Receiver detects error, discard that frame and all future frames until error frame received correctly
- Transmitter must go back and retransmit that frame and all subsequent frames(Go Back N)

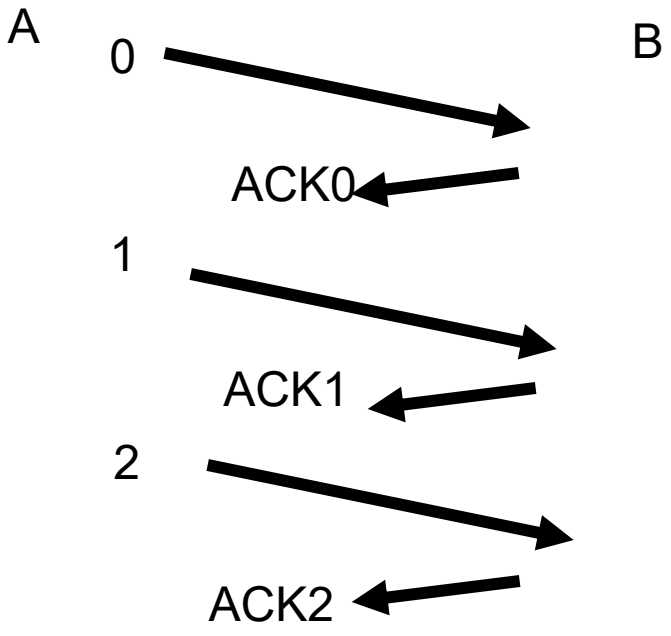


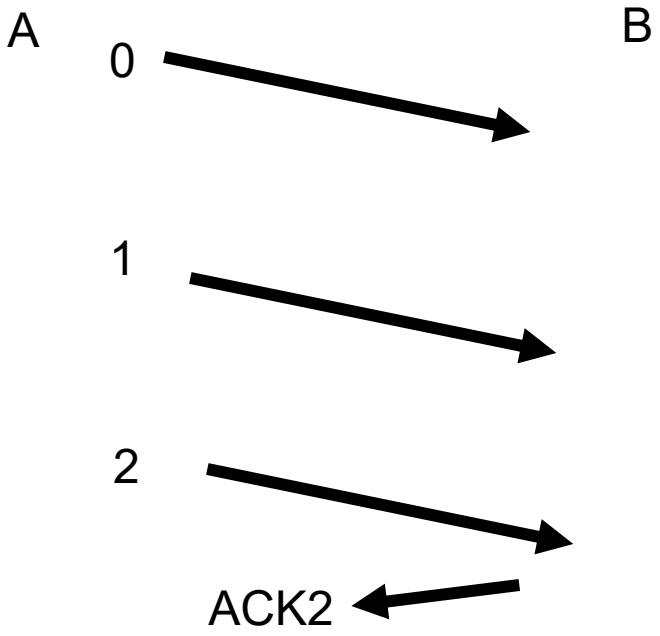
Review

- Sliding Window Protocols
  - reliable, connection-oriented service
  - channel: full-duplex
  - Error control: ARQ (CRC+retransmission)
  - Flow control: sliding windows

	protocol 4: One-Bit Sliding Window	protocol 5: Go Back N
Frames	Data, Ack	Data, Ack
W_T	1	>1 (pipelining)
W_R	1	1
n	1	n
Acknowledges	Ack, Piggybacking	Ack, Piggybacking, Cumulative Ack
Timers	frame_timer, Ack_timer	frame_timer, Ack_timer
Buffers	sender_buffer (1)	>1
PDU	**Data** + **Data seq_number** + ACK seq_number	**Data** + **Data seq_number** + **ACK seq_number**

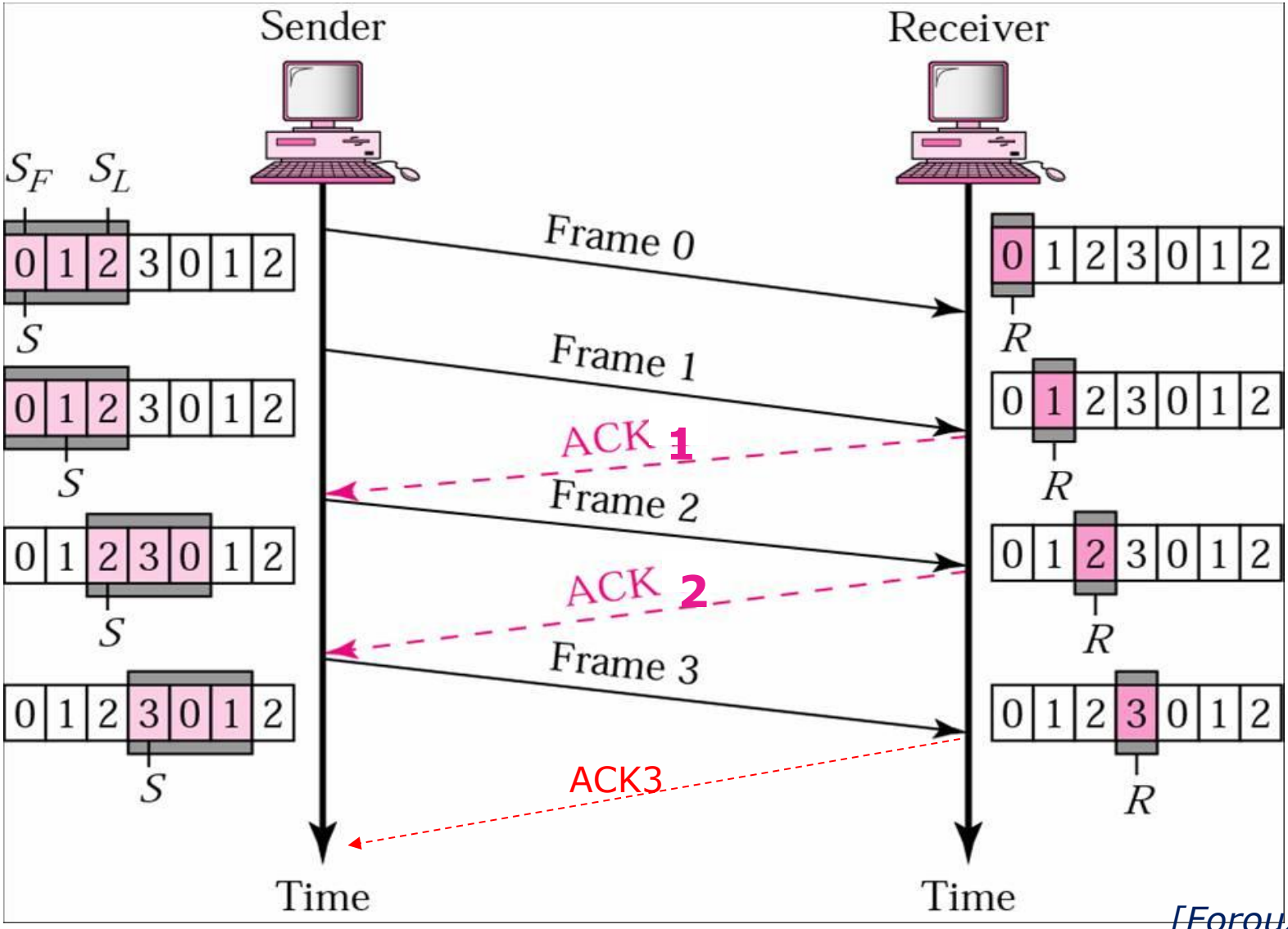
Go Back N: No Need to ACK Every Frame





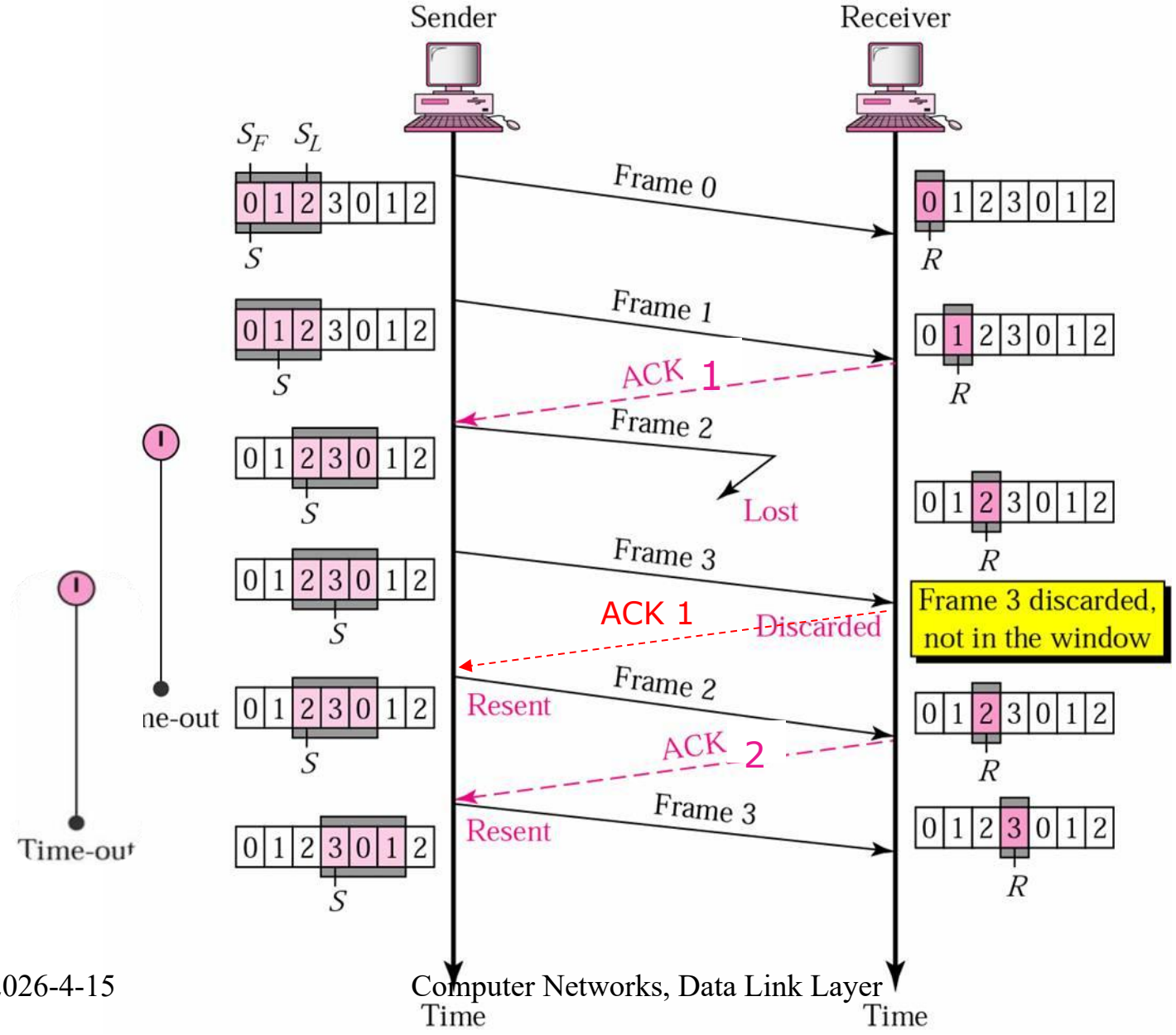
Cumulative ACK
（累积ACK）

Go Back N: Normal Operation



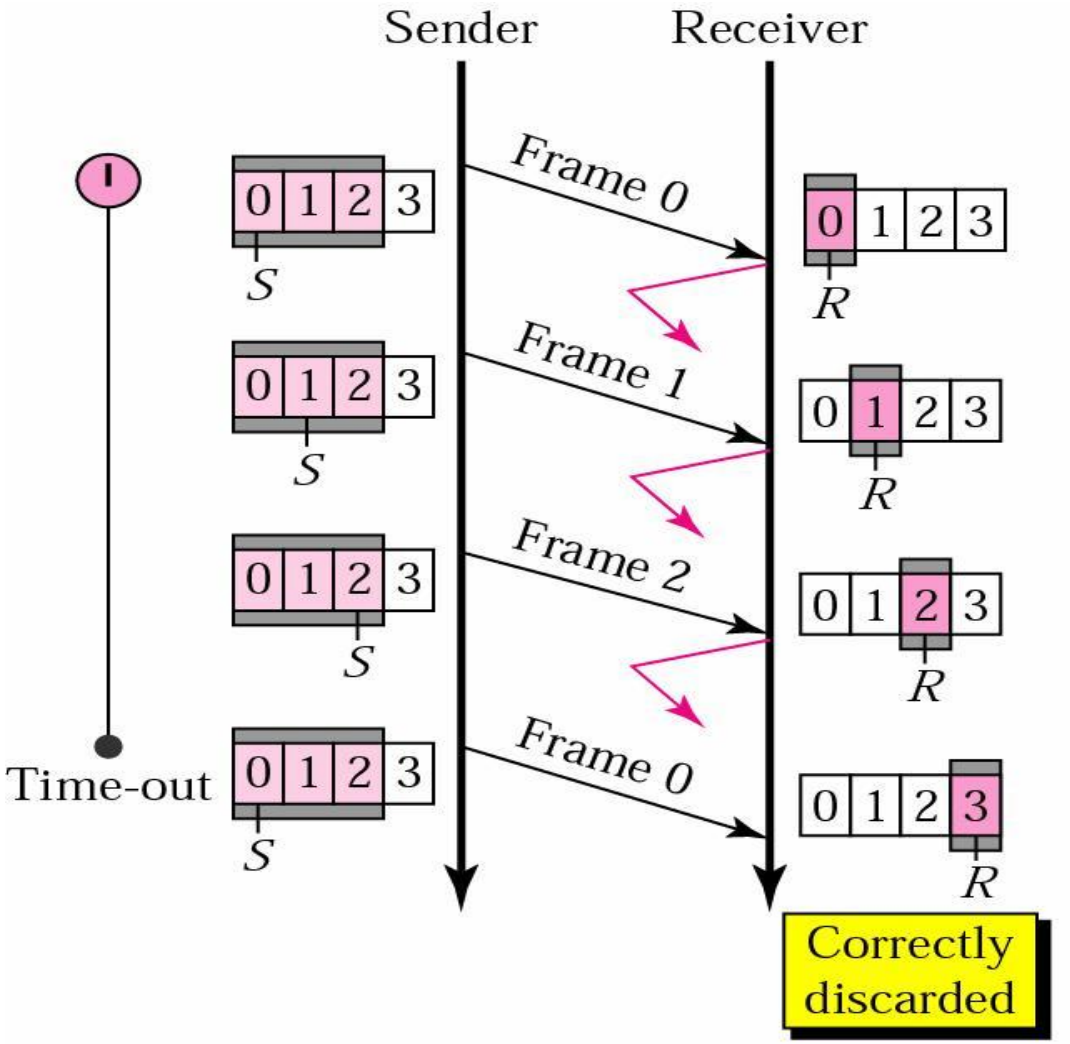
[Forouzan]

Go Back N: Frame Lost



[Forouzan]

Go Back N ARQ: ACK Lost



[Forouzan]

Go Back N: Frame Damaged

- If Data Frame is Damaged
  - Receiver detects error in frame *i*
  - Receiver discard that frame *i* and all subsequent
  - When timeout, sender retransmits frame *i* and all subsequent
- If ACK Frame is Damaged
  - Receiver gets frame *i* and send acknowledgement *ACK(i)* which is lost
  - Acknowledgements are cumulative, so next *ACK(i+n)* may arrive before sender times out on frame *i*, thus no need for retransmission
  - If sender times out, retransmits frame *i* and all subsequent

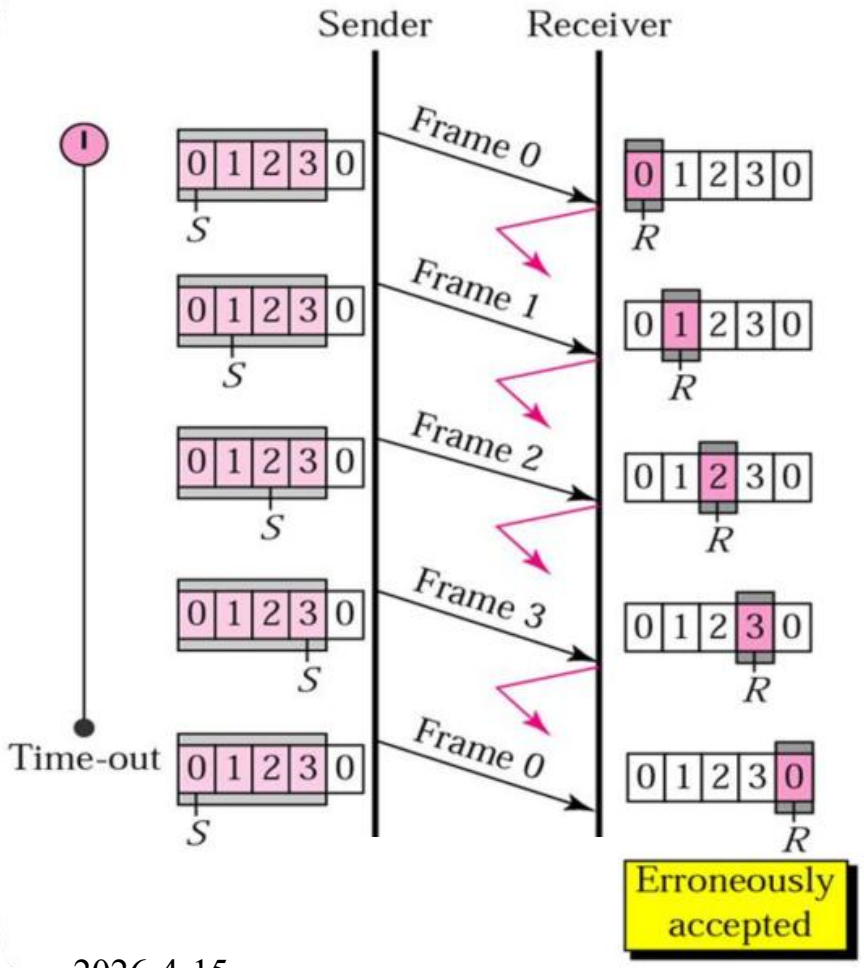
Channel Efficiency of Go Back N

- Consider a 50-kbps satellite channel with a 500-msec round-trip propagation delay, send 1000-bit frames, W_T=26
  - t=0: sender starts sending
  - t=20 ms: the 1st frame has been completely sent
  - t=270 ms: frame fully arrived at the receiver
  - t= 520 ms: the 26th frame has been sent
  - t=520 ms: ACK arrived back at the sender
- Channel efficiency
  - 520/520=100%
  - Pipelining（流水线）

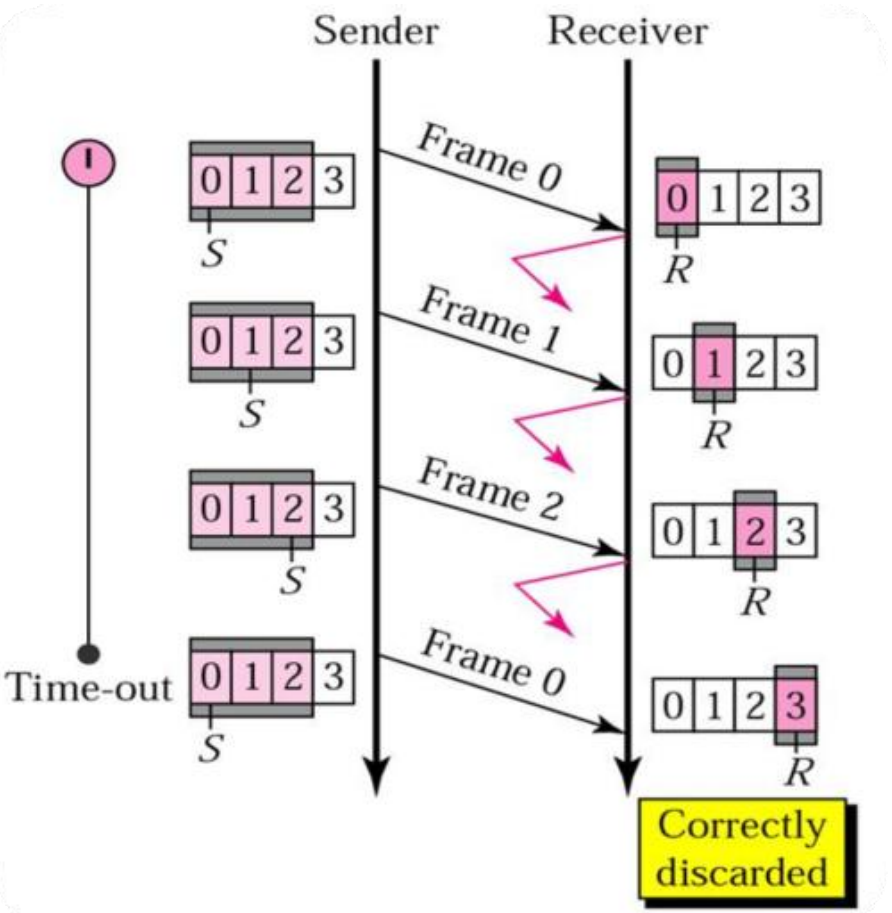
Maximum of Sending Window Size

- Can window size be 2^n for n-bit sequence number?(eg.n=2)

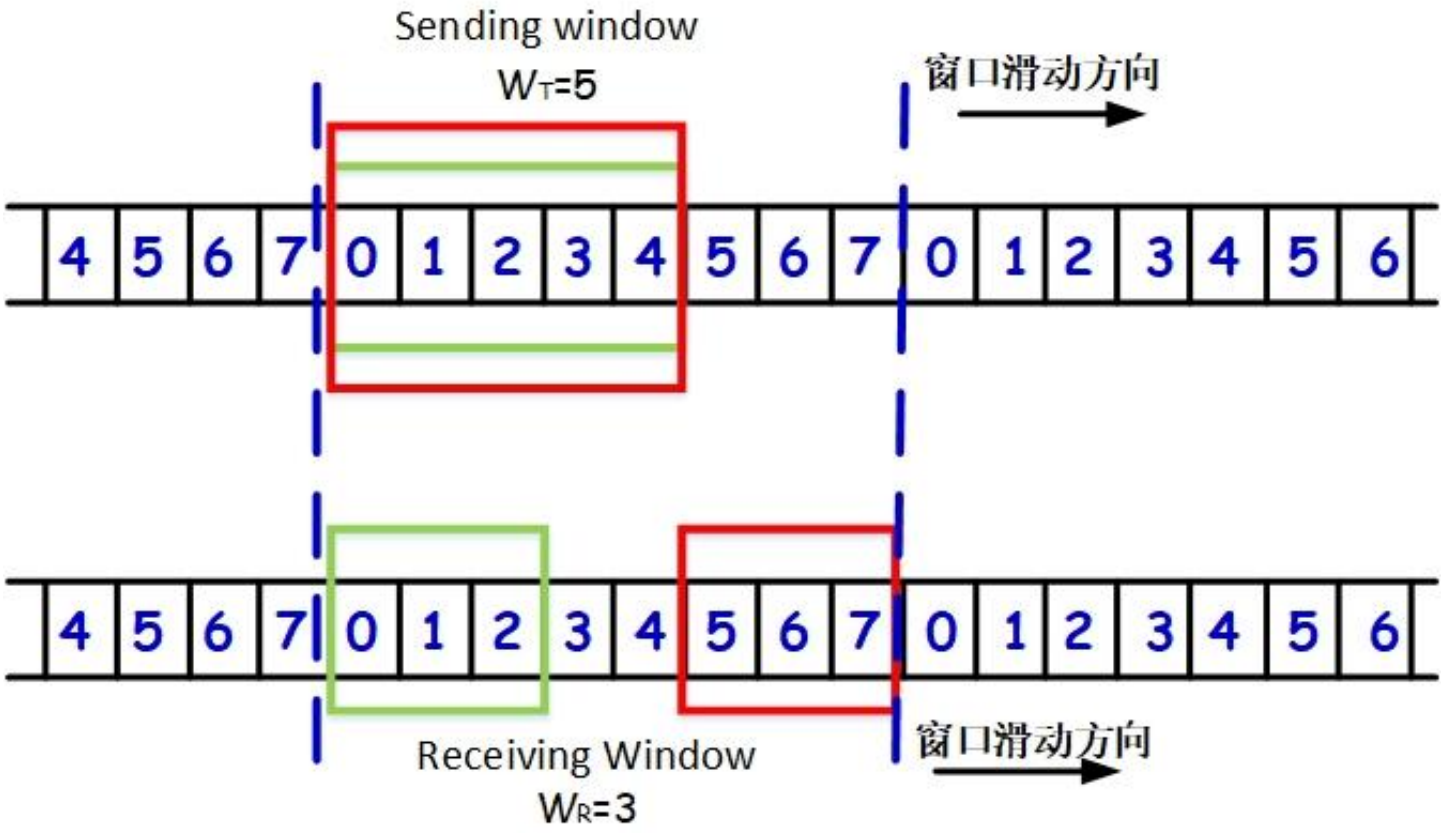
(1) sender window=2^n



(2) sender window=2^n-1



Sending window & Receiving window

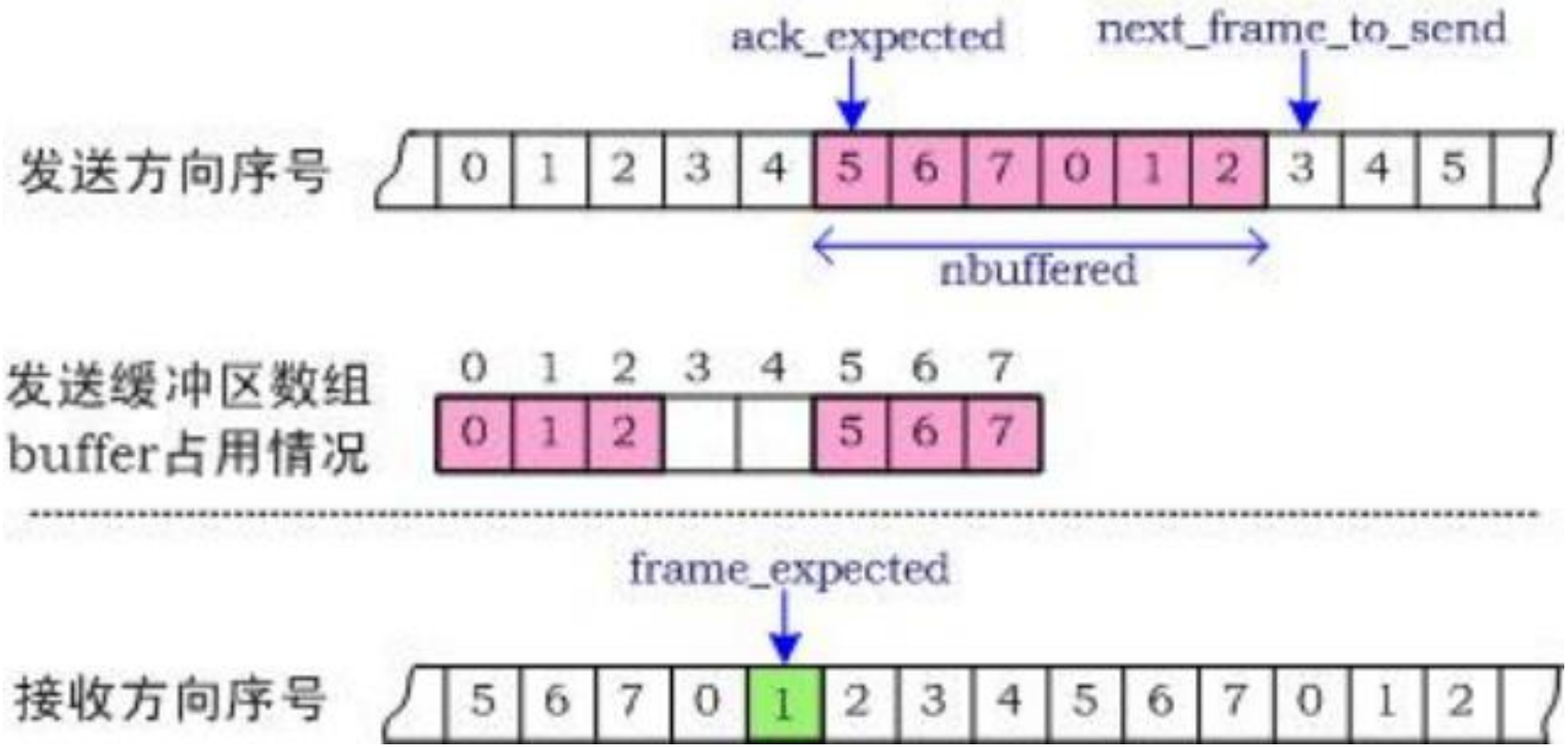


Maximum of Sending Window Size

- Problem: How does receiver detect the duplicate frame?
- 原则
  - 当发送方发送完发送窗口内所有帧，在未收到ACK之前，接收方 正确接收到所有发送来的帧，接收窗口向前推移，此时，必须保证发送窗口与接收窗口在序号上不能重叠
  - For n-bit sequence number： $(W_T+W_R) \leq 2^n$
  - Go Back N：$W_R=1$ $W_T \leq 2^n-1$

Protocol 5:Go Back N Protocol

3-bit Sequence Number, Max sending window= 7



```
/* Protocol 5 (Go-back-n) allows multiple outstanding frames. The sender may transmit up
   to MAX_SEQ frames without waiting for an ack. In addition, unlike in the previous
   protocols, the network layer is not assumed to have a new packet all the time. Instead,
   the network layer causes a network_layer_ready event when there is a packet to send. */

#define MAX_SEQ 7
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Return true if a <= b < c circularly; false otherwise. */
  if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
      return(true);
   else
      return(false);
}

static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
{
/* Construct and send a data frame. */
  frame s;                                  /* scratch variable */

  s.info = buffer[frame_nr];                /* insert packet into frame */
  s.seq = frame_nr;                         /* insert sequence number into frame */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);/* piggyback ack */
  to_physical_layer(&s);                    /* transmit the frame */
  start_timer(frame_nr);                    /* start the timer running */
}
```

Receiver:(1)frame到达且校验和正确;(2)frame到达但校验和错误;
Sender:(3)frame-timer超时;(4)上层有数据要发送

判断**ack序号=b**是否在以序号a和序号c为边界的发送窗口内

frame序号

根据该序号计算ack序号(ack序号是该序号的前一个序号

计算frame-expected的前一个序号

last unacknowledged frame sent+1

Lower Window Edge

```
void protocol5(void)
{
  seq_nr next_frame_to_send;          /* MAX_SEQ > 1; used for outbound stream */
  seq_nr ack_expected;                /* oldest frame as yet unacknowledged */
  seq_nr frame_expected;              /* next frame expected on inbound stream */
  frame r;                            /* scratch variable */
  packet buffer[MAX_SEQ + 1];         /* buffers for the outbound stream */
  seq_nr nbuffered;                   /* # output buffers currently in use */
  seq_nr i;                           /* used to index into the buffer array */
  event_type event;

  enable_network_layer();             /* allow network_layer_ready events */
  ack_expected = 0;                   /* next ack expected inbound */
  next_frame_to_send = 0;             /* next frame going out */
  frame_expected = 0;                 /* number of frame expected inbound */
  nbuffered = 0;                      /* initially no packets are buffered */
```

```
while (true) {
  wait_for_event(&event);              /* four possibilities: see event_type above */

  switch(event) {
    case network_layer_ready:          /* the network layer has a packet to send */
         /* Accept, save, and transmit a new frame. */
         from_network_layer(&buffer[next_frame_to_send]); /* fetch new packet */
         nbuffered = nbuffered + 1;    /* expand the sender's window */
         send_data(next_frame_to_send, frame_expected, buffer);/* transmit the frame */
         inc(next_frame_to_send);      /* advance sender's upper window edge */
         break;

    case frame_arrival:                /* a data or control frame has arrived */
         from_physical_layer(&r);      /* get incoming frame from physical layer */

         if (r.seq == frame_expected) {
              /* Frames are accepted only in order. */
              to_network_layer(&r.info);  /* pass packet to network layer */
              inc(frame_expected);   /* advance lower edge of receiver's window */
         }
```

已发送且未收到确认的帧数

receiver:处理数据

帧序号正确，提交网络层，接收窗口前移1格

sender:
处理Ack

```
/* Ack n implies n – 1, n – 2, etc.  Check for this. */
while (between(ack_expected, r.ack, next_frame_to_send)) {
        /* Handle piggybacked ack. */
        nbuffered = nbuffered – 1; /* one frame fewer buffered */
        stop_timer(ack_expected); /* frame arrived intact; stop timer */
        inc(ack_expected);       /* contract sender's window */
}
break;

case cksum_err: break;                  /* just ignore bad frames */

case timeout:                           /* trouble; retransmit all outstanding frames */
        next_frame_to_send = ack_expected;     /* start retransmitting here */
        for (i = 1; i <= nbuffered; i++) {
                send_data(next_frame_to_send, frame_expected, buffer);/* resend 1 frame */
                inc(next_frame_to_send);   /* prepare to send the next one */
        }
}

if (nbuffered < MAX_SEQ)
        enable_network_layer();
else
        disable_network_layer();
}
}
```

ack序号落在已发送且未被确认的范围内(cumulative ack)，发送窗口向前滑动n格

go back n
将全部已发送且未收到确认的帧重传一遍

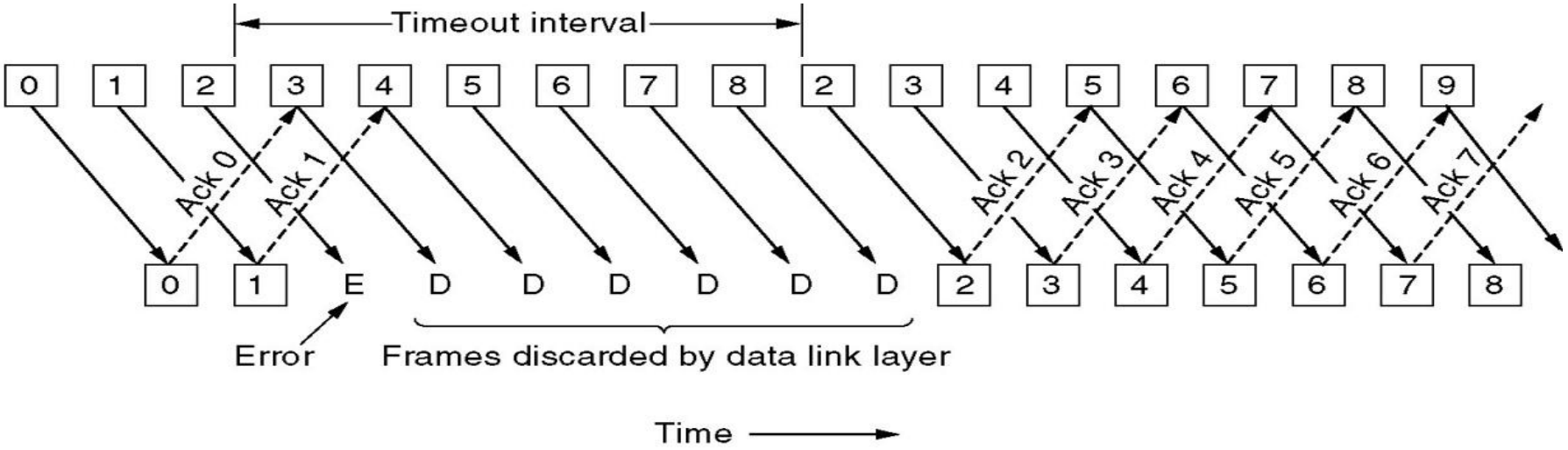
已发送且未收到确认的帧数量未达到发送窗口值

Summary

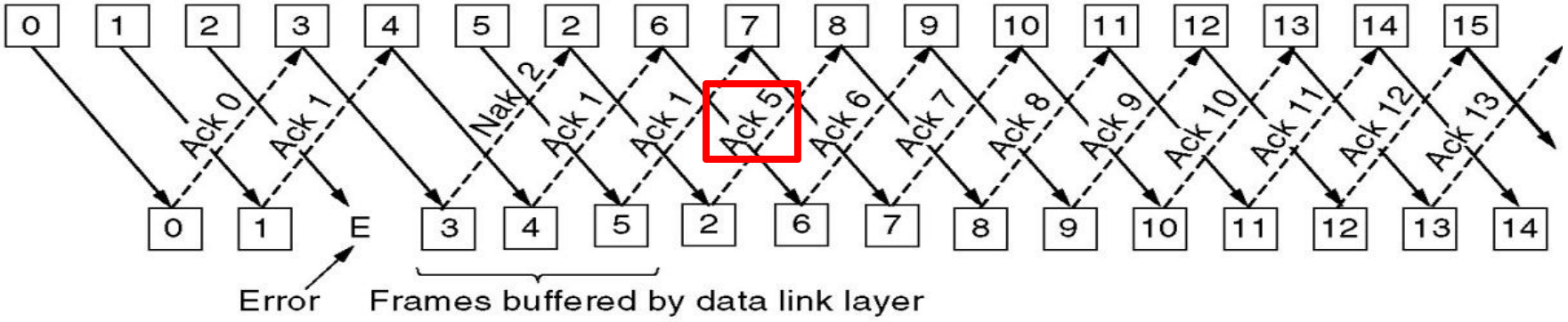
- Sliding Window Protocols
  - reliable, connection-oriented service
  - channel: full-duplex
  - Error control: CRC+retransmission
  - Flow control: sliding windows

	protocol 4: One-Bit Sliding Window	protocol 5: Go Back N
Frames	Data, Ack	Data, Ack
n	1	n
W_T	1	2^n-1
W_R	1	1
Acknowledges	Ack, Piggybacking	Ack, Piggybacking, Cumulative Ack
Timers	frame_timer, Ack_timer	frame_timer, Ack_timer
Buffers	sender_buffer (1)	sender_buffer (2^n)
Error frame	retransmit error frame	retransmit error frame and all subsequent frames(Go Back N)

Problems of Go Back N



Receiver's window size is 1.



Receiver's window size is 4.

Basic Flow Control Protocols

- 3.3 Elementary Protocols
  - 3.3.1 An Unrestricted Simplex Protocol
  - 3.3.2 A Simplex Stop-and-Wait Protocol
  - 3.3.3 A Simplex Protocol for a Noisy Channel
- 3.4 Sliding Window Protocols
  - 3.4.1 A One-Bit Sliding Window Protocol
  - 3.4.2 A Protocol Using Go Back N
  - *3.4.3 A Protocol Using Selective Repeat*
  - Performance of sliding window protocols

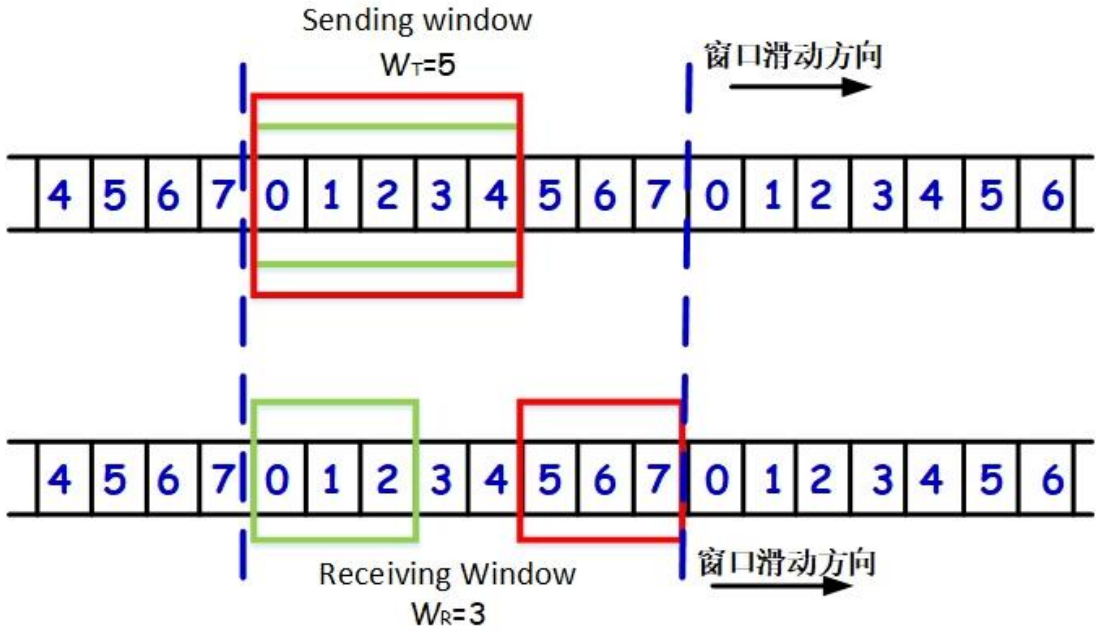
3.4.3 Selective Repeat（选择重传） ARQ

- Also called Selective Reject
- Only rejected frames are retransmitted
- Subsequent frames are accepted by the receiver and buffered
- Minimizes retransmission
- Receiver must maintain large enough buffer
- More complex

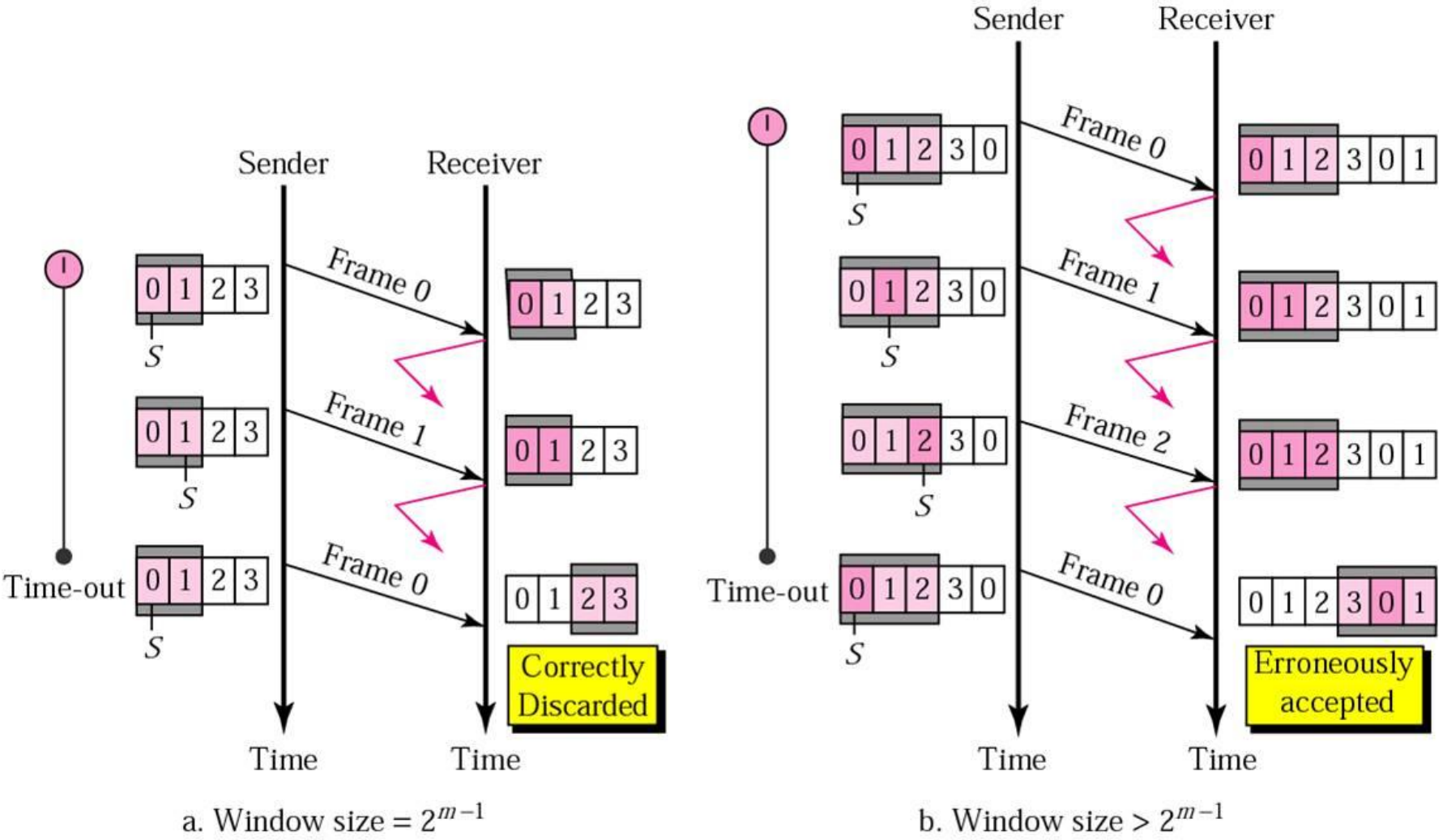
Protocol 6: Selective Repeat-- Maximum Window Size

- The maximum window size should be at most half the range of the sequence numbers

- $W_T + W_R <= 2^n$
- $W_T >= W_R$
- In practice, usually $W_T = W_R = 2^{n-1}$



Maximum of Sending Window Size: Example of 2-bit SN(m=2)



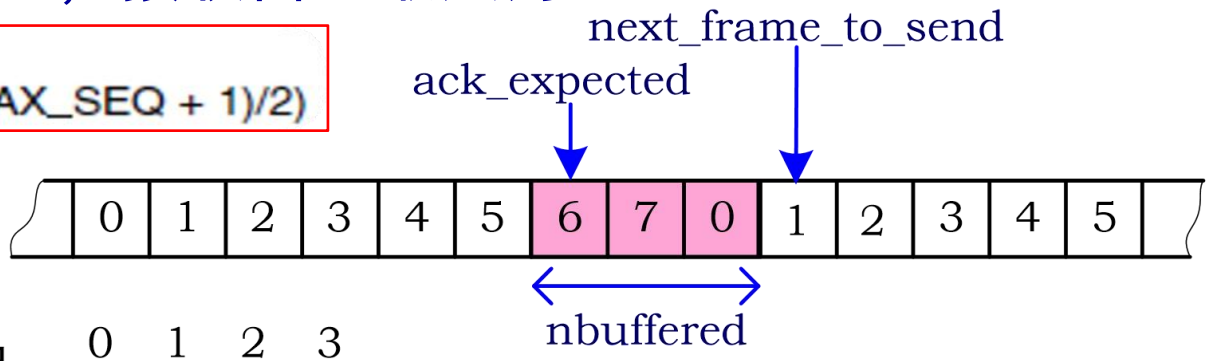
Protocol 6: Selective Repeat--Buffers

- How many buffers must the receiver have?
  - The number of buffers needed is equal to the receiving window size, not to the range of sequence numbers

Protocol 6: Selective Repeat--Buffers

发送窗口最大为4，接收窗口最大为4

```
#define MAX_SEQ 7
#define NR_BUFS ((MAX_SEQ + 1)/2)
```



Protocol 6: Selective Repeat--Timers

- frame_timer
  - 数据重传所用的定时器操作
    - *start_timer(frame_nr%NR_BUFS)*
    - *stop_timer(frame_nr%NR_BUFS)*
  - 计时器数量：与sending buffer number相关，one sending buffer required one timer

Protocol 6: Selective Repeat--Timers

- 设置ACK timer
  - 在没有数据要发送时，发送ACK帧
  - 用于回送ACK的定时器操作

    start_ack_timer()和stop_ack_timer()
  - ACK定时器时限的设计（考虑线路往返时间延迟，接收方物理层发送排队时延等）
    - The timeout associated with the auxiliary timer be appreciably shorter than the timer used for timing out data frames

      2*propagating delay+ack_timer<frame_timer
  - ack_timer数量: 只需要一个即可，以第一个为准

    ack_timeout

Protocol 6: Selective Repeat--NAK

- Whenever the receiver has reason to suspect that an error has occurred, it sends NAK frame back to the sender
  - NAK frame is a request for retransmission
- There are two cases when the receiver should be suspicious
  - A damaged frame has arrived
  - A frame other than the expected one arrived
- Avoid making multiple requests for retransmission of the same frame
  - Receiver keeps track of whether a NAK has already been sent for a given frame

```
/* Protocol 6 (Selective repeat) accepts frames out of order but passes packets to the
   network layer in order. Associated with each outstanding frame is a timer. When the timer
   expires, only that frame is retransmitted, not all the outstanding frames, as in protocol 5. */

#define MAX_SEQ 7                                   /* should be 2^n – 1 */
#define NR_BUFS ((MAX_SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;
#include "protocol.h"
boolean no_nak = true;                              /* no nak has been sent yet */
seq_nr oldest_frame = MAX_SEQ + 1;                  /* initial value is only for the simulator */

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Same as between in protocol 5, but shorter and more obscure. */
  return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}

static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{
/* Construct and send a data, ack, or nak frame. */
  frame s;                                          /* scratch variable */

  s.kind = fk;                                      /* kind == data, ack, or nak */
  if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
  s.seq = frame_nr;                                 /* only meaningful for data frames */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
  if (fk == nak) no_nak = false;                    /* one nak per frame, please */
  to_physical_layer(&s);                            /* transmit the frame */
  if (fk == data) start_timer(frame_nr % NR_BUFS);
  stop_ack_timer();                                 /* no need for separate ack frame */
}
```

判断**序号=b**是否在以序号a和序号c为边界的窗口内

任意位置发送窗口内的帧一定在不同buffer中

Continued →

```
void protocol6(void)
{
  seq_nr ack_expected;                  /* lower edge of sender's window */
  seq_nr next_frame_to_send;            /* upper edge of sender's window + 1 */
  seq_nr frame_expected;                /* lower edge of receiver's window */
  seq_nr too_far;                       /* upper edge of receiver's window + 1 */
  int i;                                /* index into buffer pool */
  frame r;                              /* scratch variable */
  packet out_buf[NR_BUFS];              /* buffers for the outbound stream */
  packet in_buf[NR_BUFS];               /* buffers for the inbound stream */
  boolean arrived[NR_BUFS];             /* inbound bit map */
  seq_nr nbuffered;                     /* how many output buffers currently used */
  event_type event;

  enable_network_layer();               /* initialize */
  ack_expected = 0;                     /* next ack expected on the inbound stream */
  next_frame_to_send = 0;               /* number of next outgoing frame */
  frame_expected = 0;
  too_far = NR_BUFS;
  nbuffered = 0;                        /* initially no packets are buffered */
  for (i = 0; i < NR_BUFS; i++) arrived[i] = false;
```

last unacknowledged frame sent+1

接收缓存没有达到的帧

Continued →

```
while (true) {
  wait_for_event(&event);                    /* five possibilities: see event_type above */
  switch(event) {
    case network_layer_ready:                /* accept, save, and transmit a new frame */
        nbuffered = nbuffered + 1;           /* expand the window */
        from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]); /* fetch new packet */
        send_frame(data, next_frame_to_send, frame_expected, out_buf);/* transmit the frame */
        inc(next_frame_to_send);             /* advance upper window edge */
        break;

    case frame_arrival:                      /* a data or control frame has arrived */
        from_physical_layer(&r);             /* fetch incoming frame from physical layer */
        if (r.kind == data) {
            /* An undamaged frame has arrived. */
            if ((r.seq != frame_expected) && no_nak)
                send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
            if (between(frame_expected, r.seq, too_far) && (arrived[r.seq%NR_BUFS] == false)) {
                /* Frames may be accepted in any order. */
                arrived[r.seq % NR_BUFS] = true;      /* mark buffer as full */
                in_buf[r.seq % NR_BUFS] = r.info;     /* insert data into buffer */
                while (arrived[frame_expected % NR_BUFS]) {
                    /* Pass frames and advance window. */
                    to_network_layer(&in_buf[frame_expected % NR_BUFS]);
                    no_nak = true;
                    arrived[frame_expected % NR_BUFS] = false;
                    inc(frame_expected);     /* advance lower edge of receiver's window */
                    inc(too_far);            /* advance upper edge of receiver's window */
                    start_ack_timer();       /* to see if a separate ack is needed */
                }
            }
        }
  }
}
```

帧序号错且没有发过NAK，则发NAK

非数据帧,固定发送buffer[0]中的数据（其实无用）

帧序在接收窗口内且对应buffer没有数据，则接收并缓存

从接收窗口下沿开始依次提交收到的帧并向前滑动接收窗口空闲buffer出现

Continued →

NAK帧，序号为已发送且未确认范围内的帧，重传出错帧

```
        if((r.kind==nak) && between(ack_expected,(r.ack+1)%(MAX_SEQ+1),next frame to send))
                send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);

        while (between(ack_expected, r.ack, next_frame_to_send)) {
                nbuffered = nbuffered _ 1;              /* handle piggybacked ack */
                stop_timer(ack_expected % NR_BUFS);     /* frame arrived intact */
                inc(ack_expected);                      /* advance lower edge of sender's window */
        }
        break;
    case cksum_err:
        if (no_nak) send_frame(nak, 0, frame_expected, out_buf);/* damaged frame */
        break;
    case timeout:
        send_frame(data, oldest_frame, frame_expected, out_buf);/* we timed out */
        break;
    case ack_timeout:
        send_frame(ack,0,frame_expected, out_buf);      /* ack timer expired; send ack */
    }
    if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
  }
}
```

ack序号落在已发送且未被确认的范围内，发送窗口向前滑动n格

Nak帧,非数据帧,仅有NAK序号,固定发送buffer[0]中的数据（其实无用）

ACK帧,仅有ACK序号,固定发送buffer[0]中的数据（其实无用）

未达到发送窗口值，继续发送

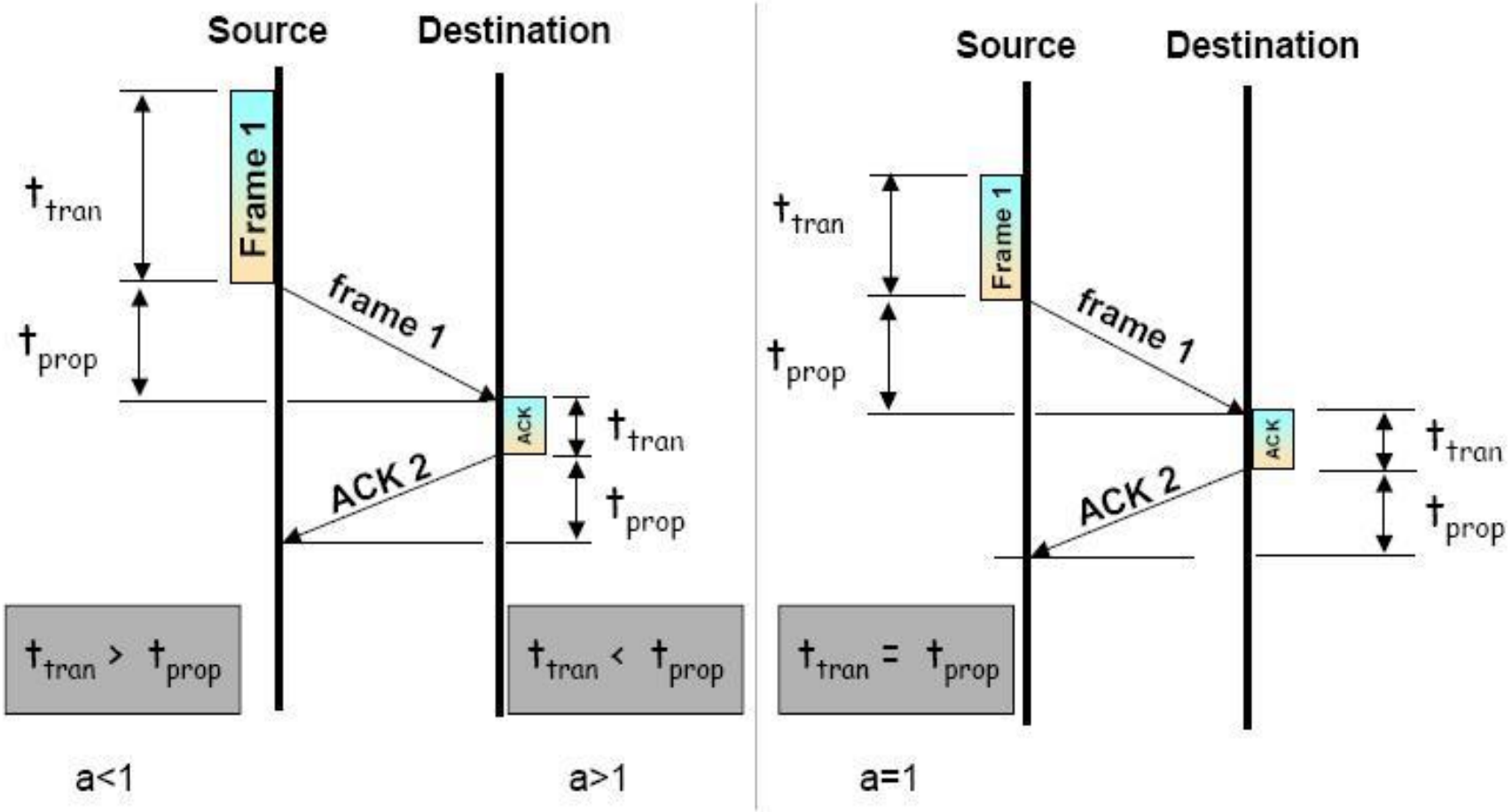
Summary

	Protocol 4	Protocol 5	Protocol 6
n	1	3	3
W_T	1	7 (2^n-1)	4 (2^{n-1})
W_R	1	1	4 (2^{n-1})
frame type	data	data	data, ACK, NAK
Acknowledgement	Piggybacking	Piggybacking, Cumulative ACK	Piggybacking or ACK, Cumulative ACK
NAK	No	No	Yes
ACK-Timer	No	No	Yes
S.ack	1-frame_expected	(frame_expected+MAX_SEQ)%2^n	(frame_expected+MAX_SEQ)%2^n
S-buffers	1	8	4
R-buffers	0	0	4

Basic Flow Control Protocols

- 3.3 Elementary Protocols
  - 3.3.1 An Unrestricted Simplex Protocol
  - 3.3.2 A Simplex Stop-and-Wait Protocol
  - 3.3.3 A Simplex Protocol for a Noisy Channel
- 3.4 Sliding Window Protocols
  - 3.4.1 A One-Bit Sliding Window Protocol
  - 3.4.2 A Protocol Using Go Back N
  - 3.4.3 A Protocol Using Selective Repeat
  - Performance of sliding window protocols

Model of Frame Transmission



$a = t_{prop}/t_{tran}$

Transfer Time & Round Trip Time

- Frame transfer time (txfr)
  - Time for receiver to receive the entire frame

$$t_{xfr} = t_{tran} + t_{prop} + t_{queue/switching}$$

- Round Trip Time (RTT)
  - May not be twice the one-way delay if data frame and ACK frame travel over different links

Performance of Error-free Stop and Wait (1)

- The total time to send the data T can be expressed as $T = nT_F$

  $T_F = t_{tran} + t_{prop} + t_{proc} + t_{ack} + t_{prop} + t_{proc}$

  - T_F : The time to send one frame and receive an acknowledgement
  - t_{tran} : time to transmit a frame
  - t_{prop} : propagation time from Source to Destination
  - t_{proc} : processing time
  - t_{ack} : time to transmit an acknowledgement.

Performance of Error-free Stop and Wait (2)

- Assume t_{proc} and t_{ack} are relatively negligible, then

  $T = n\,(t_{tran} + 2t_{prop})$

- Utilization

$$U = \frac{n \times t_{tran}}{n\left(2t_{prop} + t_{tran}\right)} = \frac{t_{tran}}{2t_{prop} + t_{tran}}$$

$$= \frac{1}{1 + 2a}$$

$$where \quad a = \frac{propogation\ time}{transmission\ time}$$

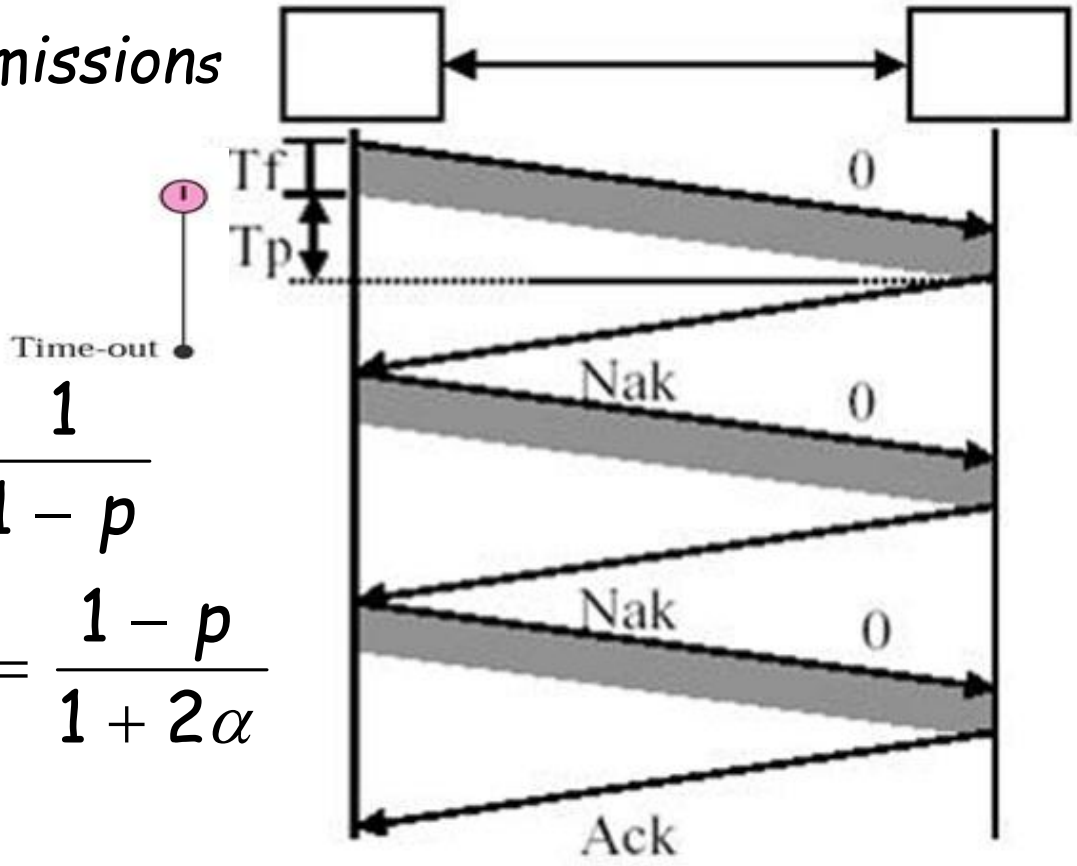
Performance of Stop and Wait: More

- Stop-and-wait with error
  - p=Probability of frame error
  - N_r=number of transmissions

$$\alpha = \frac{T_{prop}}{T_{tran}}$$

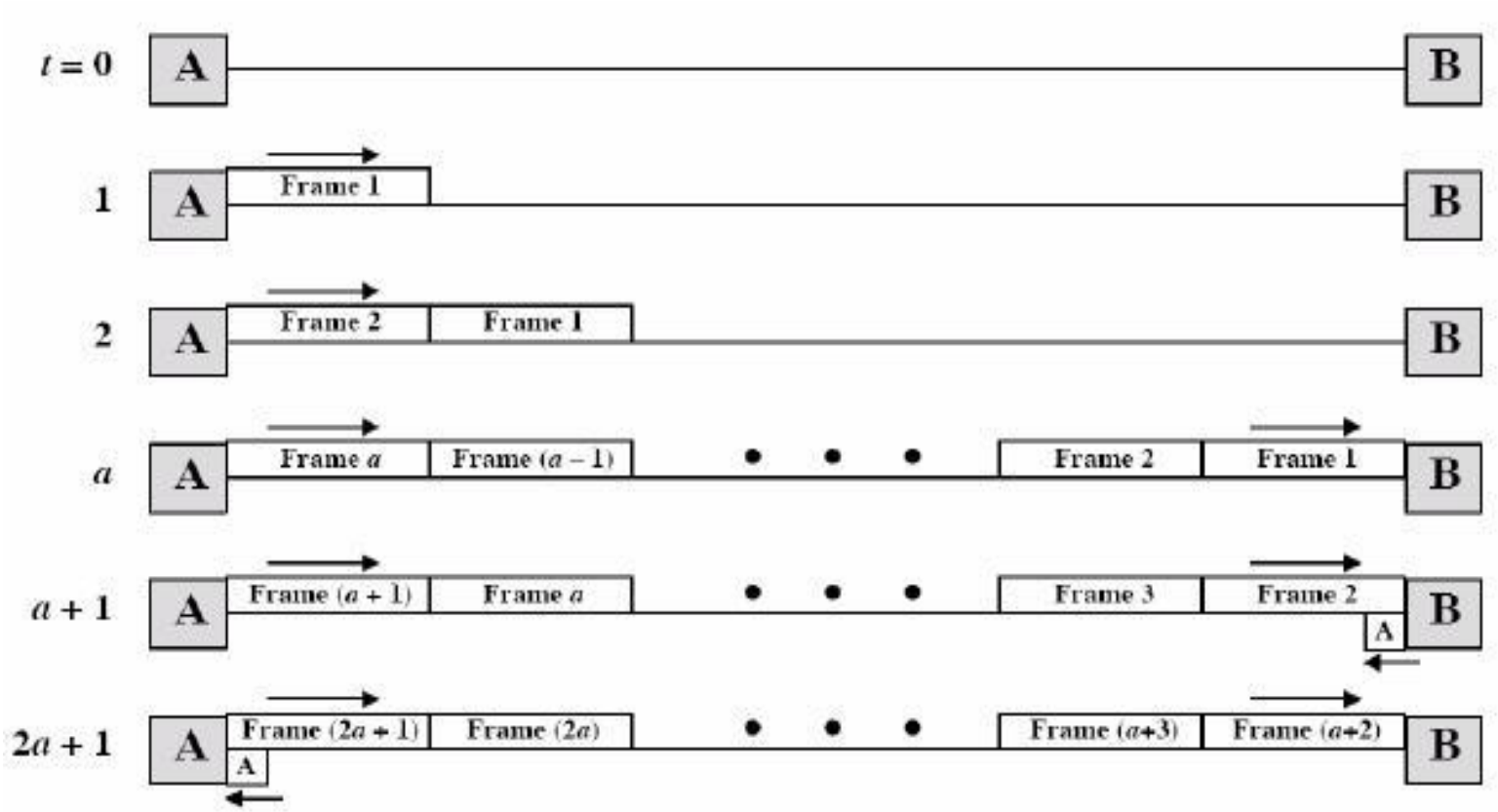
$$N_r = \sum_{i=1}^{\infty} ip^{i-1}(1-p) = \frac{1}{1-p}$$

$$U = \frac{T_{tran}}{N_r(T_{tran} + 2T_{prop})} = \frac{1-p}{1+2\alpha}$$



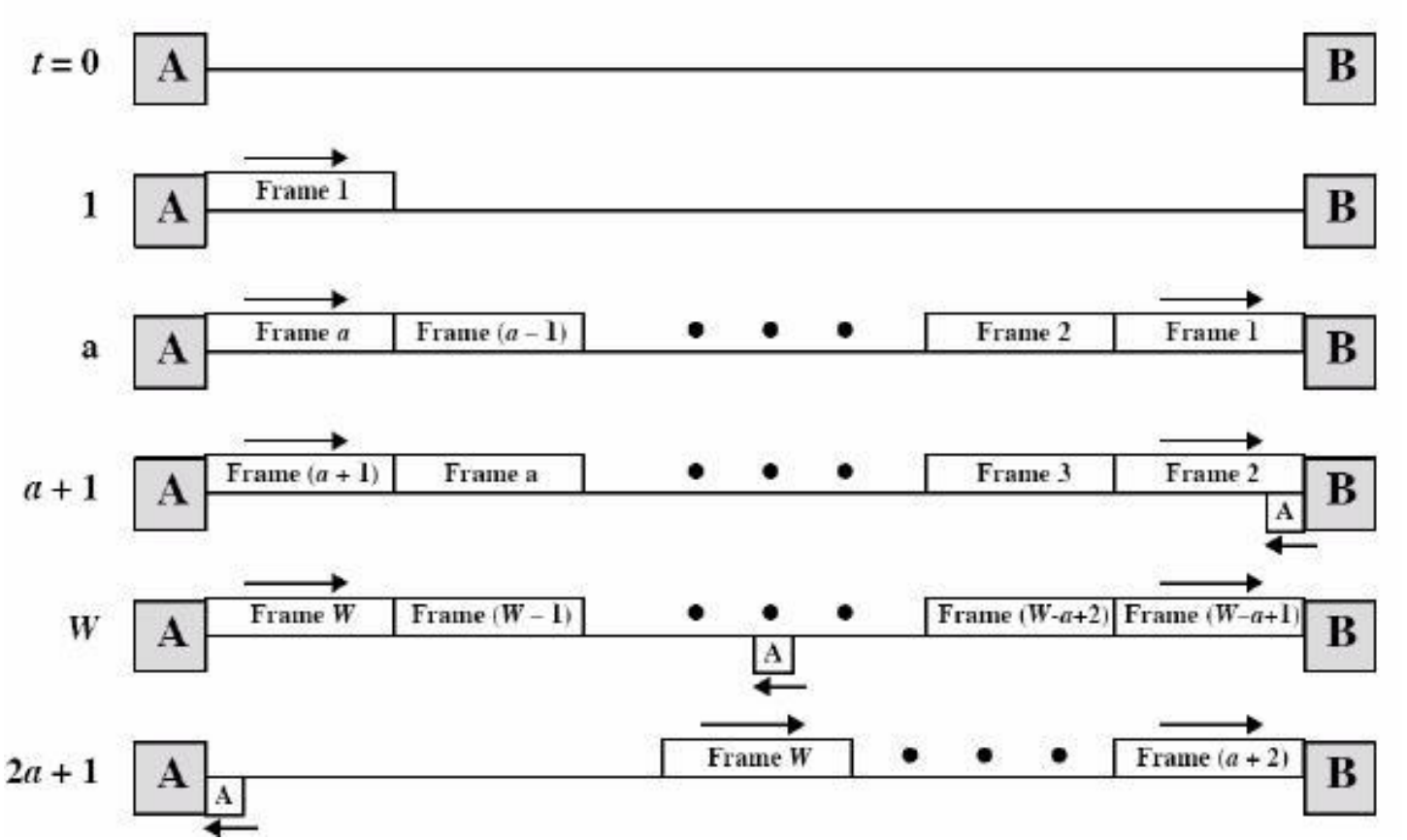
Performance of Error-Free Sliding-Window Flow Control (1)

- Normalize the transmission time to 1, and the propagation time is *a*
  - Case 1: W≧2a+1, where W is window size



Performance of Error-Free Sliding-Window Flow Control (2)

- Case 2: $W < 2a+1$



Performance of Error-Free Sliding-Window Flow Control (3)

- Therefore we can express the utilization

$$U = \begin{cases} 1 & W \geq 2a+1 \\ \dfrac{W}{2a+1} & W < 2a+1 \end{cases}$$

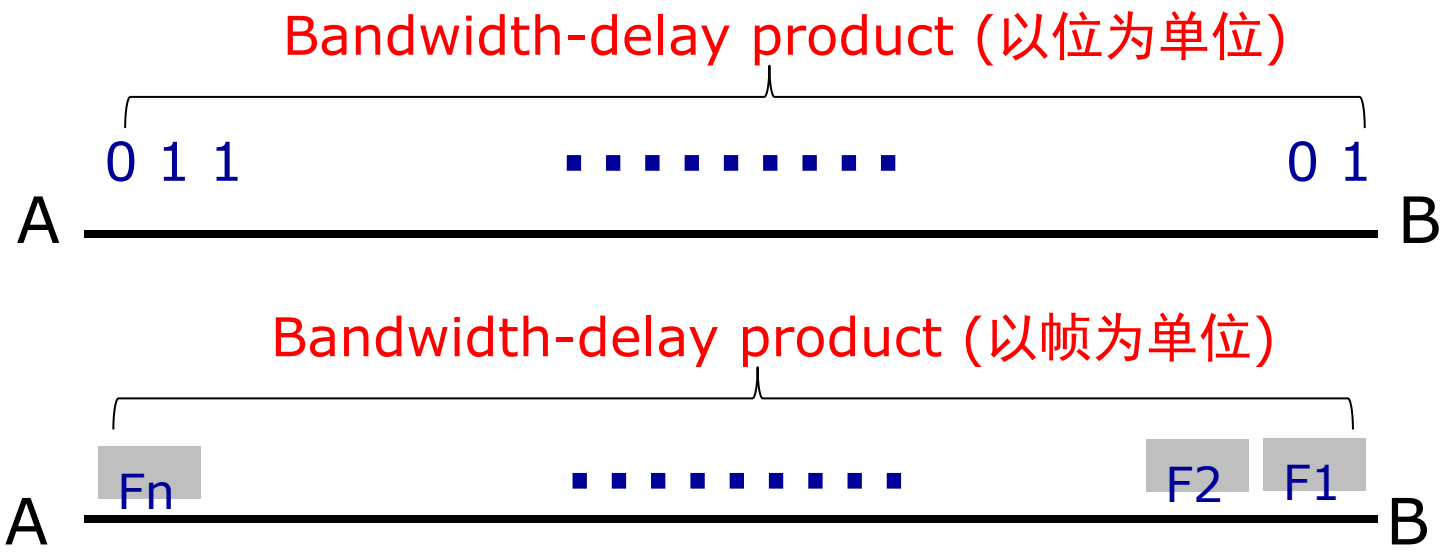
- Another formula in textbook

$$\text{link utilization} \leq \frac{w}{1+2BD}$$

其实是一样的

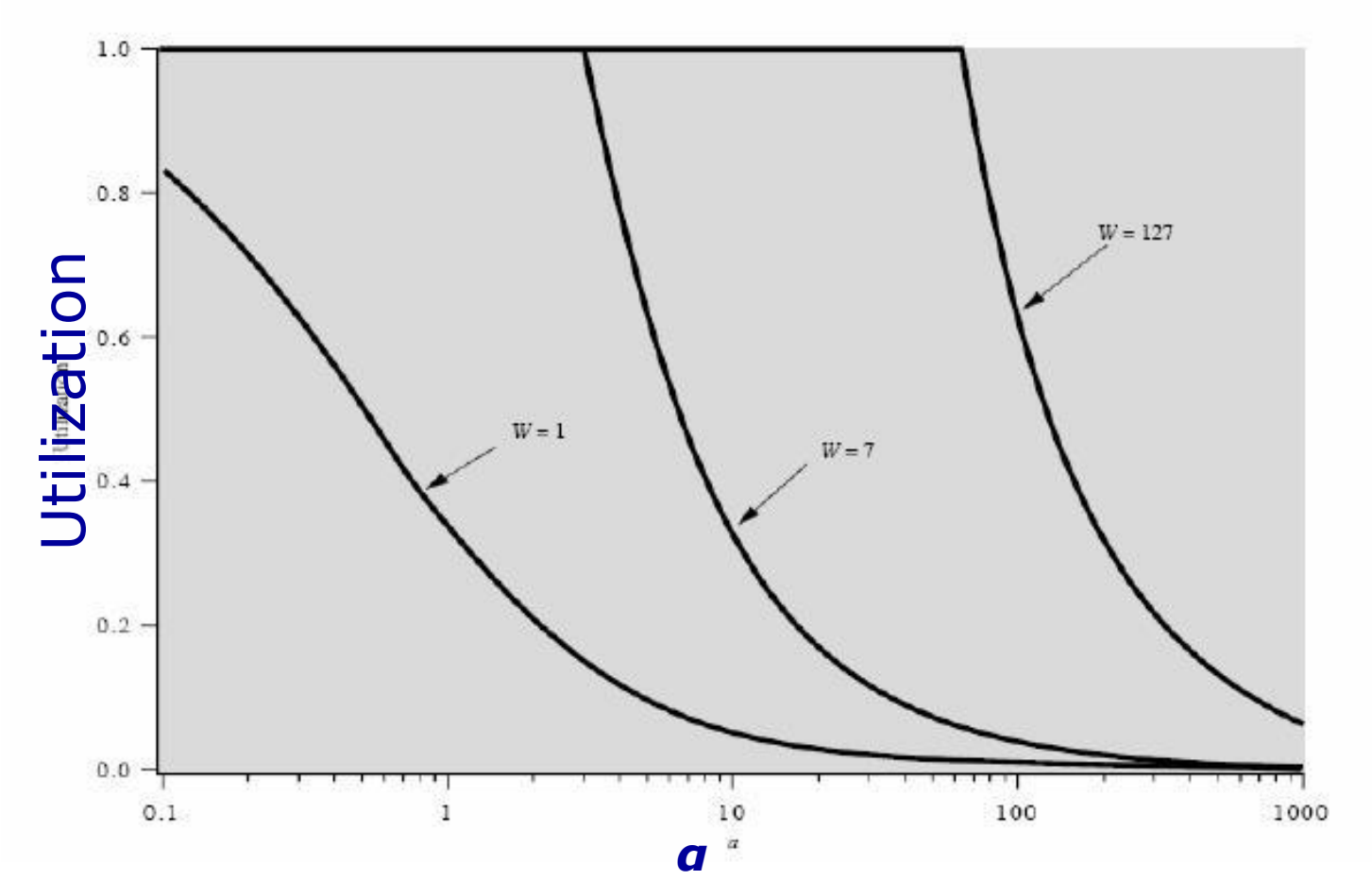
Bandwidth-delay product(带宽时延积)

- Bandwidth: maximum bit rate(bps)
- Delay：propogation delay（传播时延）
- Bandwidth-delay product：在单程传播时延的时间内，信道可以容纳多少位数据



Performance of Error-Free Sliding-Window Flow Control (4)

- Sliding Window Utilization as a function of α



Performance of Error-Free Sliding-Window Flow Control: Piggybacking

- Piggybacking: (捎带确认/搭载ACK)
  - U=W/(2a+2) = W/2(a+1)

Summary

- Performance ($U \leq 1$)
  - sending window=W (Error-Free)
    - ack: $U=W_T/(1+2a)$
    - piggybacking: $U=W_T/(2+2a)$
  - *p=Probability of frame error*
    - $U'=(1-p)*U$
- Window Size
  - n
  - $W_T+W_R \leq 2^n$

Review

- Sliding Window Protocols
  - reliable, connection-oriented service, full-duplex
  - Error control: ARQ(CRC+retransmission)
  - Flow control: sliding windows

	4:One-Bit Sliding Window	5: Go Back N ARQ	6: SR ARQ
Frames	Data, ACK/NAK	Data, ACK/NAK	Data, ACK/NAK
n	1	n	n
W_T	1	2^n-1 (Pipelining)	2^{n-1} (Pipelining)
W_R	1	1	2^{n-1}
Acknowledges	Ack, Piggybacking	Ack, Piggybacking, Cumulative Ack	Ack, Piggybacking, Cumulative Ack
Timers	frame_timer, Ack_timer	frame_timer, Ack_timer	frame_timer, Ack_timer
Buffers	sender_buffer (1)	sender_buffer (2^n)	sender_buffer(2^{n-1}) receiver_buffer(2^{n-1})
Performance	U=1/(1+2a) $a=\frac{propogation\ time}{transmission\ time}$	ACK: U=W_T/(1+2a) Piggybacking: U=W_T/(2+2a)	
Error frame	retransmit error frame	retransmit error frame and all subsequent frames (Go Back N)	retransmit error frame

随堂练习

1. 假设主机采用停-等协议向主机乙发送数据帧，数据帧长与确认帧长均为 1000B。数据传输速率是 10kbps，单项传播延时是 200ms。则甲的最大信道利用率：

A、80%；　B、66.7%；　C、44.4%；　D、40%

(1)发送时延=M/B=(1000*8)/10k=800ms

(2)a=传播时延/发送时延=200ms/800ms=0.25　(3) U=1/(2+2a)=1/2.5=0.4

2 主机甲采用停-等协议向主机乙发送数据，数据传输速率是3kbps，单向传播延时是200ms，忽略确认帧的传输延时。当信道利用率等于40%时，数据帧的长度为_____。

A. 240 比特　B. 400 比特　C. 480 比特　D. 800 比特

(1) U=1/(1+2a)=40% ⟶ a=0.75

(2) a=传播时延/发送时延

(3)发送时延=M/B

传播时延=200ms

带宽B=3kbps

} 数据帧长度M=800bit

随堂练习

Frames of 2000 bits are sent over a 1Mbps channel using a geostationary satellite whose propagation time from the earth is 198 msec. Acknowledgements are always piggybacked onto data frames.

最大吞吐率=信道使用效率*信道带宽=Bandwidth*U

(1)What is the maximum throughput for sending window sizes of 10 and 250?

(2)For GoBackN ARQ protocol, to achieve maximum channel utilization, what size of sending window shall be used? How many bits shall be used for the sequence number?

发送时延=M/B=2000/1M=2ms

a=传播时延/发送时延=198/2=99

信道使用效率 $U=W_T/(2+2a)=W_T/200$

最大吞吐率=信道使用效率*信道带宽=U*1Mbps

W_T=10, 最大吞吐率=10/200*1Mbps=50kbps

W_T=250, U=250/200>1

最大吞吐量=信道带宽=1Mbps

信道使用效率$U=W_T/(2+2a)=W_T/200=1$

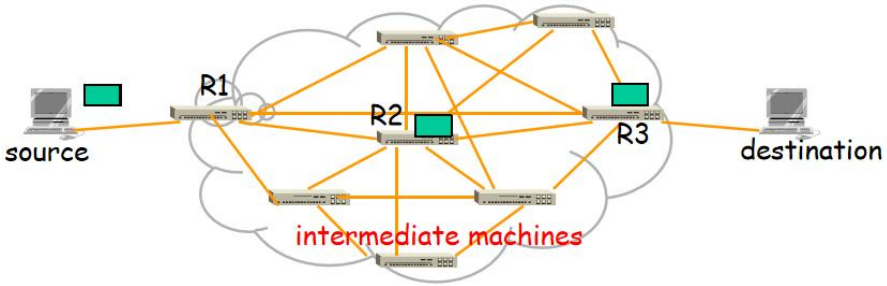
$W_T \geq 2^n-1$　　$n \geq 8$

Outline

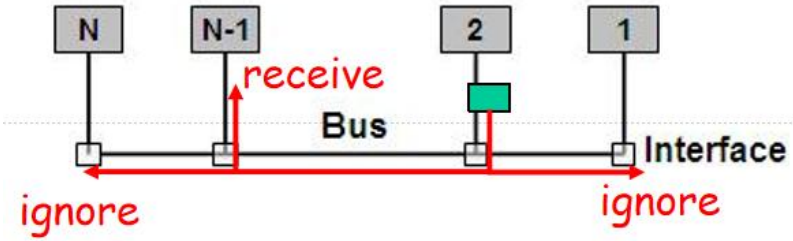
- Position, Functions and Services(include 3.1.1)
- 3.1.2 Framing
- 3.2 Error Detection and Correction (include 3.1.3)
  - 3.2.1 Error-Correcting Codes (ECC)
  - 3.2.2 Error-Detecting Codes (EDC)
- Flow control (textbook 3.1.4 & 3.3 & 3.4)
  - 3.3 Elementary Data Link Protocols
  - 3.4 Sliding Window Protocols
- 3.5 Example Protocols(PPP、HDLC)

Data Link Types

- Point to point links(点到点链路)
  - WAN
  - One sender, one receiver
  - Address not be necessary
  - (Reliable) communication
  - Sample protocols: HDLC, PPP…



- Broadcast links(广播链路)
  - LAN
  - Accessing a shared medium
  - Many senders and potential receivers.
  - Addressing is necessary
  - Sample protocols(Chapter 4): ALOHA, CSMA/CD…



Example of Point-to-point Data Link Protocols

- *HDLC (High-Level Data Link Control)*

  （高级数据链路控制）

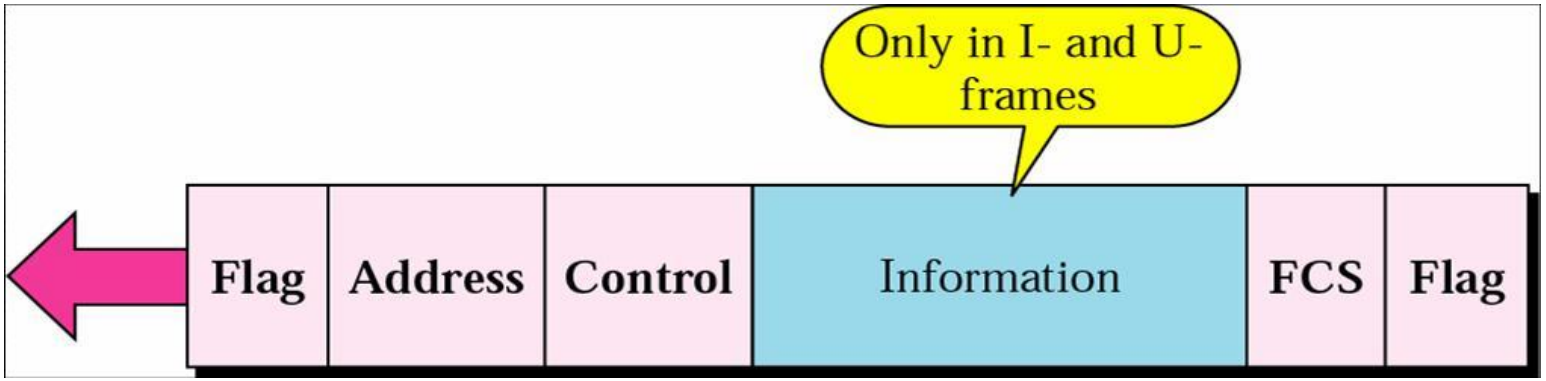
- The Data Link Layer in the Internet
  - PPP (Point-to-Point Protocol)

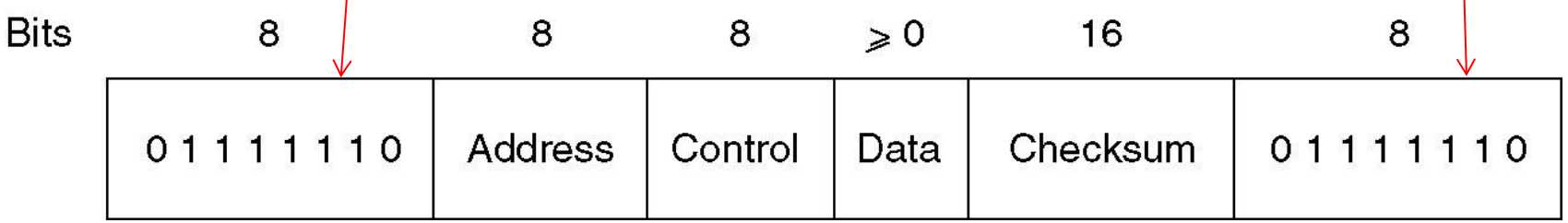
HDLC: High-Level Data Link Control

- Reliable, connection-oriented transmission
  - flow control, error control
  - GoBackN ARQ, Selective Repeat ARQ
- Synchronous serial transmission 同步串行链路传输
- Error detection：CRC
- 不支持相关链路和网络参数协商
- 不支持认证

HDLC: Frame Structure

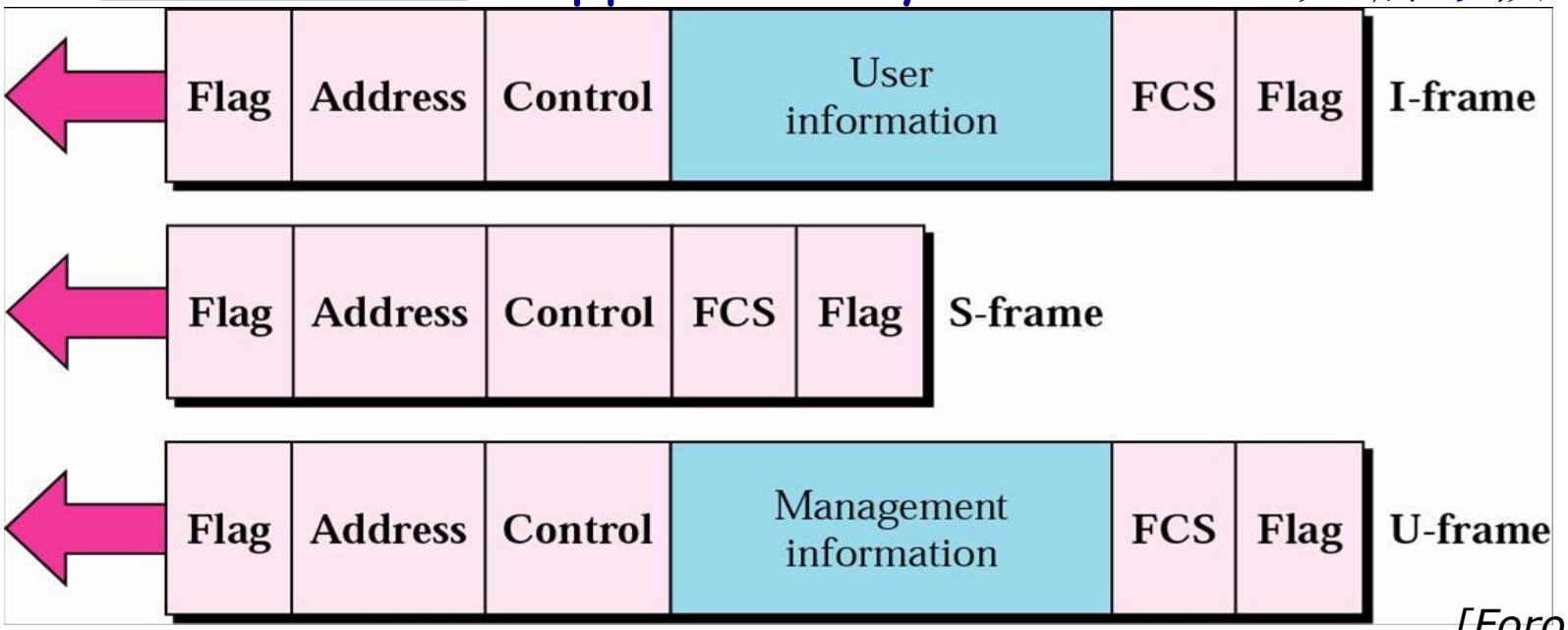
- Bit-oriented, bit stuffing for transparent transmission（零比特填充）





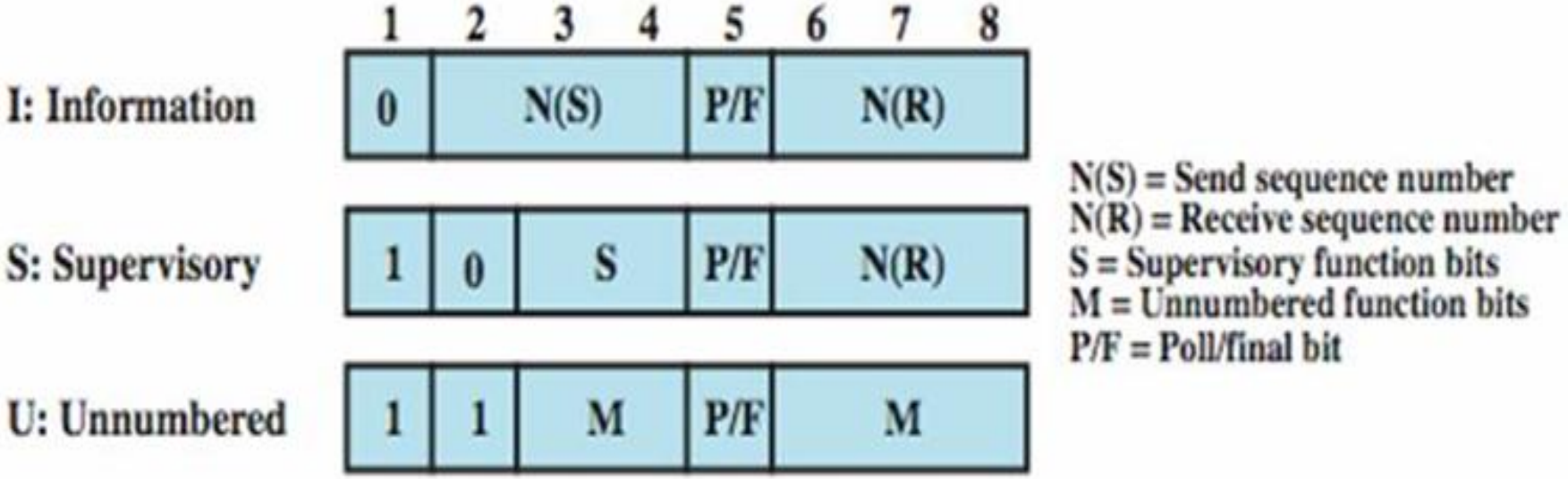
HDLC: Frame Types

- Information: data to be transmitted to next layer up
  - Flow and error control piggybacked on information frames 信息帧
- Supervisory: ACK or NAK when piggyback not used 监督帧
- Unnumbered: supplementary link control 无编号帧



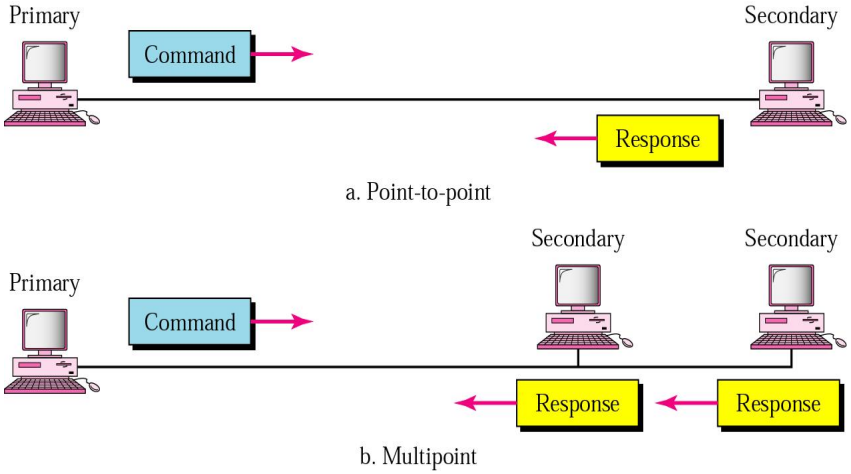
[Forouzan]

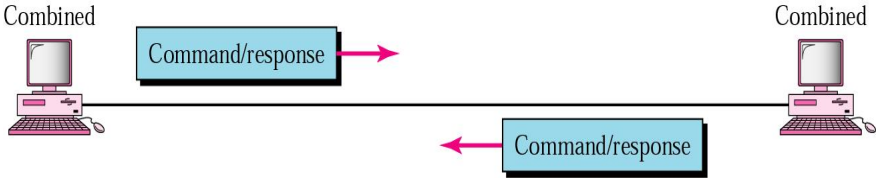
HDLC: Control Fields



HDLC Station Types

- Primary station
  - Controls operation of link
  - Frames issued are called commands
  - Maintains separate logical link to each secondary station
- Secondary station
  - Under control of primary station
  - Frames issued called responses
- Combined station
  - May issue commands and responses





Example of Point-to-point Data Link Protocols

- HDLC (High-Level Data Link Control)
- The Data Link Layer in the Internet
  - *PPP (Point-to-Point Protocol 点对点协议)*

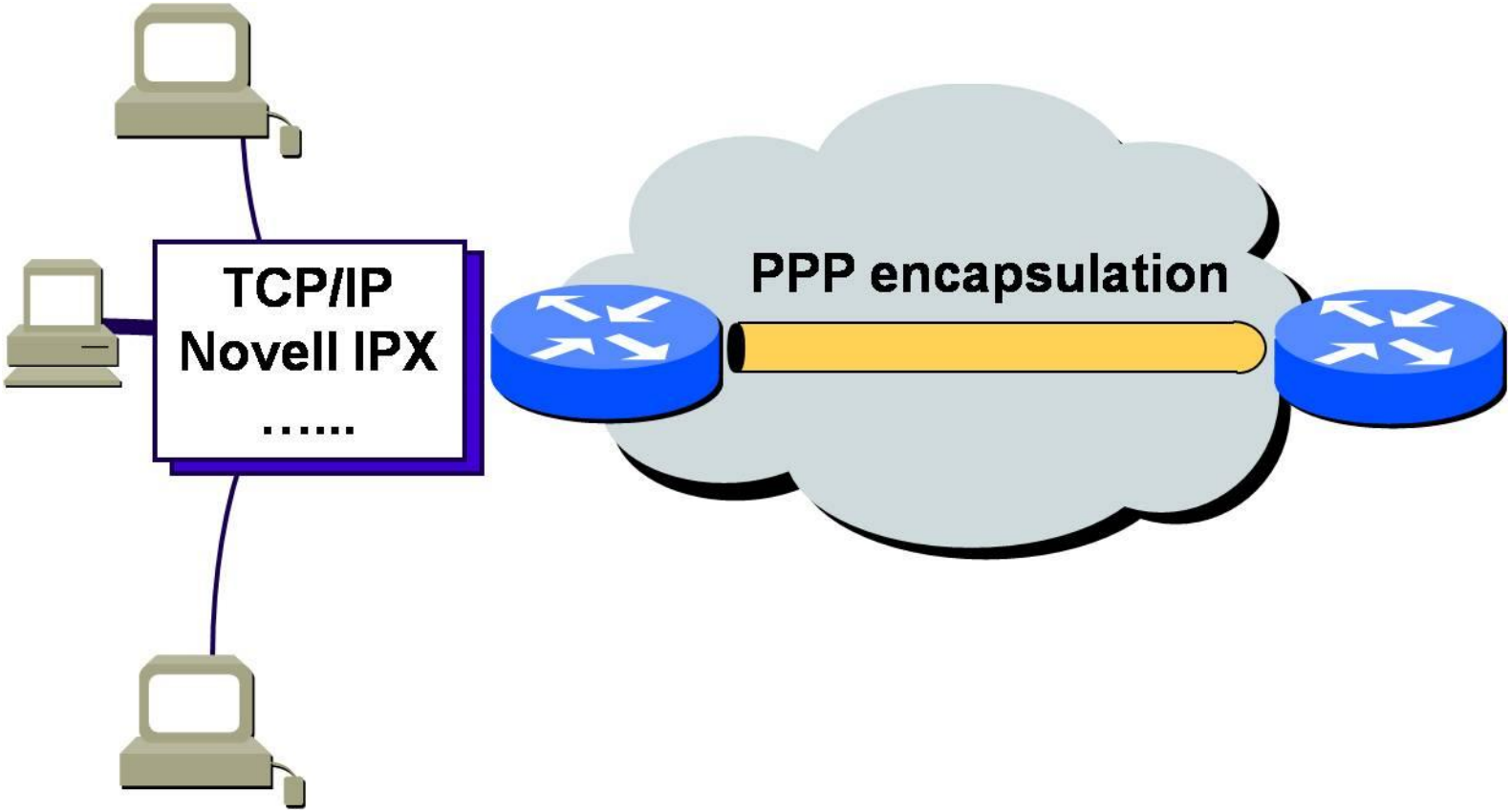
PPP vs. HDLC

- Point-point link vs. Multiple configurations
- Byte oriented vs. Bit oriented
  - Byte stuffing vs bit stuffing
- Connectionless unacknowledged service vs. Reliable connection-oriented service

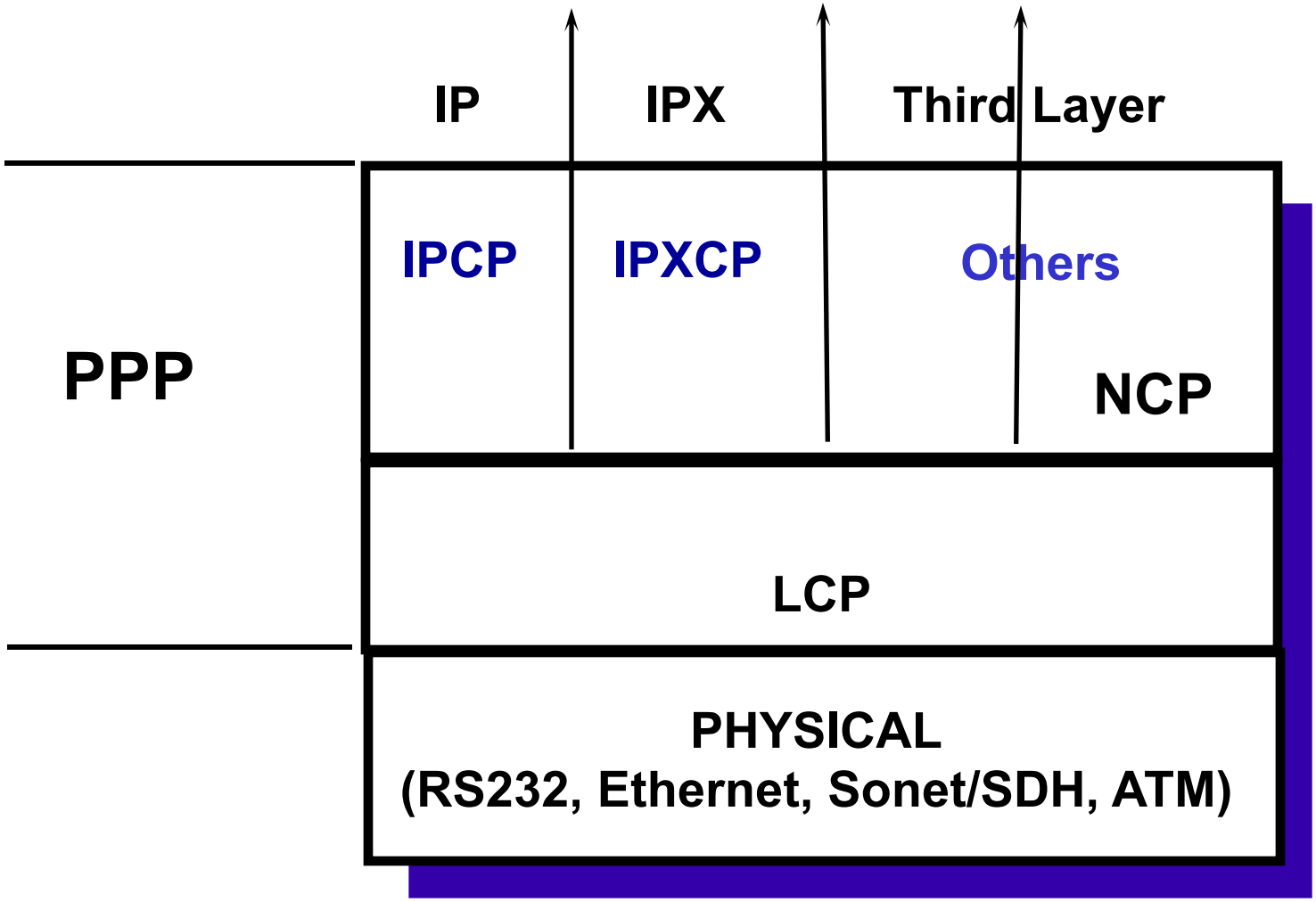
PPP compared with HDLC

- Simpler
- What are enhanced
  - Working with numerous network layer protocols
  - Negotiation of connections（dynamic IP address）
  - User authentication (身份认证)
  - Negotiation of data compression
  - Physical layer can be asynchronous or synchronous
- What are removed
  - Error correction
  - Flow control
  - Sequence number

PPP: Multiple network protocols



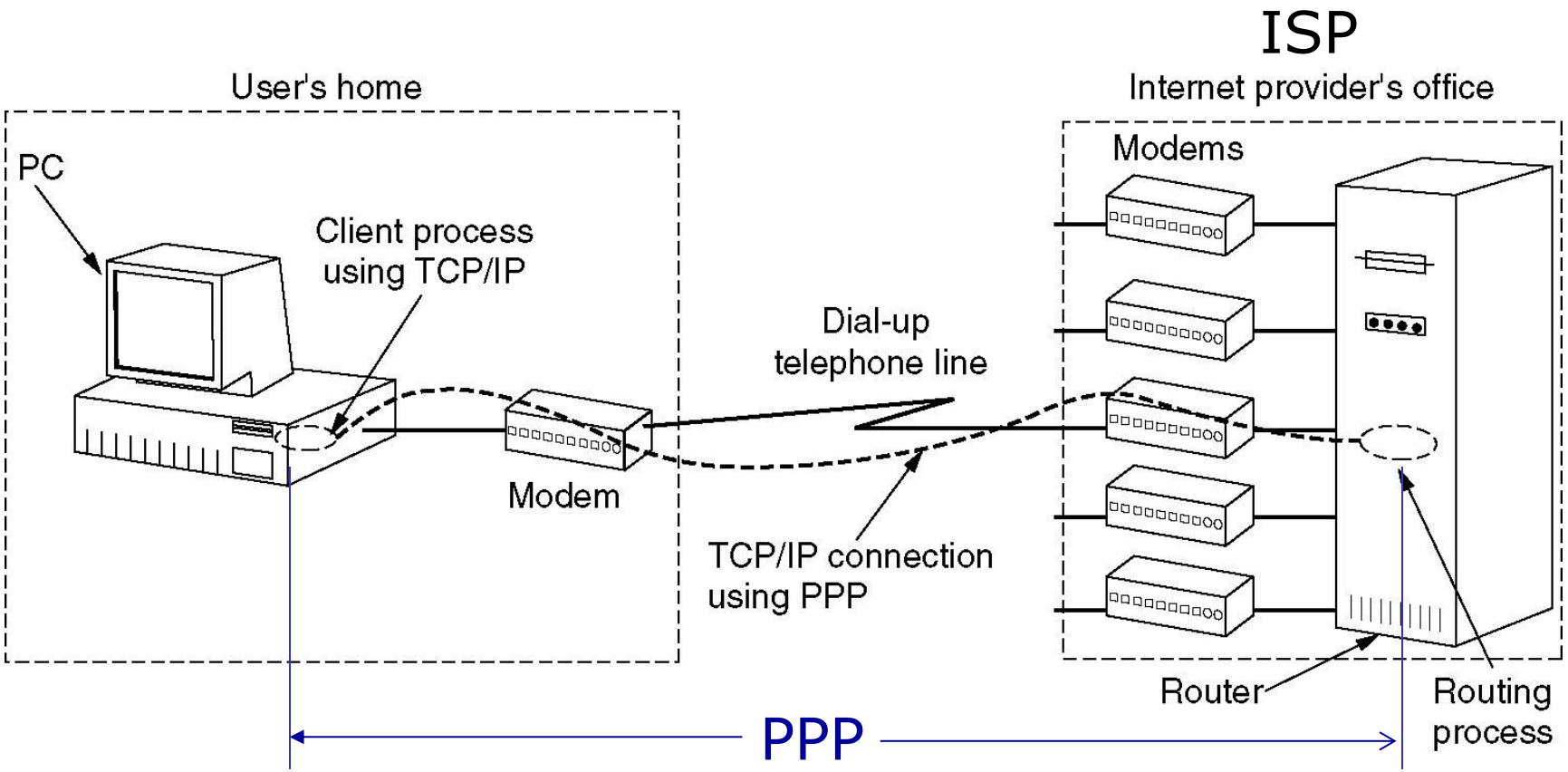
PPP: Sub-layers(1)



PPP: Sub Layers (2)

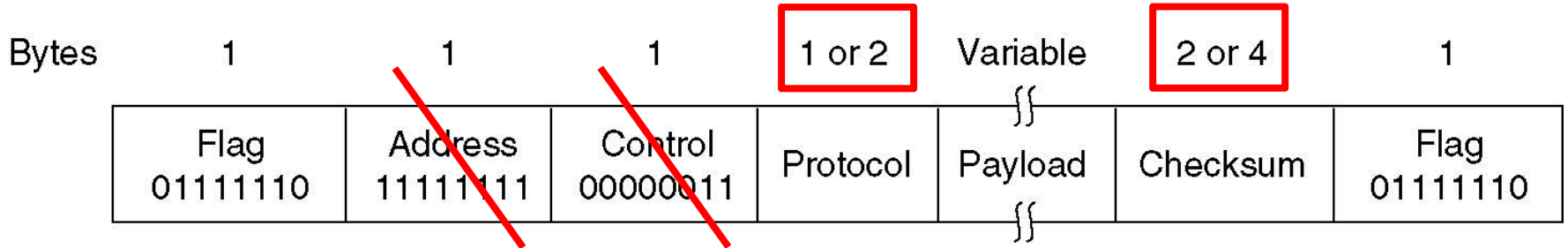
- Protocol specifications:
  - RFC1661,1662,1663
- Layer 1
  - Synchronous circuits (bit-oriented encodings, PPP over SONET)
  - Asynchronous circuits (byte-oriented encodings)
  - PPPoE (PPP over Ethernet, ADSL)
- LCP: Link Control Protocol
  - Error detection (CRC)
  - Permits authentication
- NCP: Network Control Protocol
  - negotiates network-layer options, such as IP address

PPP used in dial-up access to Internet



A home personal computer as an Internet host

PPP for PPPoE: Frame Format

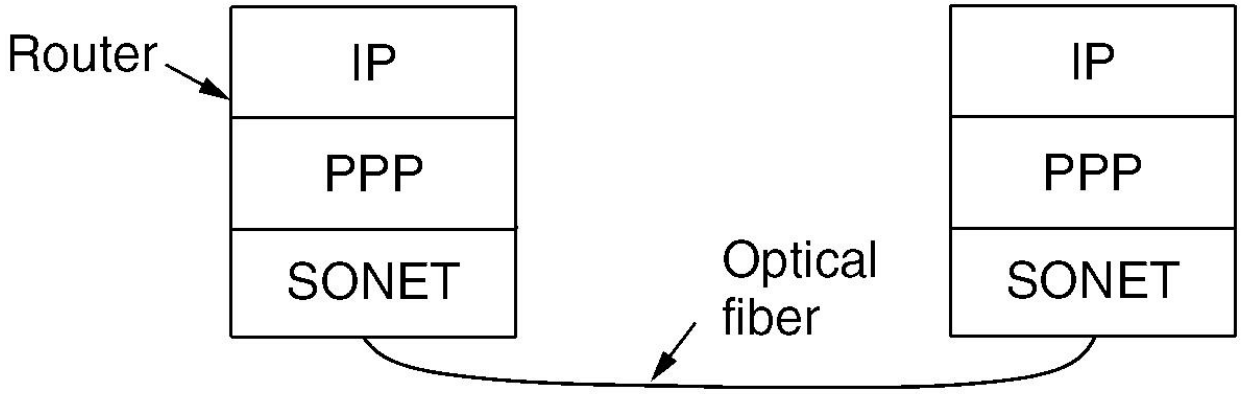


- Unnumbered mode operation 无序号
- Byte stuffing 字节填充
  - 0x7D as escape byte
  - Adding 0x7D before an ASCII control character (⩽0x20) : 0x03→ 0x7D 0x03
- *Address*: 0xFF indicates "all stations"
- *Control*: 0x03 indicates unnumbered mode
- *Protocol* : indicating type of payload, supporting LCP, NCP, IP, IPX, AppleTalk...
- *Checksum*: using CRC to check errors

Packet over SONET

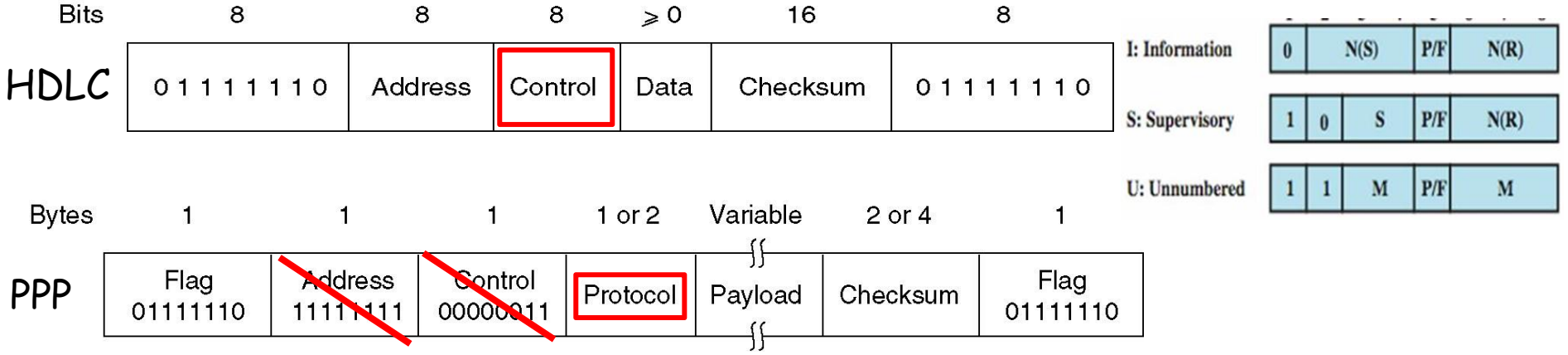
SONET

- the physical layer protocol that is used over **the wide-area optical fiber links** that make up the backbone of communications networks, including the telephone system.
- provides a **bitstream** that runs at a well-defined rate, for example 2.4 Gbps for an OC-48 link, some **framing mechanism** is needed.



Summary：HDLC vs PPP

	HDLC	PPP
Applications	WAN, Multiple configurations	WAN, Point-point link, dial-up access
service	Reliable connection-oriented	Unacknowledged connectionless
Phy	Synchronous serial transmission	PPPoE, asynchronous or synchronous
PDU	Bit oriented	Byte oriented
framing	bit stuffing	byte stuffing
DEC	CRC	CRC
Functions	error control+flow control GoBackN ARQ, SR ARQ	numerous network layer protocols negotiation parameters, authentication



Key points of Data Link Layer

- Functions and services of data link layer
- Framing method
  - Byte stuffing
  - Bit stuffing
  - Physical layer coding violations
- Error Control
  - Correcting Code: Hamming code
  - Error Detecting Code: CRC
- Flow control: ARQ
  - Stop-and-wait: $W_T=W_R=1$, $U=1/(1+2a)$
  - Go-back-N: $1<W_T<2^m$, $W_R=1$
  - Selective Repeat: $W_T=W_R=2^{m-1}$

$$\longrightarrow \quad U=\begin{cases} 1 & W \geq 2a+1 \\ \dfrac{W}{2a+1} & W < 2a+1 \end{cases}$$

- Practical protocols
  - HDLC: Usage, frame types
  - PPP: Usage, frame structures

Terms

缩写	英文全称	中文
	access control	访问(接入)控制
ACK	Acknowledgement	确认(帧)
	addressing	寻址
ARQ	Automatic Repeat reQuest	自动请求重发
ATM	Asynchronous Transfer Mode	异步传输模式
	bandwidth-delay product	带宽时延积
BER	Bit Error Rate	误码率
	bit rate	数据率(比特率)
	bit stuffing	(零)比特填充
	byte stuffing	字节填充
	burst error	突发差错
	carrier	载波
	character count	字符计数法
	Character Stuffing	字符填充
	codeword	码字

缩写	英文全称	中文
	cumulative acknowledgement	累积ACK
CRC	Cyclic Redundancy Check	循环冗余校验
	data link layer	数据链路层
	dial-up	拨号
	encapsulation	封装
	error control	差错控制
	error syndrome	差错综合症
	error correction	差错纠正
	Error Correcting Code	纠错码
	error detection	差错检测
	Ethernet	以太网
	flag byte	标志字节
FCS	Frame Check Sequence	帧校验序列
FDM	Frequency Division Multiplexing	频分复用

缩写	英文全称	中文
	flow control	流量控制
FEC	Forward Error Correction	前向纠错
	Feedback-based flow control	基于反馈的流量控制
	frame	帧
	framing	成帧
	frame header	帧头
	full-duplex	全双工
	Generator polynomial	生成多项式
	Go-Back-N	回退N步(协议)
	Hamming code	汉明(海明)码
	hamming distance	汉明距离
	HUB	集线器
LAN	Local Area Network	局域网
LAP	Link Access Procedure	链路接入规程

缩写	英文全称	中文
LAPB	Link Access Procedure – Balanced mode	链路接入规程——平衡模式
LAPD	Link Access Procedure, on D channel	D信道上的链路接入规程
LCP	Link Control Protocol	链路控制协议
	Line Utilization	线路利用率
LLC	Logical Link Control	逻辑链路控制
MAC	Media Access Control	媒体(介质)访问控制
	Manchester Encoding	曼彻斯特编码
HDLC	High-Level Data Link Control	高级数据链路控制
	hop	跳
	hop by hop	逐跳
NAK	Negative acknowledgement	否认(帧)
NCP	Network Control Protocol	网络控制协议
NIC	Network Interface Card	网络接口卡
	Odd & Even Parity	奇偶校验

缩写	英文全称	中文
PAR	Positive Acknowledgement with Retransmission	带确认的重传
	packet	分组/包
	Physical layer coding violations	物理层编码违例法
	Packet Switching	分组交换
	Parity check	奇偶校验
	payload	净荷（数据）
	piggybacking	捎带确认(应答)
	pipelining	流水线
PPP	Point-to-Point Protocol	点对点协议
PPPoE	PPP over Ethernet	以太网之上的点对点协议
	Polynomial Code	多项式编码
	preamble	前导码
	propagation delay	传播时延
	rate-based flow control	基于速率的流量控制
	round-trip propagation time	环回传播时间

缩写	英文全称	中文
SLIP	Serial Line IP	串行线IP
SONET	Synchronous Optical Network	同步光网络
	Selective Repeat	选择重传(协议)
	sliding window	滑动窗口
	Stop-and-wait	停止等待(协议)
	store-and-forward switching	存储转发交换
	subnet	子网
	switch	交换机
	switched Ethernet	交换式以太网
TDM	Time Division Multiplexing	时分复用
	Timeout Timer	超时定时器
	transparent transmission	透明传输
	transmission delay	发送时延
	transport layer	传输层
WAN	Wide Area Network	广域网
WLAN	Wireless Local Area Network	无线局域网

Copyright Information

- Some figures used in this presentation have been either directly copied or adapted from following materials
  - Lecture notes of book "Computer Networks, a Top-down Approach", J.Kurose and W.Ross, 3rd Edition, which are marked with *[Kurose]*
  - "Data communications and networking", B.A.Forouzan, which are marked with *[Forouzan]*
  - "Data and Computer Communications", William Stallings, which are marked with *[Stallings]*
  - "计算机网络", 谢希仁，第五版，电子工业出版社，2008年，which are marked with *[谢]*

偶校验

海明编码解码—简便法

例： 数据 = 1011，

b1	b2	b3	b4	b5	b6	b7
P1	P2	1	P3	0	1	1

编码简便法：将码字中为1的各位码字位号表示为二进制码，再按模2求和，所得结果就是校验码。

b3 = 011
b6 = 110
⊕ b7 = 111
P3P2P1 = 010

发送的码字：

b1	b2	b3	b4	b5	b6	b7
0	1	1	0	0	1	1

收到的码字：

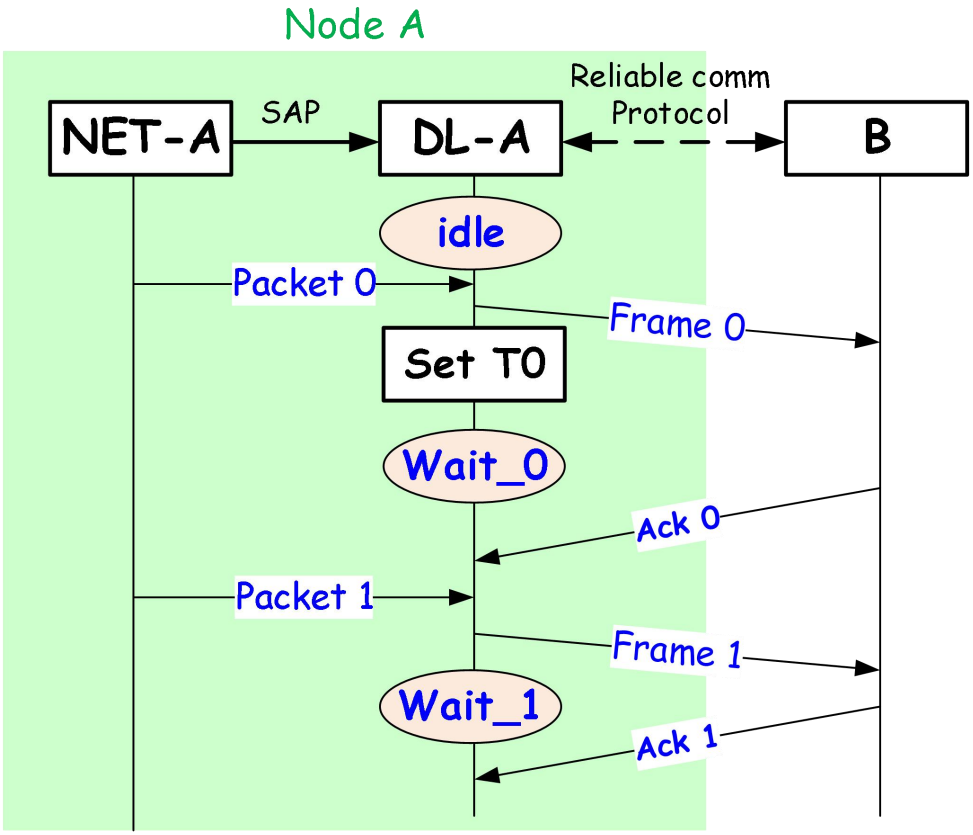
b1	b2	b3	b4	b5	b6	b7
0	1	1	0	0	1	0

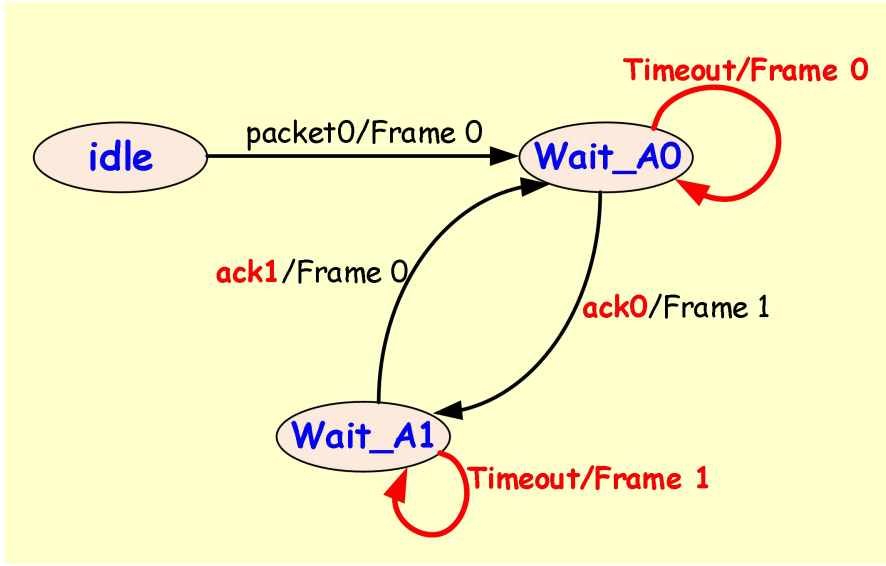
差错位

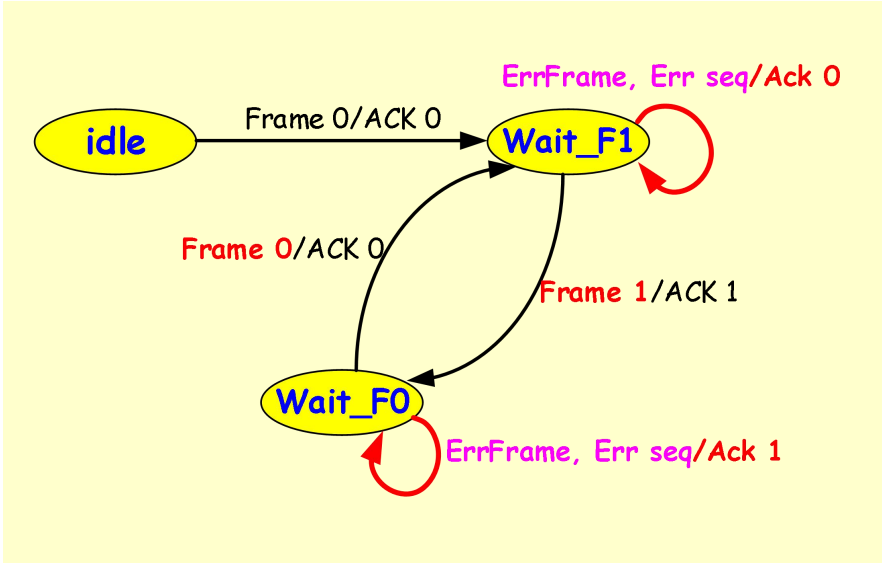
解码简便法：将码字中为1的各位码字位号表示为二进制码，再按模2求和，若和为0，则无差错。若和不为0，则指明差错的位号。

b2 = 010
b3 = 011
⊕ b6 = 110
S4S2S1 = 111

讨论


Node A
NET-A
SAP
DL-A
Reliable comm Protocol
B
idle
Packet 0
Frame 0
Set T0
Wait_0
Ack 0
Packet 1
Frame 1
Wait_1
Ack 1


idle
packet0/Frame 0
Wait_A0
Timeout/Frame 0
ack1/Frame 0
ack0/Frame 1
Wait_A1
Timeout/Frame 1


idle
Frame 0/ACK 0
Wait_F1
ErrFrame, Err seq/Ack 0
Frame 0/ACK 0
Frame 1/ACK 1
Wait_F0
ErrFrame, Err seq/Ack 1